



Synthetic Price Path Generation for European Gas Markets Using Generative Adversarial Networks with Applications to Option Pricing

Yeva Galstyan

Master's Thesis in Global Software Development (M.Sc.)
Department of Applied Computer Science
Fulda University of Applied Sciences

Matriculation number: 1507481
Supervisor: Prof. Dr. Christoph Scheich
Co-supervisor: Dr. Alexander Jungwirth

Submitted on September 26, 2026

Abstract

Since the crisis of 2021–2022, the European natural gas market has experienced larger supply disruptions than ever before, and these disruptions can cause extreme price volatility. Widely used option pricing models such as Black-76 assume log returns that are normally distributed with constant volatility. However, observed gas prices show heavy tails, volatility clustering, and asymmetry, and these stylized facts are not consistent with the assumptions of Black-76. Generative adversarial networks (GANs) learn the distribution of the data directly from observations. This thesis applies the conditional GAN frameworks ForGAN and Fin-GAN to the daily Dutch Title Transfer Facility (TTF) futures returns. A grid of ten cost function variants and eleven network and training configurations is evaluated on three stylized facts. The selected model is the one closest to the test split among the configurations that remain stable across seeds. European options are priced by Monte Carlo simulation. The resulting implied volatilities are compared with the market quotes and with two benchmark models in three variants. The selected model reproduces the stylized facts in part, but the option prices it generates stay below the market and below both benchmarks. The spread of the simulated returns correlates 0.758 with the realized volatility in the condition window. No benchmark responds to the returns preceding the valuation date. The conditioning works, while the level of the generated volatility remains for future work.

Acknowledgements

I would like to thank Prof. Dr. Christoph Scheich for supervising this thesis and for his guidance throughout the work. I also thank Dr. Alexander Jungwirth for co-supervising this thesis and for his detailed feedback on the draft. Special thanks go to Joel Val, who helped shape the topic, pointed me to the Fin-GAN paper, and made the access to the TTF futures and options data possible. Finally, I thank my family and my partner for their support during the writing of this thesis.

Contents

Acknowledgements	III
Contents	IV
List of Figures	VI
List of Tables	VII
List of Symbols	VIII
List of Abbreviations	X
1 Introduction	1
1.1 Problem Statement and Motivation	1
1.2 Research Questions	2
1.3 Own Contribution	3
1.4 Structure	3
2 Financial Fundamentals	4
2.1 Financial Markets	4
2.2 European Natural Gas Markets	5
2.3 Futures Contracts	6
2.4 Options	7
2.5 Implied Volatility	8
2.6 Stylized Facts of Asset Returns	10
3 Machine Learning Fundamentals	12
3.1 Neural Networks	12
3.1.1 Training	12
3.1.2 Recurrent Neural Networks and LSTM	14
3.2 Generative Adversarial Networks	15
3.2.1 Basic Generative Adversarial Networks	15
3.2.2 Conditional GANs	17
3.2.3 ForGAN	18
3.2.4 Fin-GAN	19
3.3 Related Work	21
4 Methodology and Implementation	23
4.1 Data Pipeline	23
4.1.1 Futures Data	23
4.1.2 Front-Month Series Construction	24

4.1.3	Return Definition	25
4.1.4	Data Split	25
4.1.5	Options Data	26
4.1.6	Options Panel Construction	27
4.2	Architecture, Setup, and Adaptations	28
4.2.1	Model Architecture	28
4.2.2	Training Configuration	29
4.2.3	Adaptations	29
4.3	Experimental Design	30
4.4	Evaluation Methodology	31
4.5	Option Pricing Setup	32
4.6	Benchmark Models	34
4.7	Software Environment	36
4.8	Summary of the Setup	37
5	Experiments	38
5.1	Properties of the TTF Return Series	38
5.2	ForGAN Baseline: Training Length	41
5.3	ForGAN Baseline: Network Capacity	43
5.4	ForGAN Baseline: Condition Window and Input Scaling	48
5.5	Fin-GAN Branches: Effect of the Cost Function Terms	49
5.6	Model Selection	53
5.7	Option Prices	54
5.8	Summary of the Results	58
6	Discussion	60
6.1	Sources of the Deviation	60
6.2	Limitations	61
7	Conclusion and Future Work	62
7.1	Conclusion	62
7.2	Future Work	63
A	Full Experimental Grid	68
	Documentation of AI Tool Usage	80
	Declaration of Independence	81

List of Figures

1	Structure of a standard GAN	16
2	Alternating update cycle of a GAN	17
3	Structure of a cGAN	18
4	Structure of ForGAN	19
5	TTF train, validation, and test data split	26
6	Condition window during the rollout	32
7	Pricing procedure for one valuation date	33
8	Daily TTF log returns by split	39
9	Autocorrelation of squared returns by configuration	46
10	Skewness and maximum, the ten cost function branches	52
11	Implied volatility against moneyness on three valuation dates	56
12	Annualized volatility on each valuation date	57

List of Tables

1	Model configurations	30
2	Stylized facts by construction	36
3	Software packages and versions	36
4	Evaluated models	38
5	TTF log returns, full sample and per data split	39
6	Test split statistics with and without the 2026 disruption	40
7	ForGAN baseline, configurations 1 and 2	41
8	MSE branch, configurations 1 and 2	43
9	ForGAN baseline, configurations 2, 4, and 10	44
10	ForGAN baseline, configurations 3, 5, and 8	44
11	ForGAN baseline, configurations 7 and 9	47
12	ForGAN baseline, configuration 6	48
13	ForGAN baseline, configuration 5 with reference and full-split scaling	49
14	Single cost function terms, configuration 8	50
15	Combinations without SR^* , configuration 8	51
16	Combinations with SR^* , configuration 8	51
17	Implied volatility error by model	55
18	Implied volatility error by moneyness	55
19	Implied volatility error of G by moneyness	56
20	Option pricing results per seed	58

List of Symbols

Symbol	Meaning
<i>Financial fundamentals</i>	
μ	Mean return over a period
σ	Volatility of the futures price
Φ	Cumulative distribution function of the standard normal
C, P	European call and put price
$C_2(\tau)$	Autocorrelation of squared returns at lag τ
d_1, d_2	Arguments of Φ in Black-76
$\hat{\mathbb{E}}$	Expectation in a risk-neutral world
$f(x)$	Probability density of the normal distribution
F_t	Futures settlement price on trading day t
F_T	Futures settlement price at expiry
K	Strike price; agreed price of a futures contract
$\mathcal{N}(\mu, \sigma^2)$	Normal distribution with mean μ and variance σ^2
P_t	Front-month settlement price on trading day t
$P(x > u)$	Fraction of returns exceeding u in absolute value
r	Risk-free rate
r_f	Risk-free rate in the Sharpe ratio
T	Time to maturity
x_t	Log return on trading day t
<i>Neural networks and GANs</i>	
$\theta, \theta_g, \theta_d$	Learnable parameters
c	Condition vector
d	Number of parameters in θ
G, D	Generator and discriminator
$J^{(G)}, J^{(D)}$	Cost functions of G and D
k	Discriminator updates per generator update
$p_z, p_{\text{data}}, p_g$	Noise, data, and generated distributions
W, b	Weight matrix and bias vector
z	Noise vector
<i>Fin-GAN cost function</i>	
$\alpha, \beta, \gamma, \delta$	Gradient-matching weights
ϵ	Mode collapse threshold
k_{tanh}	Sharpness of the tanh approximation
MSE	Mean squared error term
n_{batch}	Number of forecasts in a minibatch
$p_i, \bar{p}$	PnL_a^* of forecast i , and its minibatch mean
PnL_a^*	Smooth profit and loss term

Symbol	Meaning
SR^*	Sharpe ratio term of the cost function
STD	Standard deviation term
$wPnL, SR_w$	Weighted strategy profit and Sharpe ratio
$\hat{x}_i$	Forecast of the realized value x_i
<i>Experimental setup and evaluation</i>	
B	Draws per condition
l	Condition window length
m	Mean set of point forecasts
n_{cond}	Rows in the window matrix
$n_{\text{grad}}, n_{\text{epochs}}$	Gradient-matching and training epochs
N	Noise dimension
R_G, R_D	Generator and discriminator hidden size
$\hat{x}_{\text{path}}, \hat{x}_{\text{pool}}$	Path and pooled evaluation sets
$\hat{X}$	Matrix of generated draws
<i>Option pricing</i>	
Δ	Time step, T/h
ε_k	Standard normal draw on step k
$\hat{F}_T$	Simulated settlement price at expiry
$\tilde{F}_T$	Simulated settlement price after the correction
h	Trading days from the valuation date to expiry
M	Simulated paths per valuation date
$\hat{x}_k$	Simulated return on step k

List of Abbreviations

Abbreviation	Meaning
AI	Artificial intelligence
BBO	Best bid and offer
cGAN	Conditional generative adversarial network
CPU	Central processing unit
€STR	Euro short-term rate
ETF	Exchange-traded fund
GAN	Generative adversarial network
GARCH	Generalized autoregressive conditional heteroskedasticity
GBM	Geometric Brownian motion
GRU	Gated recurrent unit
GTS	Gasunie Transport Services
ICE	Intercontinental Exchange
iid	Independent and identically distributed
LNG	Liquefied natural gas
LSTM	Long short-term memory
MCM	Market Correction Mechanism
MSE	Mean squared error
OHLCV	Open, high, low, close, volume
PnL	Profit and loss
ReLU	Rectified linear unit
RMSProp	Root mean square propagation
SD	Standard deviation
SGD	Stochastic gradient descent
TTF	Title Transfer Facility
VAR	Vector autoregression

1 Introduction

Changes in the natural gas price affect macroeconomic conditions and the costs of individual companies and households. Since transmission capacity cannot be expanded in the short term, supply responds slowly, and imbalances are resolved primarily through price changes. A price rise reduces consumption until demand matches the available supply. Consequently, the gas price is sensitive to weather conditions, industrial activity, and the prices of coal and emission allowances. The strongest price movements follow supply disruptions. In 2025, the import dependency of the European Union for natural gas reached 87.6 % (Eurostat 2026). This dependency exposes the European market to external supply shocks. The most severe of these shocks occurred in 2021 and 2022, when demand recovered after the COVID-19 pandemic and Russian pipeline flows declined. The 2026 disruption is the most recent example.¹ Such shocks create price risk for all participants in the gas market. In this context, the question of how to model gas price movements has become increasingly relevant.

This chapter introduces the problem of generating realistic natural gas price paths for option pricing. The difficulties of modeling gas price movements are explained first. GANs are then introduced as a data-driven approach to this problem. On this basis, the research questions are formulated, followed by the contribution of the thesis. Finally, the structure of the remaining chapters is outlined.

1.1 Problem Statement and Motivation

In Europe, natural gas is traded at virtual hubs, of which the TTF is the most liquid (Heather 2026). Futures and options on the TTF are traded on ICE Endex. Physical and financial market participants can use these contracts to limit their exposure to price volatility (Geman 2005, p. 241). To quote these contracts, value their positions, and manage risk, market participants rely on pricing models. The price of an option is its expected payoff in a risk-neutral world, discounted at the risk-free rate (Hull 2022, p. 292). For this reason, pricing models rest on an assumption about the distribution of the futures price at expiry. A widely used model for pricing options on futures is Black-76 (Black 1976). It assumes that the futures price follows a geometric Brownian motion (GBM), under which log returns are normally distributed with constant volatility.

Natural gas futures prices do not match this assumption. They exhibit seasonality, revert to a long-term level, and show sudden price spikes (Geman 2005, p. 244). Hence, the implied volatility² of options on natural gas is not constant across strike prices, as it would be under the assumptions of Black-76. This pattern, known as the volatility smile, is skewed to the right, since the market is more sensitive to price rises (Geman 2005, pp. 101–105).

¹Both disruptions are discussed in Sec. 2.2.

²Implied volatility and the volatility smile are defined in Sec. 2.5.

Parametric stochastic models add features of observed prices to the GBM. The Merton jump-diffusion model extends the GBM with a jump process, which allows for sudden large price changes (Merton 1976). Commodity models add mean reversion (Schwartz 1997) and the convenience yield, the value of holding the physical commodity. The convenience yield moves around a long-term level (Gibson and Schwartz 1990). The GARCH model targets the constant volatility assumption by letting the volatility depend on the size of the past returns (Bollerslev 1986). Fitting a GARCH model requires choosing the model order and the distribution of the random shocks in advance. The accuracy of these models depends on whether the assumed form matches the actual behavior of the underlying asset.

In contrast, machine learning models do not require specifying the functional form of the price process in advance. A large literature applies neural networks to option pricing, using characteristics such as moneyness and time to expiry to reproduce observed prices (Ruf and Wang 2020). These models do not simulate the path of the underlying futures price. Generative models address this limitation. GANs, introduced for image generation³, learn the distribution of the data by training two competing networks, a generator that produces samples and a discriminator that judges them (Goodfellow, Pouget-Abadie, et al. 2014). Conditional GANs (cGANs) extend GANs by providing a condition for controlling the generated samples (Mirza and Osindero 2014). ForGAN is a cGAN for time series forecasting, where the condition is the sequence of past observations (Koochali et al. 2019). Fin-GAN extends ForGAN by rewarding correct directional forecasts during training and was evaluated on equity returns (Vuletić, Prenzel, et al. 2024). Returns generated by such a model can be compounded into price paths, and the option price is estimated as the average discounted payoff over these paths.

GAN architectures for financial time series (Yoon et al. 2019; Wiese, Knobloch, et al. 2020; Ni et al. 2020; Zhang et al. 2026) and GAN-based simulators of option markets (Wiese, Bai, et al. 2019; Vuletić and Cont 2024) have been developed and evaluated on equity data. In energy markets, GANs have been applied to scenario simulation (Walter and Wagner 2024; Korn and Ramadani 2025). For natural gas, GANs have been used to estimate crisis-risk indicators (Bohórquez Correa et al. 2026). None of these studies generates price paths and evaluates them through option prices. Results on equity data cannot be assumed to transfer to natural gas. Whether GAN-generated returns reproduce the properties of TTF futures returns, and whether the derived option prices match the market, remain open.

1.2 Research Questions

This thesis examines whether GANs can generate TTF futures returns with the stylized facts of the observed data (RQ1). These include heavy tails, volatility clustering, and asymmetry. In addition, it assesses whether price paths generated by recursive rollout reproduce the market prices of TTF options (RQ2). A positive answer would provide a

³The GAN architecture is explained further in Sec. 3.2.

model that conditions on the market state without assuming a parametric form for the return distribution.

1.3 Own Contribution

This thesis applies ForGAN and Fin-GAN to daily TTF front-month futures returns. The generated returns are compared with the test data across training lengths, hidden sizes of the generator and discriminator, condition window lengths, and input scaling methods. Each configuration is trained with five random seeds and reported as mean and seed spread. The cost function terms of Fin-GAN are compared under the configuration closest to the test split among those that remain stable across seeds. The branch with the smallest seed spread is selected. The selected model is used to generate price paths by recursive rollout. The paths are adjusted to the risk-neutral world with a correction adapted from Duan and Simonato (1995). From these paths, call options are priced by Monte Carlo simulation on 20 valuation dates, each about one month before a monthly expiry. The resulting prices are converted into implied volatilities and compared with the market implied volatilities across moneyness ranges. GBM and two bootstrap modes serve as benchmarks.

1.4 Structure

Chapter 2 introduces the European gas market, options, and implied volatility. The machine learning fundamentals, the GAN frameworks, and the related work follow in Chapter 3. Chapter 4 describes the data, the model architecture, the option pricing procedure, and the benchmark models. Chapter 5 compares the statistical properties of the generated returns across the model configurations and cost function terms. The selected model is then used to price options. The results and the limitations are discussed in Chapter 6. Finally, Chapter 7 summarizes the results and gives an outlook.

2 Financial Fundamentals

This chapter provides the financial background for the remainder of the thesis. It begins with general concepts of financial markets, returns, and risk, and contrasts equity with commodities as asset classes (Sec. 2.1). The chapter then focuses on the European natural gas market and the TTF hub (Sec. 2.2). The futures and option contracts traded on it are introduced next (Secs. 2.3 and 2.4), together with implied volatility (Sec. 2.5). Finally, the stylized facts of asset returns relevant for this thesis are described (Sec. 2.6).

2.1 Financial Markets

A financial market is an organized venue in which participants trade financial assets. A financial asset is a claim to future value. An exchange is a market that operates under standardized rules, which specify what may be traded and how.

Buyers need to find sellers that are willing to sell the asset at a low price, while sellers need to find buyers that are willing to pay a high price. The bid is the highest price at which a participant is willing to buy an asset, and the ask is the lowest price at which a participant is willing to sell it. A transaction occurs when the two sides agree on a price. As a result, the observed price of an asset is not a fixed property but rather the outcome of the most recent transaction. A market is considered liquid if many participants can buy or sell continuously, without affecting the asset price (Harris 2003, pp. 3–7).

The return of an asset measures the relative change in its price over a given time period. The risk of an asset is the uncertainty of its return. It is generally measured with the standard deviation of its return, which is commonly referred to as volatility. The risk-free rate is the return of an investment that carries no risk, such as a short-term government bond. The Sharpe ratio is a measure of the risk-adjusted return of an asset

$$SR = \frac{\mu - r_f}{\sigma}, \quad (1)$$

where μ is the mean return over the period, r_f is the risk-free rate, and σ is the standard deviation of the returns. SR is commonly annualized.

Hull (2022, p. 33) differentiates three main categories of traders. Hedgers trade to reduce the risk they are exposed to from future price movements. Speculators trade to take a position on the future direction of the price. Lastly, arbitrageurs combine long and short positions in two or more instruments, so that a profit is guaranteed regardless of the price movements. A participant who holds a long position benefits from a price increase, and a participant who holds a short position benefits from a price decrease.

Assets are grouped into asset classes according to their characteristics. Equity and commodities are two asset classes that are relevant for this thesis. Equity represents ownership in a company. The holder of equity is entitled to a share of the profits of the company, so the value of equity depends on the profits expected in the future. Shares are traded on stock exchanges, financial markets that specialize in equity. Commodities are physical goods that are produced and consumed. The supply of a commodity is

determined by production and by the inventories that are held. Demand is generally inelastic, since the good is essential and has few substitutes in the short run. As a result, commodity prices are sensitive to physical conditions such as weather, storage levels, and available transport capacity.

As stated above, equity and commodities differ in how their prices are formed. They also differ in what a trade eventually leads to. A trade in equity transfers the ownership of shares, while a trade in a commodity can end in the delivery of the physical good, which the buyer has to receive and store. Lastly, the two differ in what their holders are exposed to. A holder of equity is affected only by the price of the shares, whereas a commodity producer is affected by the price of the commodity and also by the quantity it sells. For energy commodities, this quantity changes with the weather and with economic conditions (Geman 2005). The European natural gas market is the commodity market studied in this thesis.

2.2 European Natural Gas Markets

Natural gas is a physical commodity that exists within a gas grid. The TTF is a virtual trading hub in the Netherlands. A virtual trading hub allows participants to transfer ownership without reference to a specific physical location. The TTF is operated by Gasunie Transport Services (GTS) within the Dutch gas transmission network. Gas futures contracts referencing the TTF are traded on ICE Endex, the European energy exchange operated by Intercontinental Exchange. Futures contracts are described in Sec. 2.3.

The TTF is the most liquid natural gas market in Europe, with traded volumes 4.5 times the combined volume of the other eight main European hubs (Heather 2026). Therefore, the TTF gas price serves as the reference price for gas contracts.

At virtual hubs such as the TTF, gas price formation is determined by short-run supply and demand (Obadi and Korcek 2020). In the short run the infrastructure of the gas grid is fixed, and the supply of gas cannot expand in response to a change in demand. Therefore, the adjustment takes place through the price, rather than the quantity.

Gas demand originates in three sectors. The first source is residential and commercial heating, which makes consumption seasonal. Deviations of weather from the seasonal norm also affect prices (Brown and Yücel 2008). Industrial production is the second source of demand. In the chemical sector, for example, gas is used as an energy source and as a raw material. The third source is power generation, where gas competes with other sources of electricity. The competition is strongest with coal, since power producers can switch between the two fuels depending on their prices. Coal emits more CO₂ than gas, so the price of emission allowances affects which of the two fuels is cheaper. As a result, the gas market is closely linked to the coal market and to the carbon market.

On the supply side, natural gas prices depend on pipeline flows and deliveries of liquefied natural gas (LNG). LNG is natural gas cooled to liquid form for transport by ship. As has been observed repeatedly in the past, the European natural gas market is highly sensitive to supply disruptions. The European gas price crisis between 2021 and 2022 is the most severe episode. During the COVID-19 pandemic in 2020, energy demand collapsed. This was followed by economies reopening, and gas demand recovered faster than production could follow. From mid-2021, the post-pandemic increase in demand was also combined with reduced Russian pipeline flows, strong Asian competition for LNG cargoes, and storage levels below the seasonal norm (Heather 2022). As a result, TTF gas prices rose from roughly 18 EUR/MWh in the first quarter of 2021 to a peak of 339 EUR/MWh on 26 August 2022 and then declined from early 2023.

The crisis eventually raised the question of whether the TTF price benchmark had become less representative of the underlying gas market. In November 2022, the European Commission proposed a Market Correction Mechanism (MCM), which would cap gas prices during crisis periods (European Commission 2022). The mechanism applied until January 2025 but was never activated, as gas prices declined from early 2023.

A more recent disruption followed the military campaign against Iran by Israel and the United States on 28 February 2026. Iran responded with attacks on shipping and energy infrastructure in the Gulf. Between 27 February and 2 March, the traffic through the Strait of Hormuz fell from 95 vessels to 4 vessels per day (Sharples and Farren-Price 2026). These events resulted in a spike of TTF gas prices from 27 EUR/MWh on 5 January 2026 to 61 EUR/MWh on 19 March 2026.

2.3 Futures Contracts

A derivative is a contract whose value depends on the price of another asset, called the underlying. Derivatives are traded on exchanges and also directly between two parties, which is referred to as over-the-counter trading. A forward contract is an agreement between two parties to buy or sell an asset at a fixed price on a specified future date. A futures contract is a standardized forward contract traded on an exchange (Hull 2022, pp. 28–30). The quantity and the delivery date are set by the exchange. The price is agreed between the parties and remains fixed for the contract. As a result, all contracts for a given month are identical in their terms and can be freely transferred. TTF gas futures are quoted in EUR/MWh and are physically delivered, with delivery spread evenly across the hours of the delivery month. Futures contracts are settled daily against a settlement price published by the exchange. The settlement price is determined by the exchange at the end of each trading day and is not necessarily a price at which a transaction occurred. The daily gain or loss is exchanged in cash between the two parties.

The payoff of a long position in a futures contract is the difference between the settlement price F_T at expiry T and the agreed price K . The payoff of a short position is

the difference between the agreed price K and the settlement price F_T . Both payoffs can be positive or negative (Hull 2022, p. 29).

Price formation of natural gas in Europe has generally taken place on the futures market rather than the spot market (Obadi and Korcek 2020). In a spot market, gas is traded for immediate delivery. A futures contract can be traded repeatedly before expiry. As a result, the market is open to hedgers and speculators without an interest in the physical commodity.

At any point in time, contracts for multiple consecutive months are listed and can be traded in parallel. The contract with the nearest delivery on a given day is the front-month contract, and the next is the second-month contract. Trading concentrates in the front-month contract. Hence, it is commonly used as the reference for the current gas price. Since each contract ceases trading before its delivery month, the contract that is the front month changes over time. The prices of the contracts sorted by delivery month form the term structure of the gas market. The price difference between two consecutive months is the calendar spread. A calendar spread describes the shape of the term structure.

2.4 Options

An option is a derivative that gives its holder the right, but not the obligation, to buy or sell an underlying asset at a fixed price. A call option is a right to buy the underlying asset, and a put option is a right to sell it. The fixed price is called the strike price K , and the date on which the right expires is the expiry date.

American options can be exercised at any point up to the expiry date. European options can only be exercised on the expiry date (Hull 2022, p. 31). TTF gas options traded on ICE Endex are European.

The payoff of an option is the value of the right at expiry. For a call option, the payoff is $\max(F_T - K, 0)$, where F_T is the settlement price at expiry. The right is exercised only if $F_T > K$. Otherwise, the option is left unused. For a put option, the payoff is $\max(K - F_T, 0)$. The payoff of an option is never negative (Hull 2022, p. 231). When the option is bought, the buyer pays the seller an amount called the premium. The premium is the price of an option.

The relation between the current price and K is described by three terms. A call option is in the money if $F_t > K$, at the money if $F_t = K$, and out of the money if $F_t < K$, where F_t is the settlement price on trading day t . For a put option, the definitions are reversed (Hull 2022, p. 248). The moneyness of an option is the ratio of the strike price to the current price, K/F_t , so a call is in the money when $K/F_t < 1$.

The settlement price at expiry is unknown when the option is bought. A risk-neutral world is a world in which investors do not require a higher expected return for taking on more risk. In a risk-neutral world, the expected return on any investment is the risk-free rate. Pricing an option under this assumption gives the correct price in the real world as well (Hull 2022, p. 292). Hull (2022, p. 407) shows that in a risk-neutral

world the expected settlement price at expiry is equal to the current settlement price, $\hat{\mathbb{E}}[F_T] = F_t$. As a result, the futures price is not expected to grow in a risk-neutral world.

Money available today is worth more than money available in the future, since money held today can be invested to earn a risk-free rate. Discounting is the process of converting a future value to its present value by applying the risk-free rate. The multiplier e^{-rT} is the discount factor, where r is the risk-free rate and T is the time to maturity. The price of a European call is its expected payoff in a risk-neutral world, discounted at the risk-free rate

$$C = e^{-rT} \hat{\mathbb{E}}[\max(F_T - K, 0)], \quad (2)$$

and the price of a European put is

$$P = e^{-rT} \hat{\mathbb{E}}[\max(K - F_T, 0)], \quad (3)$$

where $\hat{\mathbb{E}}$ denotes the expected value in a risk-neutral world. The option is valued at time zero, so that the expiry date and the time to maturity are both equal to T . The probabilities entering $\hat{\mathbb{E}}$ are those of a risk-neutral world and not those observed in the price series. The price of an option depends on the distribution of F_T , rather than on its expected value alone. The price of a call option has a lower and an upper bound. The option pays at least $F_T - K$ when it is exercised, so its price cannot fall below $\max(e^{-rT}(F_t - K), 0)$ (Hull 2022, p. 406). The option cannot pay more than F_T at expiry, so its price cannot exceed $e^{-rT}F_t$.

An option that is in the money would pay a positive amount at the current price, while an option that is out of the money would pay nothing. As a result, the value of an option depends on the probability that the price ends beyond the strike at expiry, and on how far beyond it ends.

$\hat{\mathbb{E}}$ in Eq. (2) and Eq. (3) cannot be evaluated in closed form unless the distribution of F_T is known. $\hat{\mathbb{E}}$ can instead be approximated by simulating many random paths for the underlying until expiry. The payoff is calculated at the end of each path, and the average of all payoffs is discounted at the risk-free rate to give the option price. The procedure is referred to as Monte Carlo simulation (Hull 2022, p. 490).

2.5 Implied Volatility

As described in Sec. 2.4, the price of an option depends on the distribution of F_T in the risk-neutral world. Black (1976) gives a closed-form solution for Eq. (2) and Eq. (3) under the assumption that F_T is lognormal, which corresponds to a price process known as GBM. The result is known as the Black-76 model. The model further assumes that σ and r are constant over the life of the option and that the option is European.

The lognormal distribution is defined through the normal distribution $\mathcal{N}(\mu, \sigma^2)$, where μ is the mean and σ^2 is the variance. A random variable X is normally distributed, $X \sim \mathcal{N}(\mu, \sigma^2)$, if its probability density is

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \quad (4)$$

where x is a value of the random variable X . The distribution is symmetric around μ and is fully described by μ and σ . For the standard normal distribution, $\mu = 0$ and $\sigma = 1$. The cumulative distribution function gives the probability that the variable takes a value less than or equal to a given threshold. For the standard normal distribution, the cumulative distribution function is

$$\Phi(x) = \int_{-\infty}^x \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right) du. \quad (5)$$

A random variable X is lognormally distributed if $\ln X$ is normally distributed. Under the lognormal assumption, the European call and put prices are given by

$$C = e^{-rT} [F_t \Phi(d_1) - K \Phi(d_2)], \quad (6)$$

$$P = e^{-rT} [K \Phi(-d_2) - F_t \Phi(-d_1)], \quad (7)$$

where F_t is the current futures price, K is the strike price, T is the time to maturity, r is the risk-free rate, and σ is the volatility of the futures price, as defined in Sec. 2.1. Eqs. (6) and (7) do not contain a growth term for F_t , since the futures price is not expected to grow in a risk-neutral world (Sec. 2.4). d_1 and d_2 are given by

$$d_1 = \frac{\ln(F_t/K) + \sigma^2 T/2}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}. \quad (8)$$

The parameters entering Eq. (6) and Eq. (7) are observable, except for σ . The risk-free rate r is not quoted with the option and is taken from a market interest rate. The implied volatility is the value of σ at which the model price matches the market price of the option. Eq. (6) and Eq. (7) cannot be solved for σ in closed form, since σ enters through Φ and appears in both d_1 and d_2 . As a result, the implied volatility is found numerically. The option price increases with σ , so exactly one value reproduces the observed price (Hull 2022, p. 358).

If the Black-76 model held, the implied volatility of options on the same futures contract with the same expiry would be constant across strikes. The plot of implied volatility against K is known as the volatility smile. In practice, the smile is not flat. The distribution of F_T implied by the market prices of options, referred to as the implied distribution, therefore departs from the lognormal (Geman 2005, pp. 101–102). The role of Black-76 here is the conversion of prices into implied volatilities. The lognormal assumption is not adopted for F_T .

In equity markets, the implied volatility decreases as the strike price increases. This pattern is referred to as the volatility skew. A falling stock price increases the leverage of the company and makes its equity riskier, so volatility rises when prices fall (Hull 2022, p. 456). In energy markets, the smile is skewed to the right. Supply disruptions cause sharp price rises (Sec. 2.2), and consumers of energy buy call options to protect themselves against these price spikes. The demand raises the prices of calls with high strikes. Consequently, their implied volatility rises as well. A right skew was found for crude oil, heating oil, gasoline, and natural gas (Geman 2005, p. 105).

2.6 Stylized Facts of Asset Returns

A diverse collection of assets, markets, and time periods shares common statistical characteristics of returns. These characteristics are referred to as stylized facts. Eleven stylized facts are summarized in Cont (2001). The price of an option depends on the distribution of F_T (Sec. 2.4). Three stylized facts describing the shape of the return distribution and its development over time are considered for TTF futures returns: heavy tails, volatility clustering, and asymmetry.

Heavy tails. The tails of a distribution are the regions far from its mean. Large and infrequent returns are located in the tails. The distribution of financial returns has more mass in the tails than a normal distribution (Cont 2001). This is referred to as heavy tails. The weight of the tails is measured by the excess kurtosis

$$\text{Kurt}_{\text{ex}}(X) = \mathbb{E} \left[\left(\frac{X - \mu}{\sigma} \right)^4 \right] - 3, \quad (9)$$

where μ and σ are the mean and the standard deviation of the random variable X . The excess kurtosis of a normal distribution is 0. The distribution is considered heavy-tailed if its excess kurtosis is positive. The tails of the return distribution determine the value of out-of-the-money options, which pay nothing unless a large move occurs. A volatility smile with higher implied volatility at strikes far from the current price, in both directions, corresponds to an implied distribution (Sec. 2.5) with heavier tails than the lognormal (Hull 2022, p. 453).

Volatility clustering. Large returns are more likely to be followed by large returns, and small returns by small returns (Cont 2001). This is measured with the autocorrelation. Autocorrelation is the correlation of a time series with itself, shifted by a fixed number of observations called the lag. The autocorrelation of returns at lag τ is

$$C(\tau) = \text{corr}(x_{t+\tau}, x_t), \quad (10)$$

where x_t is the return on trading day t . The autocorrelation lies between -1 and 1 . A value close to 0 means that past returns carry no linear information about future returns. The returns themselves show almost no autocorrelation, while absolute or squared re-

turns are positively correlated (Cont 2001). Squaring removes the sign of the return, so volatility clustering is measured on squared returns

$$C_2(\tau) = \text{corr}(x_{t+\tau}^2, x_t^2). \quad (11)$$

C_2 remains positive through squaring and decays slowly over several days. The distribution of F_T covers the life of the option rather than a single day. As the length of the period over which the returns are calculated increases, the distribution of returns approaches a normal distribution. This property is referred to as aggregational Gaussianity (Cont 2001). Volatility clustering slows the convergence to a normal distribution. As a result, the distribution of F_T keeps heavier tails than the lognormal over the life of the option.

Gain/loss asymmetry. The distribution of returns is asymmetric around its mean. Asymmetry is measured with the skewness

$$\text{Skew}(X) = \mathbb{E} \left[\left(\frac{X - \mu}{\sigma} \right)^3 \right], \quad (12)$$

where μ and σ are the mean and the standard deviation of the random variable X . The skewness of a normal distribution is 0. The skewness is positive when large positive returns are more likely than large negative returns, and negative in the opposite case. The distribution is considered asymmetric if its skewness is non-zero. A volatility smile that is higher on one side than the other corresponds to an implied distribution with a heavier tail on that side (Hull 2022, p. 456). The skewness of the implied distribution is therefore non-zero.

Eq. (12) and Eq. (9) are defined over a random variable X . In practice, the distribution of returns is unknown and only a sample of n observations $x_1, \dots, x_n$ is available. Therefore, the mean μ and the standard deviation σ are computed from the sample, and $\mathbb{E}[\cdot]$ becomes an average over the n observations. The resulting sample estimators of skewness and excess kurtosis are

$$g_1 = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^3}{\left[\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \right]^{3/2}} \quad \text{and} \quad g_2 = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^4}{\left[\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \right]^2} - 3, \quad (13)$$

where $\bar{x}$ is the mean of the n observations.

Mean reversion and seasonality are commodity-specific stylized facts (Geman 2005, pp. 64–68). Mean reversion is the tendency of a price to return to its long-term average after a period of deviation. Seasonality is the tendency of a price to follow a predictable pattern over the course of a year. These stylized facts describe the behavior of the price level over months and years, while this thesis models the distribution of daily returns.

3 Machine Learning Fundamentals

This chapter provides the machine learning background for the remainder of the thesis. It begins with the terminology of neural networks, their training, and recurrent architectures with the long short-term memory (LSTM) cell (Sec. 3.1). Afterwards, the generative adversarial network (GAN) is introduced, followed by its conditional, forecasting, and financial variants (Sec. 3.2). Finally, the related work on GANs for financial time series, option markets, and energy markets is reviewed (Sec. 3.3).

3.1 Neural Networks

Feedforward neural networks are a class of machine learning models that define a mapping $y = f(x; \theta)$, where x is the input, y is the output, and θ are the learnable parameters. The model is a composition of functions

$$f(x) = f^{(L)}(\dots f^{(2)}(f^{(1)}(x)) \dots), \quad (14)$$

where $f^{(L)}$ is the output layer, and L is the depth of the network. $f^{(1)}, f^{(2)}, \dots, f^{(L-1)}$ are referred to as the hidden layers. The dimensionality of the hidden layers is called their width. Computing $f(x)$ by evaluating the functions in order is called forward propagation (Goodfellow, Bengio, et al. 2016, p. 204).

Several types of layers exist, including fully connected, convolutional, and recurrent layers. Only fully connected layers, also called dense layers, and recurrent layers are used in this thesis. Recurrent layers are described in Sec. 3.1.2. In a dense layer, the input is multiplied by a weight matrix W , and a bias vector b is added. The operation is linear, so a composition of linear functions would again be linear. A nonlinear activation function g is applied to the output

$$f^{(i)}(x) = g(W^{(i)}x + b^{(i)}). \quad (15)$$

The weights and biases of all layers form the learnable parameters θ . The rectified linear unit (ReLU) and the sigmoid are two commonly used activation functions. ReLU is defined as $g(x) = \max(0, x)$, and the sigmoid is defined as $g(x) = 1/(1 + \exp(-x))$. The sigmoid is often used in the output layer for binary classification to map the output to $(0, 1)$. ReLU is often used in the hidden layers (Goodfellow, Bengio, et al. 2016, p. 174).

3.1.1 Training

Training is the process of finding parameters θ that fit the model to a set of examples, called training data. Learning problems are generally categorized as supervised and unsupervised. In supervised learning, the model learns a mapping from input to output using labeled data. Labeled data consists of input-output pairs, where the output is the target value. In unsupervised learning, the data is unlabeled, and the model learns the probability distribution that produced it.

The parameters θ must be chosen so that the model fits the training data. The cost function measures how badly the model performs on the training data. In supervised learning, it measures the difference between the model output and the target value. In unsupervised learning, it measures how poorly the distribution of the model matches the data. Training minimizes the cost function with respect to θ .

Classification is a supervised learning problem where the model predicts which category each input belongs to. These categories are called classes. In binary classification, there are two classes. Cross-entropy is a commonly used cost function for binary classification. This cost function is defined as

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)], \quad (16)$$

where n is the number of training examples, $y_i \in \{0, 1\}$ is the true class, and $\hat{y}_i$ is the predicted probability. J is small when the predicted probability is close to the true class. J grows without bound as the predicted probability approaches the opposite class.

The gradient $\nabla_{\theta} J$ is the vector of partial derivatives of J with respect to θ . $\nabla_{\theta} J$ points in the direction of the greatest increase of J . Moving in the direction of the negative $\nabla_{\theta} J$ decreases J . This procedure is called gradient descent. Each step of gradient descent updates the parameters θ according to

$$\theta' = \theta - \eta \nabla_{\theta} J, \quad (17)$$

where η is the step size, called the learning rate. The step is repeated until the cost stops decreasing. Backward propagation computes the gradient of J with respect to θ by applying the chain rule to the composition in Eq. (14). The norm of a vector is a measure of its length. The gradient norm measures the magnitude of $\nabla_{\theta} J$ and is defined as

$$\|\nabla_{\theta} J\| = \sqrt{\sum_{i=1}^d \left(\frac{\partial J}{\partial \theta_i} \right)^2}, \quad (18)$$

where d is the number of parameters in θ . Computing the gradient over the entire training data is computationally expensive. Instead, the training data is divided into subsets called minibatches. The gradient is computed, and the parameters are updated for each minibatch. The minibatches are drawn randomly, so each gradient is a noisy estimate of the gradient over the full training data. This is called stochastic gradient descent (SGD). One pass over the entire training data is called an epoch. The training data is shuffled at the start of each epoch, so that the minibatches differ between epochs.

The cost function of a neural network is generally non-convex. A convex function has no local minima other than its global minimum, while a non-convex function can have several local minima. Gradient descent may converge to a local minimum, which may not be the global minimum. Neural networks are trained with optimizers, variants of gradient descent that adapt the step size during the training process. RMSProp is an

example of such an optimizer. It divides the learning rate by a running average of the squared partial derivatives for each parameter. As a result, each parameter has its own effective learning rate.

The initial values of the parameters θ affect the convergence of the training process. If the initial values are too small, the gradients become very small, and the training process is slow. If the initial values are too large, the gradients grow, and the training process diverges. Two commonly used initialization methods are Xavier (Glorot and Bengio 2010) and He initialization (He et al. 2015). Both methods draw the initial values from a normal distribution with mean zero. The standard deviation differs between the two methods

$$s_{\text{Xavier}} = \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}, \quad s_{\text{He}} = \sqrt{\frac{2}{n_{\text{in}}}}, \quad (19)$$

where n_{in} and n_{out} are the input and output dimensions of the layer.

Apart from learnable parameters θ , neural networks have hyperparameters that are set before training. Hyperparameters include the learning rate, the number of epochs, the minibatch size, and the width of the layers. They are not learned during training, but they affect the training process and the final model. Hyperparameters are chosen by experiment.

The capacity of a neural network is its ability to represent a wide range of functions. Adding more layers to the model or increasing the width of the layers increases the capacity. A model with insufficient capacity underfits. Underfitting occurs when the output of the model does not fit the training data well. A model with excessive capacity is prone to overfitting. Overfitting occurs when the model fits the training data closely but performs poorly on unseen data.

3.1.2 Recurrent Neural Networks and LSTM

Time series data is an ordered sequence: the position of each value in the sequence carries information. A dense layer has separate parameters for each input position. What a dense layer learns from one position does not transfer to other positions. A recurrent layer processes a sequence of values, using the same parameters θ for each position (Goodfellow, Bengio, et al. 2016, pp. 378–381). A neural network consisting of recurrent layers is called a recurrent neural network. Recurrent layers are updated sequentially, one step at a time. At each step, a recurrent layer takes the current input and a hidden state carried from the previous step to produce the output and the next hidden state

$$h^{(t)} = f(h^{(t-1)}, x^{(t)}; \theta), \quad (20)$$

where $h^{(t)}$ is the hidden state at step t and $x^{(t)}$ is the input at step t . The final hidden state is a fixed-length vector representation of the entire sequence. The dimension of the hidden state is called the hidden size of the recurrent layer and determines the capacity of the layer. The hidden size is a hyperparameter that is set before training.

In a recurrent layer, the gradient $\nabla_{\theta} J$ is propagated backward through every step of the sequence. The same parameters θ are applied at each step, so $\nabla_{\theta} J$ is multiplied by the same factor at each step. As a result, $\nabla_{\theta} J$ can grow without bound or vanish to zero over long sequences. This is commonly called the vanishing and exploding gradient problem (Goodfellow, Bengio, et al. 2016, pp. 401–403).

The long short-term memory (LSTM) cell (Hochreiter and Schmidhuber 1997) addresses the vanishing gradient problem. It introduces a memory cell $c^{(t)}$, carried alongside the hidden state. Three gates with entries in $(0, 1)$ control the memory cell. They determine how much of the memory cell is retained, how much of the current input is stored in the memory cell, and how much of the memory cell is passed to the output (Goodfellow, Bengio, et al. 2016, pp. 408–411). The memory cell is updated by addition, so the gradient can flow through many steps without vanishing. The gated recurrent unit (GRU) (Cho et al. 2014) is a related gated architecture.

3.2 Generative Adversarial Networks

Generative modeling is an unsupervised learning approach (Sec. 3.1.1). A generative model takes a set of training examples and learns the probability distribution that produced them. GANs are generative models first introduced by Goodfellow, Pouget-Abadie, et al. (2014). GANs have been particularly successful in image and video generation. They have also been used to create synthetic data for training other machine learning models and to fill in missing data (Goodfellow, Pouget-Abadie, et al. 2020).

3.2.1 Basic Generative Adversarial Networks

GANs are based on game theory, which studies situations where several decision-makers are present. Each participant chooses an action, and the outcome depends on the choices of all participants. A GAN has two decision-makers, the generator G and the discriminator D , typically implemented as neural networks. G takes a vector of random noise as input and generates a sample as output. D takes a real or generated data sample as input and outputs a probability that the input sample is real. The goal of G is to fool D by generating realistic samples. The goal of D is to distinguish real samples from generated ones.

Assume that the data x follows an unknown distribution p_{data} , from which new samples need to be generated. The generator is a neural network $G(z; \theta_g)$, where θ_g denotes the learnable parameters. The input z is a noise vector drawn from a prior distribution p_z . A Gaussian distribution is commonly used to define p_z , but the GAN framework does not restrict this choice. The probability distribution of the generated samples is denoted p_g . The discriminator is a neural network $D(x; \theta_d)$, with learnable parameters θ_d . It outputs the estimated probability that the input was drawn from p_{data} rather than p_g . G and D have cost functions, respectively $J^{(G)}(\theta_d, \theta_g)$ and $J^{(D)}(\theta_d, \theta_g)$. Both G and D need to minimize their cost functions, while they can only control their learnable pa-

rameters, respectively θ_g and θ_d . Hence, the solution is a Nash equilibrium, rather than a minimum of a single cost function.

The GAN framework is flexible in the choice of the cost functions applied to G and D . The simplest version of the game is a zero-sum game. In a zero-sum game, the sum of the cost functions is always zero,

$$J^{(G)}(\theta_d, \theta_g) = -J^{(D)}(\theta_d, \theta_g). \quad (21)$$

The cost function of D is defined as a standard cross-entropy (Sec. 3.1.1)

$$J^{(D)}(\theta_d, \theta_g) = -\frac{1}{2} \mathbb{E}_{x \sim p_{\text{data}}} \log D(x) - \frac{1}{2} \mathbb{E}_{z \sim p_z} \log(1 - D(G(z))). \quad (22)$$

Learning the zero-sum game minimizes the Jensen–Shannon divergence between p_{data} and p_g . Convergence holds only if both players are updated in function space. In function space, D and G are updated freely among all possible functions. In practice, D and G are neural networks and updated in parameter space. In parameter space, the reachable functions are limited to those the network architecture can represent. D minimizes a cross-entropy while G maximizes the same quantity. Generated samples are rejected with high confidence at the start of training, and the gradient of G vanishes at the point at which it performs worst (Goodfellow 2017). Therefore, the cost function of G is constructed by flipping the target of the cross-entropy,

$$J^{(G)}(\theta_d, \theta_g) = -\frac{1}{2} \mathbb{E}_{z \sim p_z} \log D(G(z)). \quad (23)$$

The cost functions of D and G do not sum to zero anymore. Hence, the game is no longer zero-sum. Alternative cost functions have been introduced over time. Fig. 1 is the visualization of a standard GAN workflow adapted from Vuletić, Prenzel, et al. (2024).

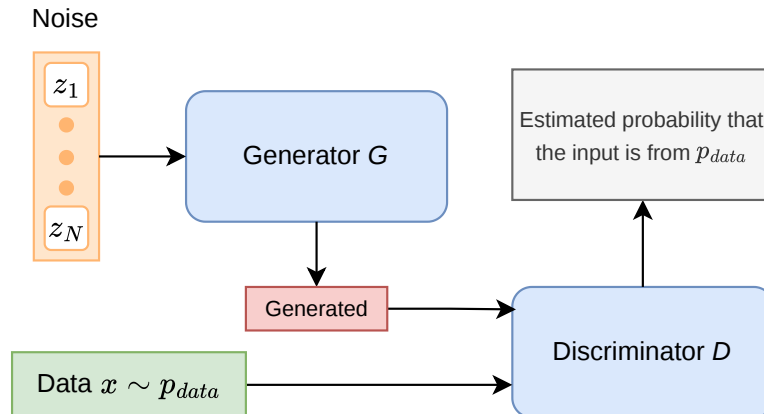


Figure 1: Structure of a standard GAN

GANs are trained by updating the two networks in turn. At each step, a minibatch of samples and noise vectors is drawn from p_{data} and p_z , respectively. D is updated with respect to θ_d while θ_g is fixed. G is updated with respect to θ_g while θ_d is fixed. D is commonly trained k times for every update of G , although Goodfellow (2017) argues that $k = 1$ is sufficient. k is treated as a hyperparameter. The update cycle is shown in Fig. 2.

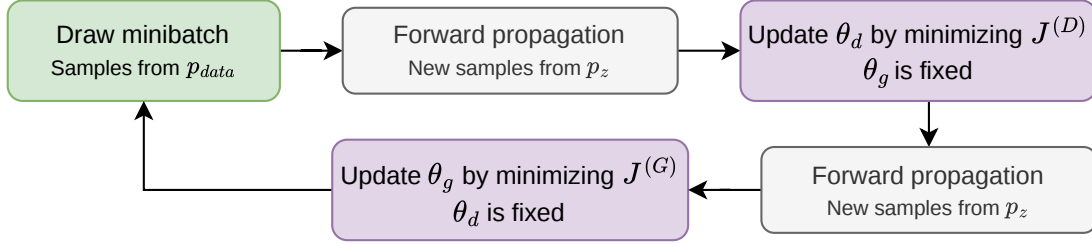


Figure 2: Alternating update cycle of a GAN. One D update is performed per G update ($k = 1$).

As stated previously, convergence is not guaranteed in practice. This means that the training of a GAN can settle in a state where $p_g \neq p_{\text{data}}$. The most common such state is mode collapse. It occurs when G repeatedly maps different noise vectors to very similar outputs. As a result, p_g covers only a small part of p_{data} , but D assigns a high probability that the generated samples are real. Mode collapse is still an unresolved challenge in GAN training. In recent years, several strategies have been proposed to mitigate its effect, such as stronger generators and discriminators, longer training, and regularization. However, each of these approaches comes with drawbacks (Barsha and Eberle 2025).

3.2.2 Conditional GANs

Standard GANs can be extended to conditional models (Goodfellow, Pouget-Abadie, et al. 2014). This extension was further developed by Mirza and Osindero (2014). Fu et al. (2019) apply cGANs to the simulation of financial time series. In cGANs, the data follows a conditional distribution $p_{\text{data}}(x | c)$, where c is auxiliary deterministic information. The condition is applied to D and G as an additional input. The cost functions for G and D are the same as for a standard GAN with the condition c ,

$$J^{(D)}(\theta_d, \theta_g) = -\frac{1}{2} \mathbb{E}_{x \sim p_{\text{data}}} \log D(x | c) - \frac{1}{2} \mathbb{E}_{z \sim p_z} \log(1 - D(G(z | c) | c)), \quad (24)$$

$$J^{(G)}(\theta_d, \theta_g) = -\frac{1}{2} \mathbb{E}_{z \sim p_z} \log D(G(z | c) | c). \quad (25)$$

Fig. 3 is the visualization of a cGAN workflow adapted from Vuletić, Prenzel, et al. (2024).

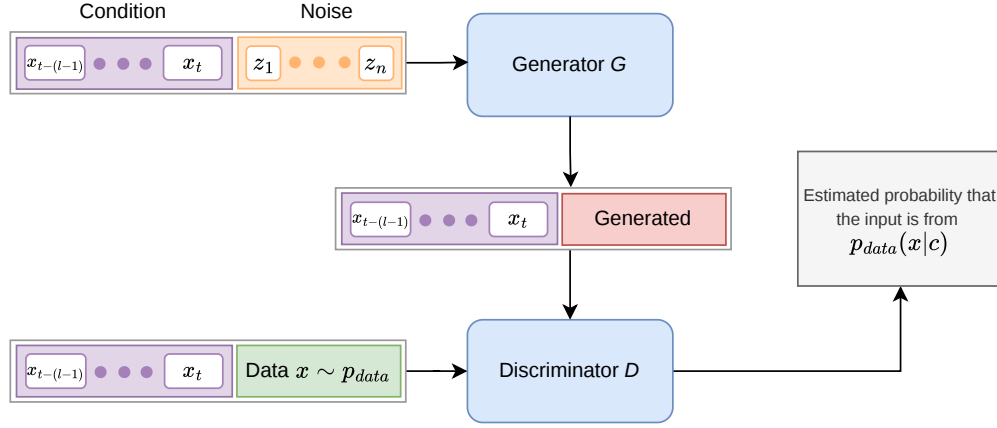


Figure 3: Structure of a cGAN

3.2.3 ForGAN

The conditional input proposed in cGANs prepared the ground for domain-specific GAN architectures. ForGAN was introduced specifically for time series forecasting (Koochali et al. 2019).

ForGAN is trained to approximate the conditional distribution of the next value in a time series. The condition is the sequence of the previous l observations, ending at time t ,

$$c = (x_{t-(l-1)}, \dots, x_t). \quad (26)$$

A sliding window of $l + 1$ observations is moved along the series. At each position, the first l observations form c and the last is x_{t+1} . G takes a noise vector z and the condition c as input and outputs a value for the target x_{t+1} . D takes a value of x_{t+1} and c and outputs the probability that x_{t+1} is from $p_{data}(x_{t+1} | c)$. The full probability distribution of x_{t+1} can be obtained through repeated sampling of G with different noise vectors and the same c .

ForGAN adds a recurrent layer to G and D . The layer processes c as an ordered sequence and constructs a vector representation. The recurrent layer can be an LSTM or a GRU (Sec. 3.1.2). The high-level architecture of the ForGAN model is shown in Fig. 4.

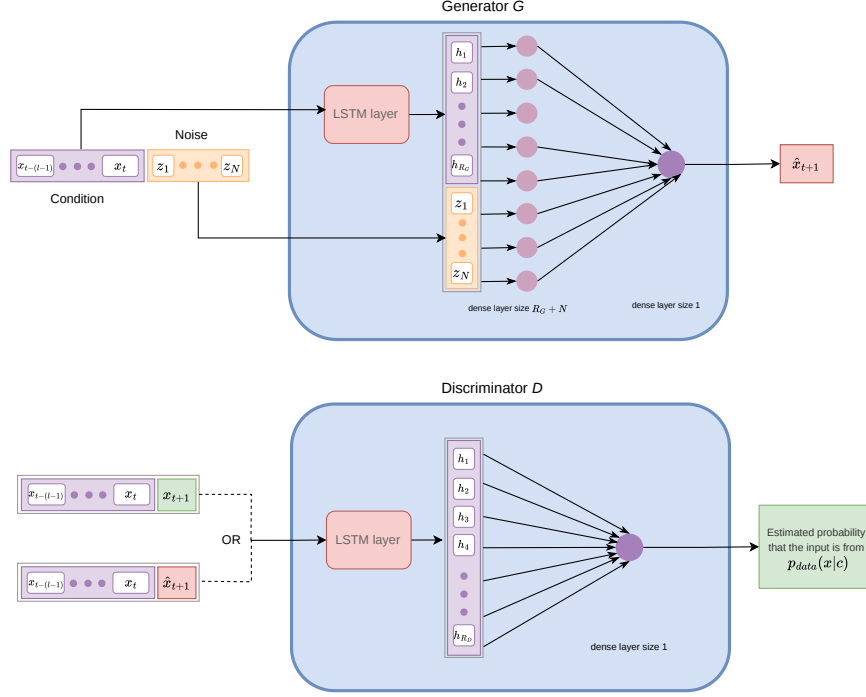


Figure 4: Structure of ForGAN

3.2.4 Fin-GAN

Fin-GAN was introduced as an extension of ForGAN (Vuletić, Prenzel, et al. 2024). It focuses on the probabilistic forecasting of financial time series, specifically equity returns. The ForGAN architecture and the cost function of D are unchanged. An LSTM is chosen as the recurrent layer.

The motivation behind Fin-GAN is to correctly forecast the sign of the return, since the sign determines whether the trade is profitable. For this purpose, economics-driven terms are introduced into the cost function of G . In contrast to the unsupervised setting of standard GANs, Vuletić, Prenzel, et al. (2024) classify Fin-GAN as supervised learning. The terms are defined over a minibatch of n_{batch} pairs. For each condition in the batch, G produces a forecast $\hat{x}_i$ of the realized value x_i . Let $\hat{x} = (\hat{x}_1, \dots, \hat{x}_{n_{\text{batch}}})$ and $x = (x_1, \dots, x_{n_{\text{batch}}})$.

Cost function. $J^{(G)}$ can be the zero-sum, cross-entropy, or Wasserstein cost function. The latter replaces the Jensen–Shannon divergence between p_{data} and p_g with the Wasserstein-1 distance (Arjovsky et al. 2017). Vuletić, Prenzel, et al. (2024) report that the Wasserstein cost function was numerically unstable during training. Hence, $J^{(G)}$ is the cross-entropy cost function given in Eq. (25). In addition, Fin-GAN introduces four economics-driven terms to the cost function of G ,

$$L^G(x, \hat{x}) = J^{(G)} - \alpha \text{PnL}_a^*(x, \hat{x}) + \beta \text{MSE}(x, \hat{x}) - \gamma \text{SR}^*(x, \hat{x}) + \delta \text{STD}(x, \hat{x}). \quad (27)$$

A positive forecast corresponds to buying an asset, and a negative forecast to selling it short. The profit and loss PnL of the trade is the sign of the forecast multiplied by the realized return $\text{PnL}(x_i, \hat{x}_i) = \text{sgn}(\hat{x}_i) \cdot x_i$. Since only the sign enters the formula, the outcome does not depend on the magnitude of the forecast. $\text{sgn}(\hat{x}_i)$ is not differentiable, so the standard term PnL would not provide a gradient for training. A smooth approximation is introduced,

$$\text{PnL}_a^*(x_i, \hat{x}_i) = \tanh(k_{\tanh} \hat{x}_i) x_i, \quad (28)$$

where $k_{\tanh}$ is a hyperparameter that controls the accuracy of the approximation. Vuletić, Prenzel, et al. (2024) use $k_{\tanh} = 100$, which is large enough to give a close approximation and small enough to maintain strong gradients.

Although the sign of the forecast is the main interest, forecasting an $\hat{x}_i$ that is close to x_i is also of importance. The mean squared error MSE measures the squared difference between the forecast and the realized value, averaged over the minibatch. SR^* is defined as the mean of the PnL_a^* series divided by its standard deviation,

$$\text{SR}^*(x, \hat{x}) = \frac{\bar{p}}{\sqrt{\frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} (p_i - \bar{p})^2}}, \quad (29)$$

where $p_i = \text{PnL}_a^*(x_i, \hat{x}_i)$ and $\bar{p}$ is its mean over the n_{batch} forecasts. SR^* deviates from Eq. (1). The risk-free rate is not subtracted, since the daily risk-free rate is small relative to the daily PnL_a^* . The ratio is not annualized, because a constant factor would be absorbed by the weight γ during gradient matching. Lastly, SR^* is computed over the n_{batch} forecasts of a minibatch rather than over a return series. As described in Sec. 3.1.1, SGD requires random minibatches. As a result, forecasts entering SR^* are not in chronological order (Vuletić, Prenzel, et al. 2024, p. 11).

The standard deviation STD term is the denominator of Eq. (29). Logically, maximizing SR^* is the same as maximizing PnL_a^* and minimizing STD at the same time. As a result, including all three terms in L^G is redundant. However, during training, SR^* and the combination of PnL_a^* and STD produce different gradients (Vuletić, Prenzel, et al. 2024).

Gradient matching. All the terms above are measured in different units. Hence, they produce incomparable gradients. As a solution, four additional hyperparameters $\alpha, \beta, \gamma, \delta$ are introduced. Fin-GAN is first trained on $J^{(G)}$ only, and gradient norms (Sec. 3.1.1) of the other terms are calculated. Each hyperparameter is the mean of the ratios between the cross-entropy gradient norms and the gradient norms of the corresponding term. Large values of $\alpha, \beta, \gamma, \delta$ make the feedback of D insignificant and move the model toward supervised learning, but small values leave it in the standard GAN setting.

If a term is not included in the objective, the hyperparameter is set to zero. Several versions of L^G can be constructed from different combinations of terms. Only combinations that include economics-driven terms are considered for Fin-GAN. Vuletić, Prenzel, et al. (2024) train eight such versions and select the combination with the highest Sharpe

ratio SR_w on the validation split. Then, the generator trained with the selected combination is evaluated on the test split. In all versions, D keeps the cross-entropy cost function of Eq. (24).

Mode collapse. Vuletić, Prenzel, et al. (2024) distinguish two types of mode collapse for time series. Distributional mode collapse happens when G generates nearly identical samples for a single condition. Mean mode collapse happens when the means of the generated distributions are nearly identical across all conditions. Both are identified by a standard deviation below a threshold $\epsilon = 0.0002$. Fin-GAN uses normal Xavier initialization (Sec. 3.1.1), since ForGAN suffered mode collapse in 67 % of runs under He initialization. However, no mode collapse happened with the economics-driven terms, under either initialization. Mode collapse is a known failure mode of GAN training in general, as described in Sec. 3.2.1.

Weighted strategy. The Sharpe ratio SR_w is not to be confused with the SR^* term of Eq. (29). SR_w is computed on the wPnL series over a full data split. SR_w measures the directional accuracy of the forecasts and not the shape of the generated distribution. For each condition, B samples are drawn from G with different noise vectors. The weighted profit and loss of trade i is

$$\text{wPnL}_2^i = 10000 (p_u^i - p_d^i) x_i, \quad (30)$$

where p_u^i and p_d^i are the fractions of non-negative and negative samples among the B samples. Vuletić, Prenzel, et al. (2024) use $B = 1,000$. Since the equity data uses two returns per day, intraday and overnight, two positions are taken per day. The wPnL^i series is the sum of the two wPnL_2^i values per day. For the equity data of Vuletić, Prenzel, et al. (2024), two returns per day give $n_{\text{days}} = \lfloor n/2 \rfloor$. The Sharpe ratio SR_w is then

$$SR_w = \sqrt{252} \frac{\overline{\text{wPnL}}}{\sqrt{\frac{1}{n_{\text{days}}} \sum_{i=1}^{n_{\text{days}}} (\text{wPnL}^i - \overline{\text{wPnL}})^2}}, \quad (31)$$

where $\overline{\text{wPnL}}$ is the mean of the wPnL^i series over the n_{days} trading days. The factor $\sqrt{252}$ annualizes the ratio, where 252 is the conventional number of trading days in a year.

3.3 Related Work

GANs have been applied to financial time series beyond the ForGAN and Fin-GAN frameworks described in Sec. 3.2. Three directions are relevant for this thesis. The first direction focuses on alternative GAN architectures and cost functions for generating realistic financial time series. The second direction investigates GAN-based simulators of option markets. The third direction examines applications of GANs in energy markets.

In the first direction, Yoon et al. (2019) introduce TimeGAN, which trains the generator with a cross-entropy cost function and an additional supervised next-step prediction term computed in an embedding space. Quant GANs keep the standard cross-entropy

cost function and replace the recurrent architecture used in earlier time series GANs with temporal convolutional networks to capture volatility clustering (Wiese, Knobloch, et al. 2020). Ni et al. (2020) propose the conditional Sig-Wasserstein GAN, which replaces the trained discriminator with an explicitly computed distance between summary statistics of whole paths. Zhang et al. (2026) introduce the Stylized Facts Alignment GAN, which adds stylized facts as differentiable constraints to the cost function of the generator.

In the second direction, Wiese, Bai, et al. (2019) build a GAN-based simulator of an equity option market and show that adversarial training reproduces the historical skewness, kurtosis, and autocorrelation of implied volatilities more accurately than a vector autoregression (VAR) model or quasi-maximum-likelihood estimation. Vuletić and Cont (2024) introduce VolGAN, a conditional GAN that simulates the return of the underlying asset and its implied volatility surface. VolGAN adds a smoothness penalty to the cost function of the generator so that the simulated surfaces avoid unrealistic jumps between neighboring moneyness levels and expiries.

In the third direction, Bohórquez Correa et al. (2026) apply TimeGAN to 25 energy markets, including natural gas, to generate synthetic price data and use it to estimate crisis-risk indicators. Walter and Wagner (2024) propose a conditional GAN architecture combining convolutional and recurrent layers to generate day-ahead electricity price scenarios for the German market. Korn and Ramadani (2025) apply the unmodified ForGAN architecture with a GRU cell to German electricity consumption data and show that it reproduces the recurring daily and weekly patterns in the series.

Across these three directions, GANs have been used to generate realistic financial time series, to simulate option markets, and to model energy market data. However, no study uses GAN-generated paths of natural gas futures to price options. The proposed GAN architectures and cost functions are developed and tested on equity or general financial data. The option market simulators are restricted to equity options, and the energy market applications generate synthetic consumption or price data for forecasting and risk monitoring. This thesis addresses this gap by applying the ForGAN and FinGAN frameworks to TTF natural gas futures data and evaluating the generated paths through option prices.

4 Methodology and Implementation

This chapter describes how the data is prepared, how the models are trained and evaluated, and how option prices are computed from the generated returns.

Two datasets are used. The TTF futures data covers January 2010 to April 2026 and provides the daily front-month returns. On this dataset the model is trained and evaluated. The TTF options data provides the market prices and implied volatilities over the test period. The model follows the ForGAN architecture and the reference Fin-GAN implementation (Vuletić, Prenzel, et al. 2024). Ten cost function branches are trained. The first is the ForGAN baseline, trained on cross-entropy alone. The other nine are Fin-GAN branches that add the economics-driven terms of Sec. 3.2.4. Each branch is trained under ten network and training configurations and one input scaling run, with five seeds each. The selected model generates price paths for pricing call options on 20 valuation dates, and GBM and two bootstrap modes serve as benchmarks.

Sec. 4.1 constructs the daily TTF front-month return series from the futures data and splits it into training, validation, and test splits. It also prepares the options data that provides the market reference. Sec. 4.2 describes the model architecture, the training setup, and the adaptations to the reference Fin-GAN implementation. Sec. 4.3 defines the configurations and the cost function branches. Sec. 4.4 describes how the generated returns are compared with the test split. Sec. 4.5 describes how the selected model generates price paths by recursive rollout and how call options are priced from these paths. Sec. 4.6 introduces the benchmark models, Sec. 4.7 lists the software environment, and Sec. 4.8 summarizes the setup.

4.1 Data Pipeline

This section describes how the two datasets were processed. The futures data is turned into the daily front-month return series and split into training, validation, and test splits (Secs. 4.1.1–4.1.4). The options data is joined and filtered into the panel that provides the market prices for the comparison in Sec. 5.7 (Secs. 4.1.5 and 4.1.6).

4.1.1 Futures Data

The futures data was obtained from the Bloomberg infrastructure of BASF and covers TTF natural gas futures for the front-month, second-month, and third-month contracts between January 2010 and April 2026. The data is distributed as three schemas: contract prices, contract metadata, and monthly listings of active contracts.

Daily settlement prices. This schema provides the price history of the contracts in the dataset. Each contract is observed once per trading day, identified by contract symbol and trading date. Each record contains opening, high, low, and daily closing prices in EUR/MWh, trading volume, and open interest. Coverage of these fields varies considerably with how actively a contract was traded and how far from expiry it is. The

closing price is the exception and is available throughout the whole dataset. Additionally, it falls outside the intraday high-low range for 7,712 of 25,884 records, or 29.8 %. The deviation is small in most cases, with a median of 0.10 EUR/MWh, but reaches up to 42.26 EUR/MWh. This confirms that it is a settlement price set by the exchange rather than a price at which trading occurred.

Contract metadata. This schema provides static attributes per unique contract in the dataset. The expiry date provided per record is used to rank contracts by time to maturity in Sec. 4.1.2. All contracts in the dataset are physically delivered instruments traded on ICE Endex and quoted in EUR/MWh. First and last trading dates, notice dates, and delivery dates are also recorded.

Contract listings. This schema records which contracts were active on the market, as monthly snapshots taken on the first calendar day of each month. Each contract is observed once per snapshot. Together with the expiry dates, the listings determine which contract is the front month on a given date, as described in Sec. 4.1.2.

4.1.2 Front-Month Series Construction

As described in Sec. 4.1.1, the contract listings and the contract metadata each hold part of the information required to identify the front-month contract. The listings record which contracts were active in a given month, while the metadata records when each contract expires. Therefore, the expiry dates were joined onto the listings by contract symbol. As a result, each listing record carries the expiry date of the contract it refers to.

The contracts were then sorted by expiry date within each monthly snapshot and the earliest-expiring one was kept. This gives the front-month contract for that month. Since the snapshots are monthly, but the series must be daily, each trading day was assigned the front-month contract from the most recent snapshot on or before that day. The settlement price of the selected contract was then added for each trading day, matching on trading date and contract symbol. Trading days without an available settlement price were discarded. The resulting series contains 3,998 trading days spanning 2010-01-04 to 2026-04-24, with settlement prices ranging from 3.509 EUR/MWh to 339.196 EUR/MWh. The construction of the front-month series is implemented in `build_front_month.py`.

Since the front-month contract is replaced at each expiry, the series is not the price history of a single instrument. The listings are monthly snapshots, so the front-month contract switches on the first trading day of each month. The days between the last trading day of the contract and the next listing do not carry a settlement price. These days were discarded from the series.⁴ Returns spanning a contract transition cover two to six calendar days, with a median of four, and reflect the change in the price level as well as the difference between the two contracts. Across the 195 transition days in the sample, the standard deviation of log returns is 0.071, compared to 0.037 on all other

⁴Trading in a TTF futures contract ceases two UK business days before the first day of the delivery month.

days. When the transition days are excluded, excess kurtosis falls from 14.91 to 14.65, and skewness from 0.70 to 0.26.⁵ Transition days contribute little to the tail weight of the series, but they are responsible for most of its positive skew. The transition effect described above is preserved and quantified rather than removed, since restricting the analysis to a single contract would reduce the sample to a few months.

4.1.3 Return Definition

The original Fin-GAN input series is built from two returns per day (Vuletić, Prenzel, et al. 2024). The first is the intraday return, from the opening to the closing price, and the second is the overnight return, from one closing price to the next opening price. The return of the corresponding sector ETF is then subtracted from each entry. This isolates the part of the movement specific to the asset.

Neither step has been adopted for the TTF futures dataset. The two-return scheme requires opening and closing prices that are both traded prices. As shown in Sec. 4.1.1, the daily price in this dataset is a settlement price set by the exchange. A return computed between the opening trade and the settlement price would therefore combine two different price concepts.

Additionally, the excess-return step requires a benchmark that leaves a meaningful residual when subtracted. In the case of equities, subtracting the sector ETF return removes the market-wide component and leaves the movement specific to the individual stock. European natural gas has no comparable instrument.

As a substitute, the second-month contract was considered. However, the difference between the front-month and second-month returns is a calendar spread return, as described in Sec. 2.3. A model that is trained on it would learn how the gap between two contracts moves, not how the gas price itself moves. Option payoffs depend on the price level at expiry, so a model trained on spread returns could not be used to price options.

The dataset was constructed from daily logarithmic returns of the front-month settlement price. Returns are approximately stationary and therefore comparable across the sample period. Additionally, log returns are additive across time and allow price levels to be recovered exactly from simulated returns. The second property is required for the Monte Carlo procedure in Sec. 4.5. The daily log return is

$$x_t = \log\left(\frac{P_t}{P_{t-1}}\right), \quad (32)$$

where P_t is the front-month settlement price on trading day t . The transformation gives 3,997 return observations.

4.1.4 Data Split

The return series was split chronologically into training (80%), validation (10%), and test (10%) splits using the proportions of Vuletić, Prenzel, et al. (2024). Training cov-

⁵Skewness and excess kurtosis are computed with the sample estimators g_1 and g_2 of Eq. (13).

4 Methodology and Implementation

ers 2010-01-05 to 2023-01-18 (3,197 returns), validation 2023-01-19 to 2024-09-04 (399 returns), and test 2024-09-05 to 2026-04-24 (401 returns).

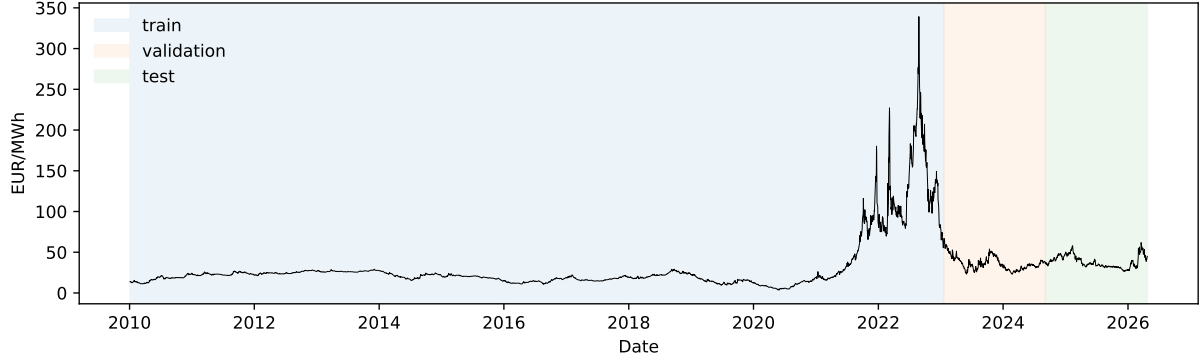


Figure 5: TTF train, validation, and test data split

The chronological split was explicitly used to avoid look-ahead bias. With this split, the 2021 to 2022 price disruption discussed in Sec. 2.2 falls entirely within the training period, and the 2026 disruption falls entirely within the test period, as seen in Fig. 5. Sliding window matrices were constructed for each split separately, so that no window spans a split boundary. The window was moved forward by one trading day.

4.1.5 Options Data

The TTF options data was obtained from Databento and covers ICE Endex TTF natural gas options. Contract records span March 2019 to June 2026, and daily settlement prices are available from March 2023 to June 2026. The price coverage contains the full test split of Sec. 4.1.4. The data is distributed as five schemas: Definition, Statistics, Best bid and offer (BBO), Open High Low Close Volume (OHLCV), and Status.

Definition. Contract metadata is provided per instrument in the dataset. All options are European and quoted in EUR/MWh, matching the futures dataset of Sec. 4.1.1. The underlying of each option is a monthly TTF futures contract. Instrument classes present in the dataset are calls, puts, calendar spread instruments, and multi-leg instruments. Calendar spread instruments hold a long position in one delivery month and a short position in another. Their value depends on the calendar spread of Sec. 2.3 rather than on the price level. Multi-leg instruments combine multiple options into a single contract. Neither class has a strike price, so the payoff described in Sec. 2.4 is undefined for them. Only call and put options were kept. This leaves 15,005,854 of the 56,882,528 records in the schema, split evenly between calls and puts. Strike prices range from 0.50 EUR/MWh to 1,000 EUR/MWh across 510 distinct values, and contracts carry 142 distinct expiry dates, one per calendar month. This schema is an event log, where each record describes a change in the metadata of a contract. The most recent record for each contract was retained, giving 35,952 contracts.

Statistics. Daily statistics are provided per statistic, contract, and trading day. The schema holds 99,307,437 records, of which 60,664,334 are put and call options. It contains settlement prices and implied volatilities, alongside further statistics that are not used, leaving 17,424,688 records. The implied volatility is published by the exchange, so the numerical procedure of Sec. 2.5 is not required for the market prices. The dataset contains final and preliminary records, of which only the final ones were kept. A total of 5,807,828 records were retained for settlement prices and 5,812,246 for implied volatilities. Of these, 392,904 contract-days carry a duplicate record, of which 4,128, around 1 %, carry a different price. These cluster on a few dates and are consistent with revisions published by the exchange. To ensure that revised prices were used, the record with the highest sequence number was retained. The resulting panel contains 11,231,588 records for 20,252 contracts, around 56 % of the 35,952 contracts in the dataset.

BBO. The schema records the best bid and ask prices in a sequence of snapshots through each trading day. A bid and an ask together are referred to as a quote. The quote is two-sided when both are present. The schema covers the period from 2023-05-02 to 2026-06-26. However, two-sided quotes are almost absent at the start of that period, with 2,485 in 2023 and 2,069 in 2024 against 25,969,129 in 2025. The test split of Sec. 4.1.4 begins on 2024-09-05, so the quotes would cover only part of it.

OHLCV. The schema records the opening, high, low, and closing prices, together with the trading volume, in one-minute bars. The schema covers 2023-03-20 to 2026-06-26. Each bar summarizes the trades in a single contract over one minute. Of the 365,892 bars, 340,313 are on-exchange and the remainder are off-market trades that print at different prices. Only 4,327 of the 20,252 contracts with settlement prices have any trades, around 21 %. Out of all the days when an option received an official settlement price, only 0.47 % also had a trade. As a result, the schema does not provide a continuous price series.

Status. The schema records whether each contract was open for trading and whether it was open for quoting. The schema does not provide price information and is not used in this thesis.

4.1.6 Options Panel Construction

The panel used for the comparison in Sec. 5.7 is built from the definition and statistics schemas. The definitions hold the strike price and the expiry date, while the statistics hold the settlement price and the implied volatility. The two schemas were joined by contract identifier. The front-month series of Sec. 4.1.2 was then joined by the trading date to provide the front-month settlement price and the moneyness defined in Sec. 2.4 for each option. The moneyness of an option is exact only when the underlying is the front-month contract, since the series carries the price of whichever contract is front month on that date. This holds close to expiry. Each row of the panel is one contract on one trading day, and also carries the number of days from the trading date to expiry. In total there are 5,615,794 records for 20,252 contracts. The days between the last trading day of a contract and the next front-month listing do not carry a front-month settlement

price, so 268,598 records have no moneyness. A further 181,950 records fall after 2026-04-24, the end of the futures series, and also do not carry moneyness. The construction of the options panel is implemented in `build_options_panel.py`.

The panel spans March 2023 to June 2026, while the test split of Sec. 4.1.4 covers September 2024 to April 2026. It was restricted to the test split, resulting in 1,763,210 records for 9,144 contracts. Three groups of records were then filtered out. The first group holds 5,062 records on the expiry date, which have a time to maturity of zero, so no pricing method applies to them. A further 81,584 records fall on the days without a front-month settlement price, described above. Lastly, a moneyness band of 0.5 to 2.0 was applied. Since the median settlement price falls smoothly with moneyness, there is no natural break in the data, so the bounds were set as follows. Below the 0.5 lower bound, the implied volatility rises and becomes erratic. Almost all the option value is the amount it would pay if exercised immediately, and the volatility is recovered from the small remainder. Above the 2.0 upper bound, the median settlement price approaches the exchange minimum tick of 0.005 EUR/MWh, so the price does not carry information. The resulting panel contains 1,250,980 records for 7,208 contracts. The filtering of the options panel is implemented in `filter_options_panel.py`.

4.2 Architecture, Setup, and Adaptations

The model follows Fin-GAN, with the architecture and training setup taken from the reference implementation of Vuletić, Prenzel, et al. (2024). Sec. 4.2.1 describes the network structure of G and D . Sec. 4.2.2 describes the optimizer, the input scaling, and the two training phases. Sec. 4.2.3 describes the changes made to the reference implementation.

4.2.1 Model Architecture

The ForGAN architecture described in Sec. 3.2.3 was used without structural modifications. In G , the condition passed through an LSTM layer, described in Sec. 3.1.2. The final hidden state was concatenated with the noise vector z . The result passed through a dense layer of width $R_G + N$, then a ReLU activation, and another dense layer of width one, where R_G is the LSTM hidden size and N is the noise dimension. In D , the candidate value ($\hat{x}_{t+1}$ or x_{t+1}) was appended to the condition, which gave a sequence of size $l + 1$. The result passed through an LSTM layer with the hidden size R_D . The final hidden state passed through a dense layer with a sigmoid activation (Sec. 3.1). The network definitions are in `FinGAN.py`.

Following Vuletić, Prenzel, et al. (2024), $N = 8$, $R_G = 8$, $R_D = 8$, and $l = 10$. One value was forecast per condition, $\hat{x}_{t+1}$. The parameters N , R_G , R_D , and l were varied in Sec. 4.3. The normal Xavier initialization described in Sec. 3.2.4 was used.

4.2.2 Training Configuration

The RMSProp optimizer, described in Sec. 3.1.1, was used for both G and D , with a learning rate of 0.0001. One discriminator update was performed for each generator update, as described in Sec. 3.2.1. The weights were updated after each minibatch of size $n_{\text{batch}} = 100$. The training data was shuffled at the start of each epoch.

The input to G and D was standardized using the mean and standard deviation of the first 100 rows of the training window matrix. Using the earliest rows avoids look-ahead bias. As described in Sec. 4.1.4, the training split starts in January 2010. This means that the reference batch covers a period of relatively low volatility. The scaling parameters are not representative of the whole dataset, especially the 2021 to 2022 period of high volatility. The effect of this choice was tested separately, as described in Sec. 4.3.

The training was performed in two phases. In the first phase, both G and D were trained on cross-entropy alone for n_{grad} epochs. The gradient norms were recorded and α , β , γ , and δ were computed as described in Sec. 3.2.4. In the second phase, the networks were trained for n_{epochs} further epochs, separately for each of the nine cost function combinations listed in Sec. 4.3. An additional tenth branch was trained on cross-entropy alone for n_{epochs} further epochs, giving the ForGAN baseline used for comparison in Chapter 5. The effective training length was $n_{\text{grad}} + n_{\text{epochs}}$, since the network weights were updated in both phases. The values n_{grad} and n_{epochs} are varied across configurations, as described in Sec. 4.3.

Vuletić, Prenzel, et al. (2024) select the cost function combination and the corresponding G with the highest SR_w on the validation split (Sec. 3.2.4). SR_w measures directional accuracy, while option prices depend on the shape of the return distribution (Sec. 2.4). Therefore, the configuration and the cost function branch were selected on the distributional statistics of Sec. 4.4 and their spread across seeds on the test split. The training procedure is implemented in `run_ablation.py`.

4.2.3 Adaptations

The implementation is based on the reference Fin-GAN code published by Vuletić, Prenzel, et al. (2024). The network definitions, the gradient norm matching procedure, and the cost function terms of Sec. 3.2.4 were adopted without modification. The training loop for the branch trained on cross-entropy alone (Sec. 4.2.2) does not call the backward propagation on the generator loss, so no gradient reaches the generator. The training loops for the other branches do call it. To ensure that the ForGAN baseline is trained under the same procedure, the backward propagation on the generator loss was added to the corresponding training loop.

The data structure with intraday and overnight returns was not adopted. As described in Sec. 4.1, the dataset was constructed from daily log returns of the front-month settlement prices of TTF futures. The data loading functions were modified accordingly. Additionally, a separate evaluation module was implemented to compute the distribu-

tional statistics of Sec. 4.4. The generator G produces one value per noise sample, $\hat{x}_{t+1}$. To generate price paths, recursive rollout was implemented, as described in Sec. 4.5.

4.3 Experimental Design

This section defines what is trained, including the cost function branches, the network and training configurations, and the number of seeds per run. The full grid of results is given in App. A. As described in Sec. 3.2.4, the eight cost function combinations of Vuletić, Prenzel, et al. (2024) were used: PnL_a^* ; SR^* ; PnL_a^* & MSE ; PnL_a^* & SR^* ; PnL_a^* & STD ; SR^* & MSE ; PnL_a^* & MSE & SR^* ; PnL_a^* & MSE & STD .

A ninth combination, using only the MSE , was added as a supervised counterpart to ForGAN. A tenth branch was trained on cross-entropy alone, giving the ForGAN baseline. The ten branches are referred to by their cost function terms throughout Chapter 5, with the nine Fin-GAN branches named after their economics-driven terms. Each combination was trained under several configurations, listed in Table 1.

Configuration	l	N	R_G	R_D	$n_{\text{grad}} + n_{\text{epochs}}$
1	10	8	8	8	25 + 100
2	10	8	8	8	100 + 500
3	10	32	8	8	100 + 500
4	10	8	8	64	100 + 500
5	10	32	8	64	100 + 500
6	30	32	8	64	100 + 500
7	10	32	8	64	100 + 1500
8	10	32	64	64	100 + 500
9	10	32	64	64	100 + 1500
10	10	8	64	64	100 + 500

Table 1: Model configurations

The configurations differ in one hyperparameter at a time, to isolate the effect of each change. The first configuration is the reference configuration, following the values and the schedule of Algorithm 1 in Vuletić, Prenzel, et al. (2024). The second configuration tests whether a longer training period improves the distributional quality. The third and fourth configurations increase N and R_D separately.

The fifth configuration increases both, matching the default values of the published implementation. The sixth configuration tests whether a longer condition window of 30 trading days carries more information about the volatility of the series. The seventh configuration extends the training further. The eighth configuration increases the generator hidden size R_G , which is not varied in the paper or in the published implementation. The ninth configuration combines the longer training with the increased R_G . Lastly, the tenth configuration combines the increased R_G with the original N . An additional test

run was performed with the scaling parameters computed from the entire training split, using configuration 5 of Table 1.

Each configuration was trained with five fixed random seeds. Training runs of the same configuration differ noticeably in their distributional statistics, due to the stochastic nature of the training process. Therefore, all results were reported as mean and seed spread. Fixing the seeds makes the runs reproducible in the environment described in Sec. 4.7.

4.4 Evaluation Methodology

This section describes how the generated returns are compared with the test split. The same evaluation is applied to every trained model. This covers each of the ten cost function branches under each configuration, and each of the five seeds.

The evaluation starts from the window matrix of the test split (Sec. 4.1.4). Each row of the window matrix consists of a condition window of l realized returns and the realized return of the next day. The conditions were passed to the trained G , and the generated values were compared to the realized values. For each condition, $B = 1,000$ samples were drawn from G with different noise vectors.

The realized returns of the split, $x = (x_1, \dots, x_{n_{\text{cond}}})$, give the reference values, where n_{cond} is the number of rows in the window matrix. Since the first l returns are used as the initial condition window, a split of n returns gives $n_{\text{cond}} = n - l$ rows. The generated draws form a matrix $\hat{X} \in \mathbb{R}^{n_{\text{cond}} \times B}$, where $\hat{x}_{i,j}$ is the j -th draw of condition i . Additional sets were derived from $\hat{X}$. The path set $\hat{x}_{\text{path}} = (\hat{x}_{1,1}, \dots, \hat{x}_{n_{\text{cond}},1})$ is the first column of $\hat{X}$, giving one generated return per trading day. The pooled set takes all $n_{\text{cond}}B$ generated entries of $\hat{X}$

$$\hat{x}_{\text{pool}} = \{ \hat{x}_{i,j} \mid i = 1, \dots, n_{\text{cond}}, j = 1, \dots, B \}. \quad (33)$$

The mean set holds the mean of all draws for each condition

$$m = (m_1, \dots, m_{n_{\text{cond}}}), \text{ where } m_i = \frac{1}{B} \sum_{j=1}^B \hat{x}_{i,j}. \quad (34)$$

The pooled set $\hat{x}_{\text{pool}}$ contains B times more observations than x , therefore giving a more reliable estimate of the tails. The path set $\hat{x}_{\text{path}}$ preserves the order in time, since it holds one generated return per trading day.

Distributional statistics were computed for x and $\hat{x}_{\text{pool}}$: mean, standard deviation, skewness, excess kurtosis, minimum, maximum, and the 1 % and 99 % quantiles. Skewness and excess kurtosis measure the asymmetry and the tail weight described in Sec. 2.6. Additionally, the fractions of returns exceeding 5 % and 10 % in absolute value were computed. The fraction of observations whose absolute value exceeds u is denoted as $P(|x| > u)$. The exceedance fractions measure the frequency of extreme returns, which determines the value of out-of-the-money options (Sec. 2.4). The autocorrelation $C_2(\tau)$

of Eq. (11) was computed for x and $\hat{x}_{\text{path}}$ at $\tau = 1, 2, 5, 10$. The pooled set $\hat{x}_{\text{pool}}$ cannot be used since it has no order in time.

Lastly, the mode collapse described in Sec. 3.2.4 was checked. For distributional mode collapse, the standard deviation of each row of $\hat{X}$ was computed. The smallest value was compared with $\epsilon = 0.0002$. For mean mode collapse, the standard deviation of m was compared with the same ϵ value. The evaluation is implemented in `ttf_eval.py`.

4.5 Option Pricing Setup

This section describes how the selected model is used to price options. The rollout generates price paths from one-day returns, the paths are made risk-neutral, and call options are priced by Monte Carlo simulation and converted into implied volatilities.

The generator G produces one-day returns for the underlying futures contract. Option pricing, described in Sec. 2.4, requires the distribution of the settlement price at expiry, which lies several weeks ahead of the valuation date. The valuation date is the trading day on which the option is priced. The generator was applied recursively to generate a path of one-day returns. The procedure is referred to as a rollout. The rollout was initialized with the l realized returns that preceded the valuation date. At each step, G took the condition window of l returns and produced the next return. The generated return was appended to the condition window, and the oldest return was removed, keeping the window size constant. The step was repeated until the number of generated returns equaled the number of trading days to expiry. After l recursive steps, the condition window consisted of generated returns only (Fig. 6).

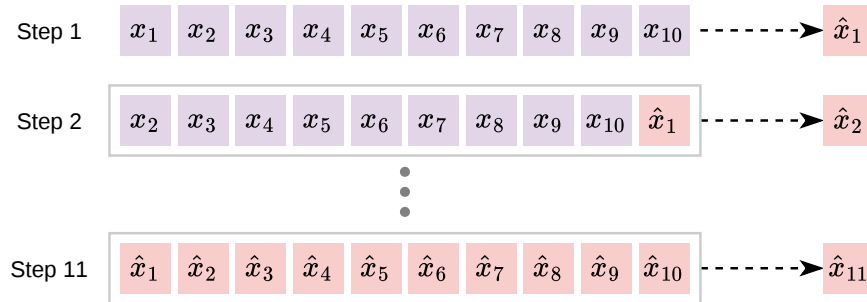


Figure 6: Condition window during the rollout

The realized returns were not used after initialization, since the returns after the valuation date are not available when the option is priced. As a result, after l recursive steps, G operates outside the regime it was trained on. Deviations of the generated returns from the realized distribution may accumulate along the path.

4 Methodology and Implementation

The generated returns were applied in sequence to the settlement price on the valuation date to yield the simulated settlement price at expiry

$$\hat{F}_T = F_t \exp \left(\sum_{k=1}^h \hat{x}_k \right), \quad (35)$$

where F_t is the settlement price on the valuation date, $\hat{x}_k$ is the simulated return on step k , and h is the number of trading days to expiry. The rollout was repeated $M = 20,000$ times with independent noise vectors to give a distribution of $\hat{F}_T$. The expected payoff of the option was calculated following the Monte Carlo procedure described in Sec. 2.4. Monte Carlo standard error was computed for the expected payoff. The error is the standard deviation of the payoff divided by $\sqrt{M}$.

The risk-neutral condition of Sec. 2.4 is not enforced in the rollout procedure. The generator was trained on realized returns, so the generated distribution reflects the observed market behavior, rather than the risk-neutral distribution. As a result, the mean of the simulated settlement prices does not equal F_t . The simulated prices were therefore rescaled so that their mean equals F_t , following Duan and Simonato (1995) with adaptations

$$\tilde{F}_T^{(i)} = F_t \frac{\hat{F}_T^{(i)}}{\frac{1}{M} \sum_{j=1}^M \hat{F}_T^{(j)}}, \quad (36)$$

where $\hat{F}_T^{(i)}$ is the simulated settlement price on path i and $\tilde{F}_T^{(i)}$ is the corrected price. The correction multiplies the simulated prices by a constant factor, so the skewness and excess kurtosis of the distribution are unchanged. For European options, Duan and Simonato (1995) apply the correction once, since the payoff depends only on the settlement price at expiry. They also apply a discount to the mean of the simulated prices, which is not required in Eq. (36), since the futures price satisfies the condition $\hat{\mathbb{E}}[F_T] = F_t$ without discounting (Sec. 2.4). The full pricing procedure for one valuation date is shown in Fig. 7.

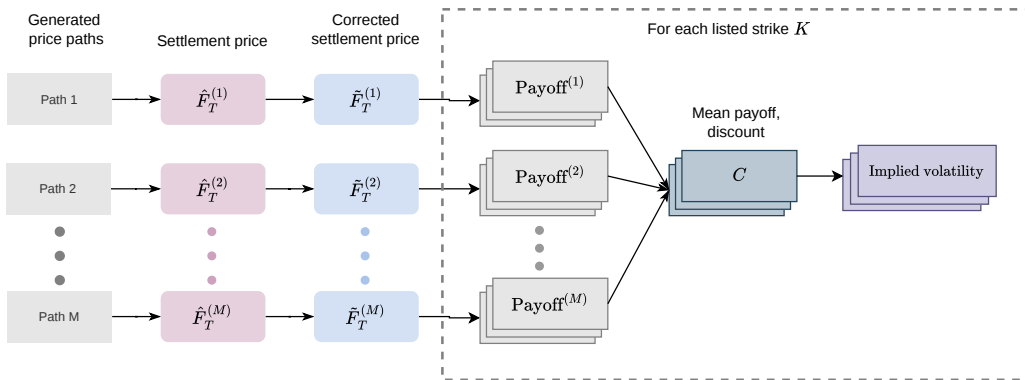


Figure 7: Pricing procedure for one valuation date

Both call and put options can be priced from the same simulated distribution of settlement prices, so pricing puts would not test an additional property of the model. Only call options were priced. The strikes were taken from the filtered panel of Sec. 4.1.6, with moneyness between 0.5 and 2.0. The test split contains 20 expiry dates, one per month, with the first expiry on 2024-09-26 and the last on 2026-04-24. The valuation dates were chosen $h = 21$ trading days before each expiry ($h = 22$ on one date). On the valuation date, the front-month series still holds the preceding contract, so F_t and the moneyness are taken from the underlying contract of the option.

The rollout was repeated M times per valuation date and seed. The configuration for G was selected in Sec. 5.6, with each of the five seeds priced separately. The results are reported as mean and seed spread, following the convention of Chapter 5. The discount factor e^{-rT} of Sec. 2.4 was applied to the expected payoff. €STR, the euro short-term rate from the European Central Bank, was used as the risk-free rate (European Central Bank 2026). €STR is quoted as a simple annual rate, so it was converted to continuous compounding, $r = \ln(1 + r_{\text{€STR}})$. The rate on the valuation date was used for the whole life of the option. €STR was missing on two valuation dates that fall on bank holidays, so the last published rate was used instead. The valuation dates and discount rates are constructed in `build_pricing_dates.py`.

The simulated distribution does not depend on the strike, so one set of paths was used for all strikes of the corresponding expiry. Option prices depend on the distance between the strike and the settlement price on the valuation date, so prices at different strikes cannot be compared directly. Each option price was converted to an implied volatility, following the procedure described in Sec. 2.5. The inversion requires the option prices to lie within the bounds of Sec. 2.4. The correction of Eq. (36) does not move the simulated prices outside the bounds (Duan and Simonato 1995). Strikes that no simulated path reaches are priced at zero, and a price of zero carries no implied volatility. At strikes that every simulated path finishes above, the expected payoff after the correction is $F_t - K$, so the price equals the lower bound and no implied volatility exists. Increasing M to 50,000 changed the percentage of inverted strikes by about one percentage point.

The implied volatility for options is published by the exchange (Sec. 4.1.5), so the inversion procedure is not needed for the market prices. The pricing procedure is implemented in `price_options.py`.

4.6 Benchmark Models

This section introduces the models that the generated option prices are compared against: two benchmark models in three variants, a GBM and a historical simulation of the realized returns in two modes. Both benchmark models replaced G with a simpler model of the returns and were priced through the procedure shown in Fig. 7 without change. The benchmarks were run with a single seed each, and the results do not carry a seed spread.

GBM is the process underlying Black-76, introduced in Sec. 2.5. Under GBM the return over a short period is normally distributed and returns over different periods are independent. The price at a future date is lognormally distributed (Hull 2022, p. 332). The futures price is not expected to grow, so the log return from the valuation date to expiry is normally distributed

$$\ln \frac{F_T}{F_t} \sim \mathcal{N}\left(-\frac{1}{2}\sigma^2 T, \sigma^2 T\right), \quad (37)$$

where $-\frac{1}{2}\sigma^2 T$ and $\sigma^2 T$ are the mean and variance of the log return. $-\frac{1}{2}\sigma^2 T$ follows from the risk-neutral condition $\hat{\mathbb{E}}[F_T] = F_t$ of Sec. 2.4 (Hull 2022, p. 328). Each path of the GBM was simulated as h returns over the trading days to expiry,

$$\hat{x}_k = -\frac{1}{2}\sigma^2 \Delta + \sigma\sqrt{\Delta}\varepsilon_k, \quad \varepsilon_k \sim \mathcal{N}(0, 1), \quad (38)$$

where $\Delta = T/h$ is the time step and ε_k are independent standard normal random variables. The returns are drawn independently, so the variance of the sum of h returns is $h\sigma^2 \Delta = \sigma^2 T$.

GBM reproduces none of the stylized facts of Sec. 2.6. The returns are normally distributed, so the simulated distribution does not reproduce heavy tails or asymmetry. The returns are also drawn independently, so there is no volatility clustering. A single σ applies to every strike, so the volatility smile of the simulated prices is flat.

Two choices of σ are considered. The first is the implied volatility published by the exchange at the listed strike closest to the settlement price on the valuation date. The published σ is obtained by inverting Black-76 at that strike, so pricing the same strike returns the market price. The second choice is the annualized realized volatility of the front-month returns over the preceding 60 trading days. The realized volatility is computed from the return series alone, without option prices. The realized volatility lies below the implied volatility on most valuation dates. The results of Sec. 5.7 are reported with the implied volatility.

The historical simulation draws paths of h returns by resampling the observed returns, a procedure known as the bootstrap (Efron 1979). The returns are drawn from the training split of Sec. 4.1, so no test data enters the benchmark. Two modes of the bootstrap are considered. In the independent and identically distributed (iid) mode, each of the h returns is drawn independently with replacement, so the same observed return can enter a path more than once. No distributional assumption is made, since the returns are drawn from the observed distribution. In the block mode, each path is a block of h consecutive returns (Künsch 1989). The order of the observed series is preserved within each block, so the volatility clustering of Sec. 2.6 is present as well. The training split contains $3,197 - h + 1$ distinct blocks, so the 20,000 paths reuse each block multiple times. The paths are not independent, so the Monte Carlo standard error of Sec. 4.5 is too small in the block mode.

Table 2 states which stylized facts each model can reproduce by construction. How far the selected model G reproduces them in practice is reported in Chapter 5. The

selected model G is the only model conditioned on the returns preceding the valuation date. GBM and the iid bootstrap differ in the distribution of the returns, while both draw the returns independently. The difference between these two benchmarks is attributable to the heavy tails and the asymmetry of Sec. 2.6 jointly. The two bootstrap modes draw from the same returns and differ only in the order, so the difference between them is attributable to the volatility clustering.

Model	Stylized facts
GBM	none
Bootstrap, iid	heavy tails, asymmetry
Bootstrap, block	heavy tails, volatility clustering, asymmetry
Selected model G	heavy tails, volatility clustering, asymmetry

Table 2: Stylized facts the benchmarks and selected model G can reproduce by construction

Further benchmark models are considered and discussed in Chapter 6. The prices of the benchmarks and of the selected model G are reported and compared in Sec. 5.7. The benchmarks are implemented in `price_gbm.py` and `price_bootstrap.py`.

4.7 Software Environment

Experiments were run on a MacBook Pro with an Apple M4 Pro processor and 24 GB of RAM, running macOS 26.6.1. The NVIDIA CUDA toolkit is not supported on macOS. Training was on CPU. The software packages and their respective versions used for the data pipeline, model training, and evaluation are listed in Table 3.

Package	Version
Python	3.12.13
PyTorch	2.13.0
NumPy	2.5.2
pandas	3.0.5
polars	1.43.2
pyarrow	25.0.1
DuckDB	1.5.5
SciPy	1.18.1
Matplotlib	3.11.1
tqdm	4.70.0

Table 3: Software packages and versions

A fixed seed was set for the Python, NumPy, and PyTorch random number generators at the start of each run.

The TTF futures data from Bloomberg and the options data from Databento are licensed and cannot be distributed. The processing steps are documented in Sec. 4.1. The pipeline is reproducible, given access to the same sources.

The implementation is based on the reference code of Vuletić, Prenzel, et al. (2024)⁶ and is publicly available on GitHub.⁷

4.8 Summary of the Setup

The daily TTF front-month return series was constructed from the futures data and split chronologically into training, validation, and test splits. The options data was joined into a panel and restricted to call options in the moneyness band of 0.5 to 2.0 over the test split. The ForGAN architecture and the reference Fin-GAN implementation were adopted, with the data loading adapted to the daily TTF returns and the missing backward propagation added to the ForGAN baseline training loop. Ten cost function branches were trained, including the ForGAN baseline, trained on cross-entropy alone, and nine Fin-GAN branches with the economics-driven terms. Each branch was trained under ten configurations and one input scaling run, with five seeds each, and evaluated on the distributional statistics of the generated returns. The selected model generates price paths by recursive rollout. From these price paths, call options are priced by Monte Carlo simulation and converted into implied volatilities. A GBM and two bootstrap modes serve as benchmarks. Chapter 5 reports the results.

⁶<https://github.com/milenavuletic/Fin-GAN>

⁷<https://github.com/YevaGalstyan/ttf-fingan-thesis>

5 Experiments

This chapter reports the experimental configurations and results for the TTF return series. Table 4 names the models evaluated in this chapter.

Name	Meaning
ForGAN baseline	Trained on cross-entropy alone
Fin-GAN branches	The nine combinations of the economics-driven terms
Configurations 1–10	The hyperparameter settings of Table 1
Selected model G	Configuration 8 with the ForGAN baseline cost function, used for pricing in Sec. 5.7

Table 4: Evaluated models

All configurations are trained with five fixed seeds, and the values are reported as mean and seed spread. The seed range of a value is its mean plus or minus one seed spread. The difference between configurations or cost function branches is treated as separable if their seed ranges do not overlap. A generated value is treated as separable from the test split if its seed range does not contain the realized value. A configuration or cost function is treated as closer to the test split if its seed ranges are narrower and contain the realized value on more statistics.

Sec. 5.1 reports the properties of the realized TTF return series. Secs. 5.2–5.4 report the ForGAN baseline across training lengths, network capacities, condition windows, and input scaling. Sec. 5.5 reports the effect of the cost function terms under configuration 8. Sec. 5.6 selects the model for pricing, Sec. 5.7 reports the option prices, and Sec. 5.8 summarizes the results. Each section reports a defined slice of the full experimental grid from App. A.

5.1 Properties of the TTF Return Series

This section reports the statistical properties of the realized TTF return series, which are the reference values for the generated returns in the following sections. The daily log returns over the full sample are shown in Fig. 8. The three data splits of Sec. 4.1.4 are marked.

5 Experiments

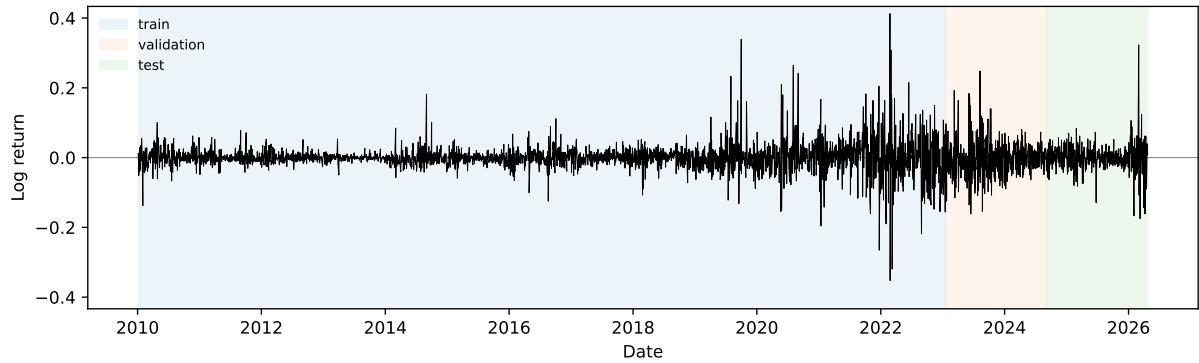


Figure 8: Daily TTF log returns with the train, validation, and test splits

Table 5 reports the distributional properties of the TTF return series, full sample and per data split.

	Full	Train	Validation	Test
n	3,997	3,197	399	401
Mean	0.0003	0.0005	-0.0014	0.0006
Standard deviation	0.0395	0.0377	0.0515	0.0401
Skewness	0.70	0.71	0.59	0.91
Excess kurtosis	14.91	19.08	2.53	13.24
Minimum	-0.3524	-0.3524	-0.1615	-0.1749
Maximum	0.4128	0.4128	0.2483	0.3229
1% quantile	-0.1153	-0.1129	-0.1248	-0.1291
99% quantile	0.1239	0.1224	0.1639	0.0885
$P(x > 0.05)$	0.117	0.099	0.253	0.130
$P(x > 0.10)$	0.030	0.027	0.058	0.025
$C(1)$	0.033	0.067	-0.096	0.008
$C_2(1)$	0.364	0.392	0.128	0.288
$C_2(2)$	0.280	0.310	0.206	0.061
$C_2(5)$	0.215	0.236	0.105	0.094
$C_2(10)$	0.112	0.117	0.148	0.011

Table 5: Distributional statistics and autocorrelation of daily TTF log returns, full sample and per data split

Heavy tails. The excess kurtosis of the full series is 14.91, compared to 0 for a normal distribution (Sec. 2.6). Absolute returns exceed 10 % on 3 % of trading days. The heavy tails are not distributed evenly across the data splits. The training split, which contains the 2021 to 2022 crisis, has an excess kurtosis of 19.08. The test split, which contains the

2026 disruption, has 13.24. Both episodes are described in Sec. 2.2. The validation split does not contain a comparable event and has an excess kurtosis of 2.53.

Asymmetry. The skewness of the full series is 0.70, indicating that large positive returns are more likely than large negative returns (Sec. 2.6). The skewness is positive for all data splits, with the training split at 0.71, the validation split at 0.59, and the test split at 0.91. As reported in Sec. 4.1.2, the transition days account for most of the positive skew. Skewness falls to 0.26 when transition days are excluded. The remaining asymmetry is consistent with the supply disruptions described in Sec. 2.2. Such events raise prices sharply, while no comparable mechanism lowers them. The positive skew is consistent with the right-skewed volatility smile reported for energy markets in Sec. 2.5.

Volatility clustering. The autocorrelation of returns at $\tau = 1$, Eq. (10), is 0.033 and stays close to 0 across all data splits. The autocorrelation of squared returns, Eq. (11), is 0.364 at $\tau = 1$, and decays slowly, to 0.280 at $\tau = 2$, 0.215 at $\tau = 5$, and 0.112 at $\tau = 10$. The slow decay is the volatility clustering described in Sec. 2.6. The training split decays smoothly as well, while the validation and test splits do not. In the validation split the autocorrelation rises from 0.128 at $\tau = 1$ to 0.206 at $\tau = 2$, falls to 0.105 at $\tau = 5$, and rises again to 0.148 at $\tau = 10$. In the test split it drops from 0.288 to 0.061 between $\tau = 1$ and $\tau = 2$, rises to 0.094 at $\tau = 5$, and falls to 0.011 at $\tau = 10$. The training split holds 3,197 of the 3,997 observations, so the decay of the full sample follows from the training split.

The heavy tails and the volatility clustering of the test split originate in the 2026 disruption described in Sec. 2.2. The test split is further examined with that period removed. Table 6 reports the distributional properties of the full test split and the test split excluding the period from 2026-02-01 to 2026-04-30. The excluded period covers 57 of the 401 trading days.

	Test	Test excluding 2026 disruption
n	401	344
Standard deviation	0.0401	0.0288
Skewness	0.91	-0.06
Excess kurtosis	13.24	1.57
$P(x > 0.10)$	0.025	0.006
$C_2(1)$	0.288	0.042

Table 6: Test split statistics with and without the 2026 disruption

When the 2026 disruption is excluded, the test split does not show heavy tails, volatility clustering, or asymmetry. The validation split has the highest standard deviation of all splits and contains no comparable single event. The returns of the validation split are uniformly large, while the returns of the test split are moderate outside a single episode. The tails of the return distribution determine the value of out-of-the-money options (Sec. 2.6). The excess kurtosis is 13.24 in the test split and 2.53 in the validation

split. For this reason, the generated returns are compared with the test split in the following sections. The model selection of Sec. 5.6 is based on it as well. The consequence of this choice is discussed in Sec. 6.2.

5.2 ForGAN Baseline: Training Length

This section reports how the training length affects the generated returns of the ForGAN baseline. Configurations 1 and 2 differ only in their training schedule. They are compared to the statistical properties of the test split from Sec. 5.1. Table 7 reports both configurations alongside the test split. The reference values were computed from the rows of the test window matrix. With $l = 10$, the 401 returns of the test split give $n_{\text{cond}} = 391$ rows. Mode collapse (Sec. 4.4) was not detected in any run of the experimental grid.

	Test	Configuration 1	Configuration 2
Mean	0.0006	-0.0003 ± 0.0019	-0.0029 ± 0.0019
Standard deviation	0.0404	0.0303 ± 0.0118	0.0365 ± 0.0034
Skewness	0.92	-0.18 ± 0.39	0.19 ± 0.58
Excess kurtosis	13.27	0.32 ± 0.42	4.51 ± 1.52
Minimum	-0.1749	-0.1533 ± 0.0474	-0.2234 ± 0.0287
Maximum	0.3229	0.1357 ± 0.0519	0.2516 ± 0.0761
1% quantile	-0.1305	-0.0739 ± 0.0251	-0.1074 ± 0.0162
99% quantile	0.0903	0.0684 ± 0.0272	0.1103 ± 0.0163
$P(x > 0.05)$	0.130	0.115 ± 0.124	0.129 ± 0.021
$P(x > 0.10)$	0.026	0.008 ± 0.016	0.026 ± 0.008
$C_2(1)$	0.289	-0.015 ± 0.056	0.125 ± 0.087
$C_2(2)$	0.061	-0.009 ± 0.036	0.129 ± 0.055
$C_2(5)$	0.094	-0.009 ± 0.035	0.164 ± 0.107
$C_2(10)$	0.012	0.018 ± 0.032	0.073 ± 0.056

Table 7: Distributional statistics and autocorrelation of generated daily TTF log returns on the test split, ForGAN baseline, configurations 1 and 2

Heavy tails. The excess kurtosis of the ForGAN baseline under configuration 1 is 0.32 ± 0.42 , compared to 13.27 for the test split. Under configuration 2, the excess kurtosis increases to 4.51 ± 1.52 , roughly a third of the test split value. The seed spread is large. However, the ranges of the two configurations do not overlap. The exceedance fractions of configuration 1 do not carry a reliable signal, as the seed spread is larger than the mean, 0.115 ± 0.124 for $P(|x| > 0.05)$. Under configuration 2 both exceedance fractions match the values of the test split, 0.129 ± 0.021 against 0.130 and 0.026 ± 0.008 against 0.026. The ForGAN baseline under configuration 2 generates large returns at the observed frequency, but does not generate the few extreme returns that increase the

excess kurtosis. The standard deviation of the ForGAN baseline under configuration 1 is 0.0303 ± 0.0118 , compared to 0.0404 for the test split. Under configuration 2, the standard deviation is 0.0365 ± 0.0034 , slightly below the test split value. Longer training increases the tail weight, but the excess kurtosis remains far below the test split.

Asymmetry. The skewness of the ForGAN baseline under configuration 1 is -0.18 ± 0.39 , negative where the test split is positive at 0.92. Under configuration 2, the skewness is 0.19 ± 0.58 , roughly a fifth of the test split value. For both configurations, the seed spread crosses zero, and the ranges of the two configurations overlap. As a result, the two configurations cannot be separated based on skewness, unlike the excess kurtosis. The minimum and maximum values provide additional information about the asymmetry. Under configuration 2 the minimum is -0.2234 ± 0.0287 and the maximum is 0.2516 ± 0.0761 , against -0.1749 and 0.3229 of the test split. The generated distribution extends further downward and less far upward than the test split. Neither configuration of the ForGAN baseline reproduces the skewness of the test split.

Volatility clustering. The autocorrelation of the ForGAN baseline under configuration 1 is -0.015 ± 0.056 for $\tau = 1$, compared to 0.289 for the test split. Therefore, the generated series shows no volatility clustering. Under configuration 2 the autocorrelation is 0.125 ± 0.087 for $\tau = 1$, roughly half of the test split value, indicating some volatility clustering. At $\tau = 2, 5, 10$ the autocorrelation under configuration 2 exceeds the test split, resulting in a flat autocorrelation. The generated series does not reproduce the autocorrelation decay described in Sec. 2.6. The test split provides a limited reference for the shape of the decay, since its autocorrelation does not decay smoothly either (Sec. 5.1).

Table 8 reports the same statistics for the MSE branch, the supervised counterpart of the ForGAN baseline, under configurations 1 and 2. The excess kurtosis of the MSE branch under configuration 1 is 0.19 ± 0.23 , against 0.32 ± 0.42 for the ForGAN baseline. Under configuration 2, the excess kurtosis of the MSE branch is 4.13 ± 1.20 , close to the 4.51 ± 1.52 of the ForGAN baseline. The two branches show similar patterns in the distributional properties. The main difference between these two branches is the skewness. The skewness of the MSE branch under configuration 2 is 0.39 ± 0.27 , compared to 0.19 ± 0.58 for the ForGAN baseline. The seed spread for MSE does not cross zero, so the MSE branch reproduces a positive skew, unlike the ForGAN baseline. However, the skewness under both configurations is still below the test split value of 0.92. The MSE branch also varies less across seeds than the ForGAN baseline on the distributional statistics, under both configurations.

	Test	Configuration 1	Configuration 2
Mean	0.0006	0.0025 ± 0.0026	-0.0011 ± 0.0020
Standard deviation	0.0404	0.0294 ± 0.0095	0.0354 ± 0.0011
Skewness	0.92	-0.07 ± 0.24	0.39 ± 0.27
Excess kurtosis	13.27	0.19 ± 0.23	4.13 ± 1.20
Minimum	-0.1749	-0.1349 ± 0.0300	-0.2151 ± 0.0396
Maximum	0.3229	0.1387 ± 0.0513	0.2592 ± 0.0469
1% quantile	-0.1305	-0.0674 ± 0.0175	-0.0976 ± 0.0083
99% quantile	0.0903	0.0709 ± 0.0248	0.1114 ± 0.0066
$P(x > 0.05)$	0.130	0.103 ± 0.102	0.124 ± 0.006
$P(x > 0.10)$	0.026	0.005 ± 0.010	0.024 ± 0.002
$C_2(1)$	0.289	0.040 ± 0.074	0.149 ± 0.063
$C_2(2)$	0.061	0.009 ± 0.056	0.141 ± 0.093
$C_2(5)$	0.094	0.007 ± 0.055	0.152 ± 0.088
$C_2(10)$	0.012	0.024 ± 0.041	0.134 ± 0.078

Table 8: Distributional statistics and autocorrelation of generated daily TTF log returns on the test split, MSE branch, configurations 1 and 2

Configuration 1, corresponding to Algorithm 1 of Vuletić, Prenzel, et al. (2024), leaves the model undertrained on the TTF futures dataset. The generated distribution is close to a normal distribution and shows no volatility clustering. Configuration 2, corresponding to the published implementation of Vuletić, Prenzel, et al. (2024), is a usable base configuration for training on the TTF futures dataset. However, the excess kurtosis and skewness remain well below the test split values. The two branches respond to the schedule in the same way, so the difference between the two configurations is a result of the training length and not the objective.

5.3 ForGAN Baseline: Network Capacity

This section reports how the network capacity affects the generated returns of the ForGAN baseline. The six configurations from Table 1 differ in the noise dimension N and the LSTM hidden sizes R_G and R_D . All six configurations follow the training length of configuration 2, so that the effect of capacity is isolated from the training schedule. Configuration 2, examined in Sec. 5.2, is carried over for reference. Table 9 holds the noise dimension at $N = 8$, and Table 10 at $N = 32$. In Table 9, configurations 2 and 4 differ only in R_D , while configurations 4 and 10 differ only in R_G . In Table 10, configurations 3 and 5 differ only in R_D , while configurations 5 and 8 differ only in R_G . Comparing corresponding columns across the two tables isolates N .

5 Experiments

	Test	Configuration 2	Configuration 4	Configuration 10
Mean	0.0006	-0.0029 ± 0.0019	-0.0002 ± 0.0014	0.0025 ± 0.0022
Standard deviation	0.0404	0.0365 ± 0.0034	0.0369 ± 0.0029	0.0368 ± 0.0022
Skewness	0.92	0.19 ± 0.58	-0.09 ± 0.38	-0.14 ± 0.61
Excess kurtosis	13.27	4.51 ± 1.52	7.95 ± 1.68	7.85 ± 3.39
Minimum	-0.1749	-0.2234 ± 0.0287	-0.3212 ± 0.0454	-0.2772 ± 0.0539
Maximum	0.3229	0.2516 ± 0.0761	0.3254 ± 0.0532	0.2850 ± 0.0803
1% quantile	-0.1305	-0.1074 ± 0.0162	-0.1228 ± 0.0128	-0.1160 ± 0.0179
99% quantile	0.0903	0.1103 ± 0.0163	0.1177 ± 0.0184	0.1131 ± 0.0106
$P(x > 0.05)$	0.130	0.129 ± 0.021	0.120 ± 0.012	0.112 ± 0.015
$P(x > 0.10)$	0.026	0.026 ± 0.008	0.033 ± 0.010	0.027 ± 0.005
$C_2(1)$	0.289	0.125 ± 0.087	0.057 ± 0.071	0.206 ± 0.086
$C_2(2)$	0.061	0.129 ± 0.055	0.123 ± 0.124	0.310 ± 0.174
$C_2(5)$	0.094	0.164 ± 0.107	0.120 ± 0.092	0.223 ± 0.147
$C_2(10)$	0.012	0.073 ± 0.056	0.079 ± 0.038	0.057 ± 0.045

Table 9: Distributional statistics and autocorrelation of generated daily TTF log returns on the test split, ForGAN baseline, configurations 2, 4, and 10

	Test	Configuration 3	Configuration 5	Configuration 8
Mean	0.0006	-0.0016 ± 0.0029	0.0001 ± 0.0018	0.0019 ± 0.0034
Standard deviation	0.0404	0.0353 ± 0.0035	0.0354 ± 0.0031	0.0369 ± 0.0036
Skewness	0.92	0.02 ± 0.68	0.41 ± 0.68	0.54 ± 0.88
Excess kurtosis	13.27	4.70 ± 1.53	7.04 ± 1.83	10.12 ± 2.59
Minimum	-0.1749	-0.2326 ± 0.0675	-0.2746 ± 0.0978	-0.2509 ± 0.0482
Maximum	0.3229	0.2478 ± 0.0444	0.3497 ± 0.0556	0.3742 ± 0.0535
1% quantile	-0.1305	-0.1013 ± 0.0213	-0.1007 ± 0.0247	-0.1055 ± 0.0304
99% quantile	0.0903	0.1020 ± 0.0167	0.1130 ± 0.0101	0.1185 ± 0.0258
$P(x > 0.05)$	0.130	0.121 ± 0.022	0.110 ± 0.015	0.104 ± 0.016
$P(x > 0.10)$	0.026	0.021 ± 0.007	0.025 ± 0.009	0.027 ± 0.006
$C_2(1)$	0.289	0.159 ± 0.097	0.121 ± 0.111	0.263 ± 0.201
$C_2(2)$	0.061	0.121 ± 0.028	0.116 ± 0.099	0.170 ± 0.113
$C_2(5)$	0.094	0.105 ± 0.098	0.132 ± 0.127	0.167 ± 0.112
$C_2(10)$	0.012	0.074 ± 0.035	0.043 ± 0.033	0.144 ± 0.103

Table 10: Distributional statistics and autocorrelation of generated daily TTF log returns on the test split, ForGAN baseline, configurations 3, 5, and 8

Discriminator hidden size. At both values of the noise dimension N , the discriminator hidden size R_D has a consistent effect on the excess kurtosis of the generated

returns. Increasing R_D from 8 to 64 raises the excess kurtosis from 4.51 ± 1.52 under configuration 2 to 7.95 ± 1.68 under configuration 4 at $N = 8$, and the seed ranges of the two configurations do not overlap. At $N = 32$, the excess kurtosis rises from 4.70 ± 1.53 under configuration 3 to 7.04 ± 1.83 under configuration 5, where the seed ranges of the two configurations overlap. Both values remain below the 13.27 of the test split. Increasing R_D also affects the largest negative returns. The minimum of the generated returns reaches -0.3212 ± 0.0454 under configuration 4, while the minimum of the test split is -0.1749 . The 1% quantile, however, moves from -0.1074 ± 0.0162 to -0.1228 ± 0.0128 with overlapping seed ranges. Both of these values remain above the 1% quantile of the test split, -0.1305 . As a result, increasing R_D lowers the minimum of the generated returns, while the level below which 1% of the returns fall is not separable from the seed variation.

Generator hidden size. The generator hidden size R_G has no separable effect on the excess kurtosis of the generated returns at either noise dimension. At $N = 8$, increasing R_G from 8 to 64 changes the excess kurtosis from 7.95 ± 1.68 under configuration 4 to 7.85 ± 3.39 under configuration 10. The seed spread doubles, from 1.68 to 3.39, so the larger generator makes the training less stable without changing the mean. At $N = 32$, increasing R_G from 8 to 64 raises the excess kurtosis from 7.04 ± 1.83 under configuration 5 to 10.12 ± 2.59 under configuration 8. However, the seed ranges of the two configurations overlap. As noted in Sec. 4.3, R_G is not varied in Vuletić, Prenzel, et al. (2024) or in the published implementation, so this comparison has no counterpart in the equity results.

Noise dimension. The noise dimension N has no separable effect on the excess kurtosis at any of the three hidden size combinations. At $R_G = R_D = 8$, increasing N from 8 to 32 changes the excess kurtosis from 4.51 ± 1.52 under configuration 2 to 4.70 ± 1.53 under configuration 3, with overlapping seed ranges. At $R_G = 8$ and $R_D = 64$, the excess kurtosis moves from 7.95 ± 1.68 under configuration 4 to 7.04 ± 1.83 under configuration 5, again with overlapping seed ranges. At $R_G = R_D = 64$, the excess kurtosis increases from 7.85 ± 3.39 under configuration 10 to 10.12 ± 2.59 under configuration 8, again with overlapping seed ranges.

Configuration 8. Configuration 8 has the highest excess kurtosis of the six configurations, 10.12 ± 2.59 , and comes closest to the 13.27 of the test split. The standard deviation is 0.0369 ± 0.0036 under configuration 8, and its seed range contains the 0.0404 of the test split. The maximum return is 0.3742 ± 0.0535 , and its seed range contains the 0.3229 of the test split. The probability of large returns, $P(|x| > 0.10)$, is 0.027 ± 0.006 , matching the 0.026 for the test split. The skewness is 0.54 ± 0.88 , against the 0.92 of the test split, with a seed range that crosses zero. Configuration 8 does not separate from the test split on the standard deviation, the maximum, or the skewness, and remains below it on the excess kurtosis.

Volatility clustering. None of the six configurations reproduces the decay of $C_2(\tau)$ observed in the test split. The autocorrelation $C_2(1)$ ranges from 0.057 ± 0.071 to 0.263 ± 0.201 across the six configurations, against the 0.289 of the test split. The autocorrelation $C_2(2)$ ranges from 0.116 ± 0.099 to 0.310 ± 0.174 , above the 0.061 of the test split. The

values are shown in Fig. 9. The seed spreads and the means of $C_2(\tau)$ are of the same order throughout both tables, so configurations cannot be separated from one another. As noted in Sec. 5.2, the test split is a limited reference for the shape of the decay; its own autocorrelation does not decay smoothly either. Configuration 8 is the closest to the test split at $\tau = 1$, at 0.263 ± 0.201 against 0.289, but its $C_2(\tau)$ remains near that level at $\tau = 2, 5$, and 10.

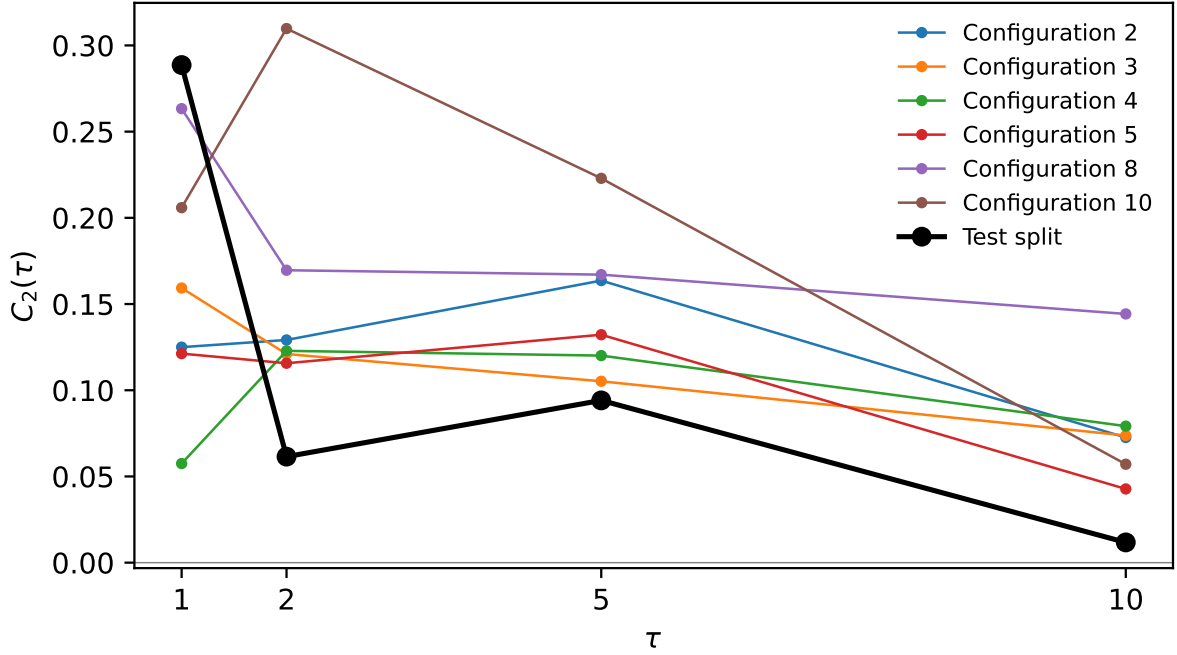


Figure 9: Autocorrelation of squared returns $C_2(\tau)$ for the six configurations and the test split

Training length at higher capacity. The training length is examined further at the higher capacity settings, so the schedule of configuration 2 is no longer held fixed. Table 11 reports the generated returns for the ForGAN baseline under configurations 7 and 9. Configurations 7 and 9 are configurations 5 and 8, respectively, with $n_{\text{epochs}} = 1500$.

The excess kurtosis of the generated returns increases from 7.04 ± 1.83 under configuration 5 to 7.97 ± 1.22 under configuration 7. The seed ranges overlap, so the difference in the excess kurtosis is not separable. The skewness changes from 0.41 ± 0.68 to 0.04 ± 0.30 , both crossing zero. The seed spreads are smaller under configuration 7 throughout the distributional statistics. As a result, a longer training schedule with a generator hidden size of $R_G = 8$ stabilizes the runs without moving them.

The excess kurtosis of the generated returns increases from 10.12 ± 2.59 under configuration 8 to 13.84 ± 8.77 under configuration 9. The seed ranges overlap, so the difference in the excess kurtosis is not separable. However, the mean of the excess kurtosis under configuration 9 reaches the 13.27 of the test split. The seed spread more than triples,

from 2.59 to 8.77, and is the largest of all configurations in this section. The skewness changes from 0.54 ± 0.88 to 1.78 ± 0.99 , the only skewness range among the eight configurations that does not cross zero. The skewness under configuration 9 overshoots the 0.92 of the test split by roughly a factor of two. The maximum generated return is 0.4608 ± 0.1127 , and its seed range excludes the 0.3229 of the test split. The exceedance fraction $P(|x| > 0.10)$ is 0.040 ± 0.010 , with a seed range excluding the 0.026 of the test split. The standard deviation is 0.0436 ± 0.0054 and its seed range contains the 0.0404 of the test split. Configuration 9 reaches the tail weight of the test split on average and produces a reliable positive skew. However, it overshoots separably on the maximum and on $P(|x| > 0.10)$, and is the least stable of all configurations in this section.

	Test	Configuration 7	Configuration 9
Mean	0.0006	-0.0004 ± 0.0011	0.0031 ± 0.0022
Standard deviation	0.0404	0.0368 ± 0.0018	0.0436 ± 0.0054
Skewness	0.92	0.04 ± 0.30	1.78 ± 0.99
Excess kurtosis	13.27	7.97 ± 1.22	13.84 ± 8.77
Minimum	-0.1749	-0.3178 ± 0.0355	-0.2355 ± 0.0246
Maximum	0.3229	0.3545 ± 0.0445	0.4608 ± 0.1127
1% quantile	-0.1305	-0.1166 ± 0.0129	-0.1085 ± 0.0082
99% quantile	0.0903	0.1163 ± 0.0084	0.1566 ± 0.0316
$P(x > 0.05)$	0.130	0.114 ± 0.007	0.137 ± 0.012
$P(x > 0.10)$	0.026	0.030 ± 0.004	0.040 ± 0.010
$C_2(1)$	0.289	0.153 ± 0.070	0.160 ± 0.084
$C_2(2)$	0.061	0.118 ± 0.088	0.199 ± 0.110
$C_2(5)$	0.094	0.143 ± 0.100	0.115 ± 0.059
$C_2(10)$	0.012	0.090 ± 0.058	0.052 ± 0.042

Table 11: Distributional statistics and autocorrelation of generated daily TTF log returns on the test split, ForGAN baseline, configurations 7 and 9

Increasing R_D from 8 to 64 is the only capacity change whose effect holds across the two tables. The generator hidden size R_G and the noise dimension N have no separable effect on the generated returns. Of the six configurations with a fixed training schedule, configuration 8 is the closest to the test split. However, it remains below the test split on the excess kurtosis. A longer training schedule does not produce a separable change at $R_G = 8$. At $R_G = 64$, a longer training schedule brings the mean of the excess kurtosis to the test split value and produces a positive skew, at the cost of stability across seeds. An increase in capacity alone does not close the gap. However, capacity combined with a longer schedule reaches the tail weight of the test split without reproducing its distribution. The condition window and the input scaling are examined in Sec. 5.4.

5.4 ForGAN Baseline: Condition Window and Input Scaling

This section reports how the condition window length and the input scaling affect the generated returns of the ForGAN baseline. Configuration 5 is the shared baseline against which the two other configurations are compared. The full-split scaling configuration is configuration 5 with the standardization applied to the full training split rather than the reference subset of $n_{\text{batch}} = 100$. Configuration 6 is configuration 5 with a longer condition window of $l = 30$. With $l = 30$ the 401 returns of the test split give $n_{\text{cond}} = 371$ rows instead of 391. As a result, the reference values of the test split differ in Table 12, and the values of configuration 5 are reported in Table 13.

	Test	Configuration 6
Mean	0.0003	-0.0006 ± 0.0026
Standard deviation	0.0408	0.0342 ± 0.0019
Skewness	0.96	0.15 ± 0.45
Excess kurtosis	13.43	3.74 ± 1.00
Minimum	-0.1749	-0.1893 ± 0.0538
Maximum	0.3229	0.2314 ± 0.0248
1% quantile	-0.1333	-0.0935 ± 0.0062
99% quantile	0.0939	0.0930 ± 0.0163
$P(x > 0.05)$	0.129	0.119 ± 0.012
$P(x > 0.10)$	0.027	0.016 ± 0.006
$C_2(1)$	0.288	0.191 ± 0.086
$C_2(2)$	0.060	0.151 ± 0.107
$C_2(5)$	0.093	0.125 ± 0.048
$C_2(10)$	0.011	0.111 ± 0.107

Table 12: Distributional statistics and autocorrelation of generated daily TTF log returns on the test split, ForGAN baseline, configuration 6

Condition window. The excess kurtosis of the ForGAN baseline decreases from 7.04 ± 1.83 under configuration 5 to 3.74 ± 1.00 under configuration 6, compared to the 13.43 of the test split. The difference is separable, since the seed ranges of the two configurations do not overlap. Configurations 5 and 6 are measured against different reference values of the test split, so the comparison between them is approximate. However, the two reference values differ by 0.16, while the excess kurtosis falls by 3.30, so the difference in reference cannot account for the effect. The maximum return is 0.2314 ± 0.0248 and its seed range does not include the 0.3229 of the test split. Both exceedance fractions fall short of the test split, 0.119 ± 0.012 against 0.129 for $P(|x| > 0.05)$ and 0.016 ± 0.006 against 0.027 for $P(|x| > 0.10)$. The skewness is 0.15 ± 0.45 and crosses zero. Configuration 6 is the furthest from the test split of the configurations trained under the schedule

of configuration 2. Increasing the condition window from $l = 10$ to $l = 30$ reduces the tail weight of the generated returns.

Input scaling. The standard deviation is 0.0354 ± 0.0031 under configuration 5 and 0.0338 ± 0.0018 under the full-split scaling configuration, with overlapping seed ranges. The excess kurtosis decreases from 7.04 ± 1.83 under configuration 5 to 5.55 ± 2.52 under the full-split scaling configuration. The seed ranges of the two configurations overlap, so the difference is not separable. The skewness for the full-split scaling configuration is 0.30 ± 0.36 , compared to 0.41 ± 0.68 for configuration 5, and both seed ranges cross zero. Computing the scaling parameters from the full training split has no separable effect on the generated returns.

	Test	Configuration 5	Full-split scaling
Mean	0.0006	0.0001 ± 0.0018	-0.0001 ± 0.0022
Standard deviation	0.0404	0.0354 ± 0.0031	0.0338 ± 0.0018
Skewness	0.92	0.41 ± 0.68	0.30 ± 0.36
Excess kurtosis	13.27	7.04 ± 1.83	5.55 ± 2.52
Minimum	-0.1749	-0.2746 ± 0.0978	-0.2350 ± 0.0682
Maximum	0.3229	0.3497 ± 0.0556	0.3091 ± 0.0713
1% quantile	-0.1305	-0.1007 ± 0.0247	-0.0953 ± 0.0119
99% quantile	0.0903	0.1130 ± 0.0101	0.1002 ± 0.0123
$P(x > 0.05)$	0.130	0.110 ± 0.015	0.109 ± 0.018
$P(x > 0.10)$	0.026	0.025 ± 0.009	0.019 ± 0.006
$C_2(1)$	0.289	0.121 ± 0.111	0.121 ± 0.121
$C_2(2)$	0.061	0.116 ± 0.099	0.133 ± 0.125
$C_2(5)$	0.094	0.132 ± 0.127	0.128 ± 0.135
$C_2(10)$	0.012	0.043 ± 0.033	0.109 ± 0.076

Table 13: Distributional statistics and autocorrelation of generated daily TTF log returns on the test split, ForGAN baseline, configuration 5 with reference and full-split input scaling

Configuration 6 differs separably from configuration 5 in the excess kurtosis. However, it reduces the tail weight instead of increasing it. The full-split scaling configuration shows no separable effect from configuration 5 in the standard deviation, the excess kurtosis, or the skewness. Neither variation of configuration 5 improves the distributional properties of the generated returns. The effect of the cost function terms is examined in Sec. 5.5.

5.5 Fin-GAN Branches: Effect of the Cost Function Terms

This section reports how the economics-driven terms of the Fin-GAN cost function affect the generated returns. The generated returns for the ten cost function branches of

5 Experiments

Sec. 4.3 are reported in Tables 14, 15, and 16. All ten branches are trained under configuration 8 to isolate the effect of the cost function. As reported in Sec. 5.3, configuration 8 is the closest to the test split among the configurations that remain stable across seeds. Table 14 reports the single cost function terms. Tables 15 and 16 report the combinations of cost function terms, separated by the presence of the SR^* term. The ForGAN baseline is reported as a reference in all three tables. The full results for all configurations and cost function branches are given in App. A.

	ForGAN	MSE	PnL_a^*	SR^*
Mean	0.0019±0.0034	0.0006±0.0023	0.0004±0.0030	0.0010±0.0005
Standard deviation	0.0369±0.0036	0.0377±0.0024	0.0414±0.0030	0.0341±0.0036
Skewness	0.54±0.88	0.23±1.29	0.93±0.30	-0.59±0.16
Excess kurtosis	10.12±2.59	10.13±4.39	12.03±5.45	8.08±2.72
Minimum	-0.2509±0.0482	-0.2644±0.0655	-0.2542±0.0468	-0.2612±0.0373
Maximum	0.3742±0.0535	0.3343±0.1207	0.3950±0.0887	0.2268±0.0474
1 % quantile	-0.1055±0.0304	-0.1155±0.0144	-0.1297±0.0187	-0.0913±0.0171
99 % quantile	0.1185±0.0258	0.1200±0.0257	0.1401±0.0235	0.0945±0.0113
$P(x > 0.05)$	0.104±0.016	0.106±0.006	0.119±0.010	0.100±0.025
$P(x > 0.10)$	0.027±0.006	0.030±0.007	0.036±0.005	0.017±0.005
$C_2(1)$	0.263±0.201	0.165±0.122	0.226±0.077	0.229±0.039
$C_2(2)$	0.170±0.113	0.181±0.097	0.185±0.144	0.377±0.156
$C_2(5)$	0.167±0.112	0.183±0.069	0.182±0.072	0.158±0.022
$C_2(10)$	0.144±0.103	0.112±0.114	0.153±0.134	0.060±0.061

Table 14: Distributional statistics and autocorrelation of generated daily TTF log returns on the test split, the single cost function terms, configuration 8

Heavy tails. None of the nine cost function branches is separable from ForGAN on excess kurtosis, as all ten seed ranges overlap. The mean excess kurtosis of all branches is below the 13.27 of the test split. The lowest excess kurtosis is 6.78 ± 1.29 for the PnL_a^* & MSE & STD branch and the highest is 12.03 ± 5.45 for the PnL_a^* branch, compared to 10.12 ± 2.59 of the ForGAN baseline. The seed ranges of the MSE, PnL_a^* , and PnL_a^* & MSE branches contain the 13.27 of the test split, while every other branch falls short. The standard deviation is the lowest for the SR^* & MSE branch at 0.0331 ± 0.0034 and the highest for the PnL_a^* branch at 0.0414 ± 0.0030 . The seed ranges of the ForGAN baseline, PnL_a^* , and PnL_a^* & MSE branches contain the 0.0404 of the test split, while every other branch falls short. The seed range of $P(|x| > 0.10)$ is separable from ForGAN only for the SR^* & MSE branch at 0.016 ± 0.004 against 0.027 ± 0.006 .

5 Experiments

	ForGAN	PnL _a [*] & MSE	PnL _a [*] & STD	PnL _a [*] & MSE & STD
Mean	0.0019±0.0034	0.0009±0.0020	−0.0007±0.0032	0.0012±0.0009
Standard deviation	0.0369±0.0036	0.0397±0.0032	0.0356±0.0017	0.0353±0.0032
Skewness	0.54±0.88	0.81±0.46	−0.09±0.95	1.11±0.42
Excess kurtosis	10.12±2.59	10.69±2.97	8.21±3.68	6.78±1.29
Minimum	−0.2509±0.0482	−0.2401±0.0302	−0.2821±0.0737	−0.1979±0.0157
Maximum	0.3742±0.0535	0.3892±0.0618	0.3181±0.0776	0.3562±0.0312
1 % quantile	−0.1055±0.0304	−0.1186±0.0200	−0.1114±0.0101	−0.0882±0.0114
99 % quantile	0.1185±0.0258	0.1273±0.0177	0.1081±0.0202	0.1236±0.0133
$P(x > 0.05)$	0.104±0.016	0.119±0.019	0.109±0.008	0.112±0.021
$P(x > 0.10)$	0.027±0.006	0.032±0.005	0.025±0.005	0.025±0.007
$C_2(1)$	0.263±0.201	0.215±0.049	0.281±0.130	0.234±0.122
$C_2(2)$	0.170±0.113	0.183±0.126	0.201±0.092	0.124±0.127
$C_2(5)$	0.167±0.112	0.239±0.132	0.129±0.041	0.081±0.110
$C_2(10)$	0.144±0.103	0.129±0.068	0.125±0.075	0.067±0.070

Table 15: Distributional statistics and autocorrelation of generated daily TTF log returns on the test split, the combinations without the SR^{*} term, configuration 8

	ForGAN	PnL _a [*] & SR [*]	SR [*] & MSE	PnL _a [*] & MSE & SR [*]
Mean	0.0019±0.0034	0.0012±0.0020	0.0007±0.0011	0.0007±0.0014
Standard deviation	0.0369±0.0036	0.0351±0.0031	0.0331±0.0034	0.0338±0.0034
Skewness	0.54±0.88	−0.64±0.20	−0.57±0.20	−0.60±0.24
Excess kurtosis	10.12±2.59	8.03±2.55	8.02±2.42	7.86±2.93
Minimum	−0.2509±0.0482	−0.2638±0.0303	−0.2553±0.0310	−0.2570±0.0406
Maximum	0.3742±0.0535	0.2207±0.0322	0.2154±0.0369	0.2231±0.0509
1 % quantile	−0.1055±0.0304	−0.0938±0.0122	−0.0889±0.0153	−0.0910±0.0147
99 % quantile	0.1185±0.0258	0.0959±0.0122	0.0921±0.0089	0.0930±0.0088
$P(x > 0.05)$	0.104±0.016	0.108±0.021	0.094±0.022	0.100±0.023
$P(x > 0.10)$	0.027±0.006	0.017±0.006	0.016±0.004	0.016±0.005
$C_2(1)$	0.263±0.201	0.223±0.089	0.202±0.059	0.225±0.103
$C_2(2)$	0.170±0.113	0.382±0.146	0.274±0.111	0.345±0.134
$C_2(5)$	0.167±0.112	0.163±0.075	0.153±0.025	0.147±0.036
$C_2(10)$	0.144±0.103	0.056±0.047	0.076±0.061	0.057±0.038

Table 16: Distributional statistics and autocorrelation of generated daily TTF log returns on the test split, the combinations with the SR^{*} term, configuration 8

Asymmetry. The seed ranges of all four branches containing the SR^{*} term are separable from ForGAN on skewness, and all four are negative (Fig. 10). The skewness is $−0.59±0.16$ for the SR^{*} branch, and the three branches containing SR^{*} are within 0.05 of it, against the $0.54±0.88$ of the ForGAN baseline and the 0.92 of the test split. The mean

5 Experiments

skewness of the PnL_a^* & STD branch is -0.09 ± 0.95 , but its seed range crosses zero and overlaps the ForGAN baseline. The highest skewness is 1.11 ± 0.42 for the PnL_a^* & MSE & STD branch. The seed ranges of the ForGAN baseline and of the MSE, PnL_a^* , PnL_a^* & MSE, and PnL_a^* & MSE & STD branches contain the 0.92 of the test split.

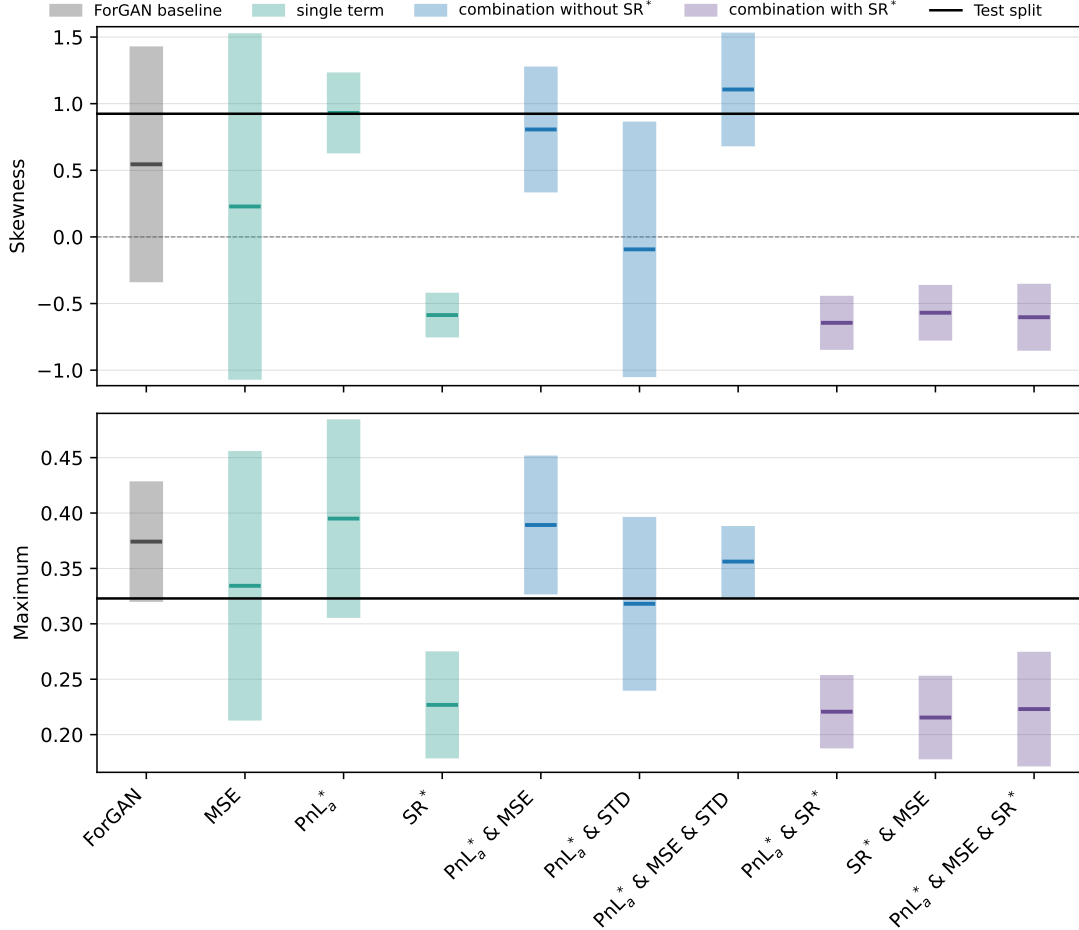


Figure 10: Skewness and maximum of the generated returns, the ten cost function branches, configuration 8

The SR^* economics-driven term determines the sign of the skewness regardless of what it is combined with. The skewness is 0.93 ± 0.30 for the PnL_a^* branch alone, compared to the -0.64 ± 0.20 of the PnL_a^* & SR^* branch. As defined in Eq. (29), SR^* is the mean of the PnL_a^* series divided by its standard deviation. A forecast with p_i far above the others increases the numerator and the denominator. As reported in Sec. 5.1, large positive returns are more likely than large negative ones in the TTF series. As a result, the forecasts made on those days can decrease SR^* , while they always increase PnL_a^* .

The seed ranges of the maximum of all four branches containing SR^* are separable from the 0.3742 ± 0.0535 of the ForGAN baseline, and all four are lower. The maximum

is 0.2154 ± 0.0369 for the SR^* & MSE branch, and the other three are within 0.012 of it. The seed ranges of the ForGAN baseline and of the MSE, PnL_a^* , and PnL_a^* & STD branches contain the 0.3229 of the test split. The maximum of the PnL_a^* & MSE branch is 0.3892 ± 0.0618 , above the test split. The seed ranges of the 99 % quantile of all four branches containing SR^* and of the PnL_a^* & STD branch contain the 0.0903 of the test split. The ForGAN baseline exceeds it across all seeds. The seed ranges of the 1 % quantile of the ForGAN baseline and of the PnL_a^* and PnL_a^* & MSE branches contain the -0.1305 of the test split. All ten branches lie below the minimum of the test split, -0.1749 . The closest is the PnL_a^* & MSE & STD branch at -0.1979 ± 0.0157 .

The seed ranges of the PnL_a^* branch contain the test split on the skewness, the maximum, and the 1 % quantile, as does the ForGAN baseline. The SR^* term compresses the largest generated returns, which brings the 99 % quantile closer to the test split but moves the skewness and the maximum away.

Volatility clustering. None of the ten branches reproduces the decay of $C_2(\tau)$ observed in the test split. The autocorrelation $C_2(1)$ ranges from 0.165 ± 0.122 to 0.281 ± 0.130 across the ten branches, against the 0.289 of the test split. The seed spreads are of the same order as the means, so the branches are not separable. The four branches containing SR^* rise from $C_2(1)$ between 0.202 and 0.229 to $C_2(2)$ between 0.274 and 0.382, where the test split falls to 0.061. The seed spreads of the four branches at $\tau = 2$ are between 0.111 and 0.156, so they do not separate from the other branches.

The SR^* , PnL_a^* & SR^* , SR^* & MSE, and PnL_a^* & MSE & SR^* branches are the only ones separable from ForGAN on the skewness and the maximum. The PnL_a^* and PnL_a^* & MSE branches come closest to the test split overall. Their seed ranges contain the test split values on the skewness, the excess kurtosis, the standard deviation, and the 1 % quantile. Overall, no economics-driven term separably improves on the ForGAN baseline. The effect of the cost function on the option prices is examined in Sec. 5.7.

5.6 Model Selection

This section selects the configuration and the cost function branch used for pricing. Configuration 8 is selected, together with the ForGAN baseline cost function. As listed in Table 1, configuration 8 uses a condition window of $l = 10$, a noise dimension of $N = 32$, generator and discriminator hidden sizes of $R_G = R_D = 64$, and $n_{\text{grad}} + n_{\text{epochs}} = 100 + 500$.

Network capacity. Configuration 9 is closer to the test split than configuration 8 on the excess kurtosis and the skewness. Its skewness seed range is also the only one that does not cross zero. However, the seed spread of configuration 9 on the excess kurtosis is more than three times as wide as the seed spread of configuration 8. Its maximum exceeds the test split, while the seed range of configuration 8 contains it (Sec. 5.3). Reproducibility across seeds is taken as the selection criterion, not proximity of the mean. As a result, configuration 8 is chosen over configuration 9.

Cost function. No branch is separable from ForGAN, so the separability criterion cannot distinguish between them. Selection among the branches is based on reproducibility across seeds, following the network capacity decision of Sec. 5.3. As reported in Sec. 5.5, under configuration 8 the seed ranges of the PnL_a^* and $\text{PnL}_a^* \& \text{MSE}$ branches contain the test split on the skewness, the excess kurtosis, the standard deviation, and the 1 % quantile. The seed range of the PnL_a^* branch contains the maximum of the test split. The $\text{PnL}_a^* \& \text{MSE}$ branch exceeds it across all seeds. The seed range of the PnL_a^* branch is closer to the test split on the standard deviation than the ForGAN baseline, and its skewness seed range is also narrower. However, the seed spread of PnL_a^* on excess kurtosis is the widest of the ten branches, and the spread of $\text{PnL}_a^* \& \text{MSE}$ is wider than the spread of the ForGAN baseline. The ForGAN baseline is chosen over the PnL_a^* and $\text{PnL}_a^* \& \text{MSE}$ branches. The selected model generates the price paths of Sec. 5.7.

5.7 Option Prices

This section reports the option prices generated by the selected model and compares them with the market and the benchmarks of Sec. 4.6. The price paths were generated with the ForGAN baseline model under configuration 8, selected in Sec. 5.6, and priced through the procedure described in Sec. 4.5. Option prices depend on the distance between the strike and the settlement price on the valuation date, so prices at different strikes are not comparable. The models are compared through the implied volatility. The error of a strike is the absolute difference between the two implied volatilities,

$$e_{d,k} = |\hat{\sigma}_{d,k} - \sigma_{d,k}|, \quad (39)$$

where $\hat{\sigma}_{d,k}$ is the implied volatility of the simulated price for strike k on valuation date d , and $\sigma_{d,k}$ is the implied volatility published by the exchange. The error of a model is the mean of the $e_{d,k}$ of a valuation date, averaged over 20 valuation dates,

$$e = \frac{1}{20} \sum_{d=1}^{20} \frac{1}{n_d} \sum_{k=1}^{n_d} e_{d,k}, \quad (40)$$

where n_d is the number of strikes with an implied volatility on valuation date d . Every valuation date carries the same weight, independently of the number of strikes. The strikes with no implied volatility are excluded from both errors, and the percentage of inverted strikes is reported alongside. The signed error s is Eq. (40) computed without the absolute value of Eq. (39). The error measures the size of the deviation from the market, and the signed error its direction. The results are reported in Table 17. The selected model was priced with each of the five seeds separately and is reported as mean and seed spread. The benchmarks carry a single seed each (Sec. 4.6).

The block bootstrap has the lowest e at 0.119, followed by the iid bootstrap at 0.142 and GBM at 0.149. The prices generated by G have the highest e at 0.256, with a seed spread of 0.043. The seed range does not overlap with any of the benchmarks. The

signed error is negative for all models. For the block bootstrap it is -0.012 , so the deviations are of both signs and cancel each other out. The signed error of G is -0.226 ± 0.057 , and negative on every seed, so the generated prices lie below the market. G also inverts the fewest strikes, at 81.0 % against 100 % for both bootstrap modes.

Model	e	s	Inverted (%)
GBM	0.149	-0.128	86.9
Bootstrap, iid	0.142	-0.091	100.0
Bootstrap, block	0.119	-0.012	100.0
G	0.256 ± 0.043	-0.226 ± 0.057	81.0 ± 13.1

Table 17: Implied volatility error, signed error, and percentage of inverted strikes

The error of Table 17 does not show where the model deviates from the market across different strikes, since it is aggregated over all strikes. The strikes were grouped into four ranges of moneyness within the band of Sec. 4.1.6, and e was computed within each range. The in-the-money strikes cover moneyness 0.5 to 0.9 and the at-the-money strikes 0.9 to 1.1. The out-of-the-money strikes cover moneyness 1.1 to 2.0 and were split at 1.5, since the range is wider than the other two. The results are reported in Table 18.

Model	0.5 to 0.9	0.9 to 1.1	1.1 to 1.5	1.5 to 2.0
GBM	0.138	0.021	0.160	0.330
Bootstrap, iid	0.107	0.147	0.140	0.214
Bootstrap, block	0.081	0.140	0.147	0.146
G	0.249 ± 0.107	0.242 ± 0.064	0.314 ± 0.067	0.348 ± 0.108

Table 18: Implied volatility error by moneyness

GBM has the lowest error for at-the-money strikes, at 0.021. The implied volatility used to price GBM is the listed implied volatility at the strike closest to the settlement price on the valuation date (Sec. 4.6). As a result, the at-the-money strikes match the market by construction and the value does not measure the accuracy of the model. The in-the-money strikes have an error of 0.138, the out-of-the-money strikes 0.160, and the far out-of-the-money strikes 0.330. A single σ produces the same implied volatility at every strike, while the market implied volatility varies across strikes. The error of the block bootstrap ranges between 0.081 for the in-the-money strikes and 0.147 for the out-of-the-money strikes. The iid bootstrap has a higher error for the far out-of-the-money strikes, at 0.214. The order of returns is the only difference between the two bootstrap modes (Sec. 4.6), so the gap reflects the volatility clustering.

The selected model has the highest error in every range, 0.242 for at-the-money strikes to 0.348 for far out-of-the-money strikes. The at-the-money error is higher than

5 Experiments

the 0.140 of the block bootstrap. The prices at the at-the-money strikes do not depend on the extreme returns, so the deviation is not caused by the tail weight alone. The results of G by moneyness are reported in more detail in Table 19.

Moneyness	e	s	Inverted (%)
0.5 to 0.9	0.249 ± 0.107	-0.225 ± 0.130	83.2 ± 18.5
0.9 to 1.1	0.242 ± 0.064	-0.208 ± 0.092	100.0 ± 0.0
1.1 to 1.5	0.314 ± 0.067	-0.290 ± 0.091	93.6 ± 12.7
1.5 to 2.0	0.348 ± 0.108	-0.312 ± 0.163	56.1 ± 44.2

Table 19: Implied volatility error, signed error, and percentage of inverted strikes of G by moneyness

The signed error is negative in every range, so the generated prices lie below the market prices in all moneyness ranges. The percentage of inverted strikes falls with the distance from the at-the-money strikes, from 100 % at the money to 56.1 % far out of the money. The seed spread of the far out-of-the-money strikes is 44.2, close to the mean of 56.1. The seeds differ in whether the simulated paths reach the outer strikes at all. The excluded strikes are those no simulated path reaches, so the reported error is smaller than the true deviation. The same holds for GBM.

The errors of Table 18 are averaged over the 20 valuation dates and do not show the shape of the deviation on a single date. Three valuation dates are reported in Fig. 11, covering the range of market conditions in the test split. 2025-10-28 has the lowest implied volatility at the at-the-money strike, at 0.321, and 2026-03-25 the highest, at 1.132. 2025-01-24 lies between them at 0.528. The curve of G is the mean across the seeds. The shaded area covers the range of the five seeds at each strike. The curves end where the inversion fails.

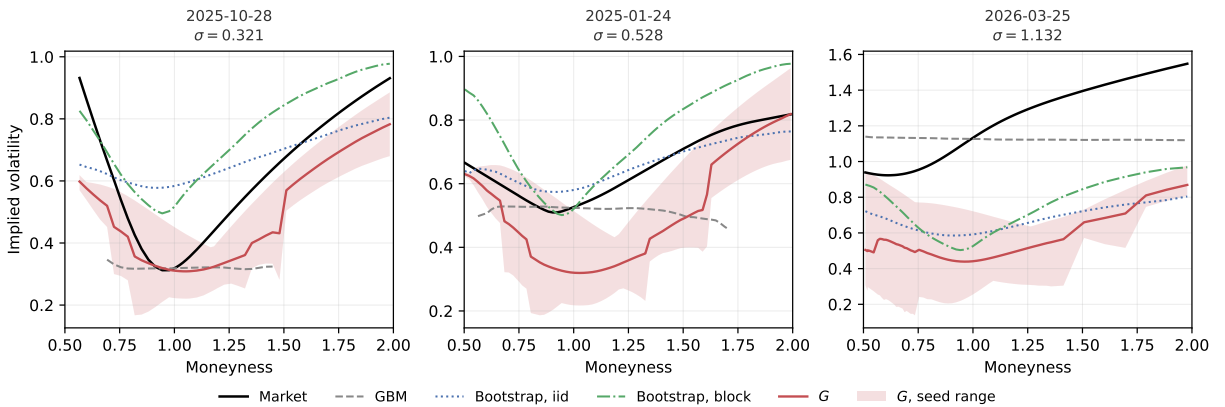


Figure 11: Implied volatility against moneyness on three valuation dates

The market implied volatility is a smile on all three valuation dates. GBM produces a flat implied volatility, since a single σ applies to every strike. Both bootstrap modes overprice the options on the first two valuation dates. The block mode captures the shape of the market, while the iid mode produces a flatter smile. The implied volatility of G lies below the market on all three dates.

The third date falls in the 2026 disruption described in Sec. 2.2. The market implied volatility rises from 1.13 for the at-the-money strike to 1.55 for the far out-of-the-money strike. Both bootstrap modes and G stay below 1.0. The bootstrap draws from the same training split on every valuation date (Sec. 4.6), so the simulated distribution does not change with the market conditions. GBM matches the market at the at-the-money strike, since its σ is taken from there. The selected model is conditioned on the returns preceding the valuation date. The annualized spread of the returns simulated by G and by the block bootstrap until expiry is reported in Fig. 12, together with the implied volatility at the at-the-money strike.

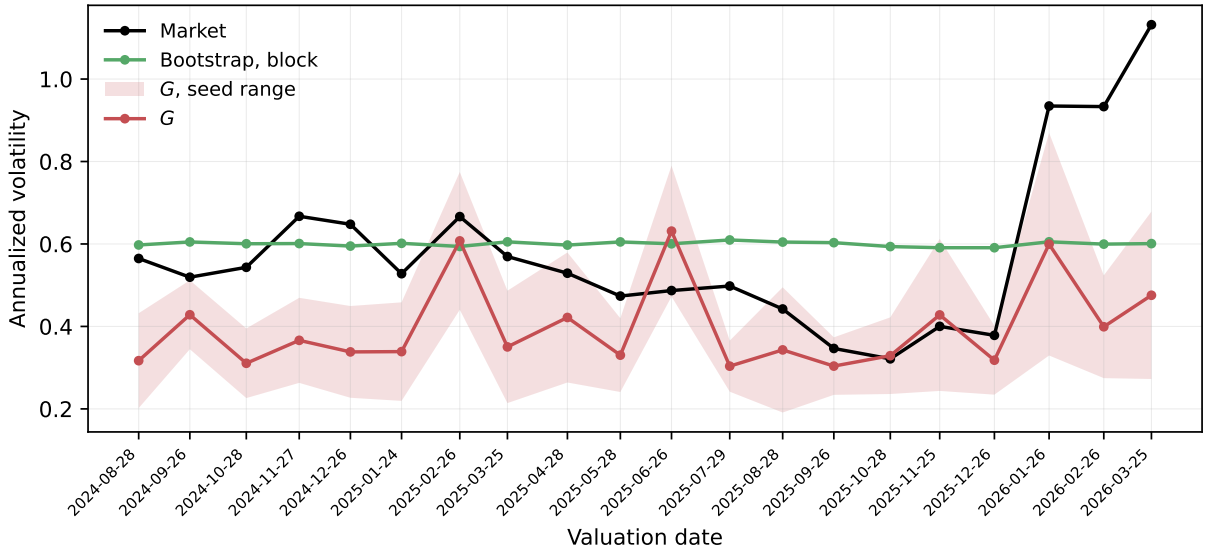


Figure 12: Annualized volatility on each valuation date

The spread of the block bootstrap ranges from 0.591 to 0.610. The spread of G ranges from 0.304 to 0.631, and its correlation with the spread of the realized returns in the condition window is 0.758. The simulated distribution of G widens when the preceding returns are turbulent. As a result, the conditioning of G produces a response. The spread of G is below the implied volatility of the market on 17 of the 20 valuation dates, so the response does not reach the level of the market.

The reported results of G were generated with the ForGAN baseline cost function. The PnL_a^* and PnL_a^* & MSE branches closest to the test split were priced as well (Sec. 5.5). The results are reported in Table 20. The error of the ForGAN baseline is the lowest among the three cost function branches. The seed spread of the ForGAN baseline is

the smallest at 0.043 against 0.377 for the PnL_a^* branch. The signed error of the PnL_a^* branch is 0.033 with a seed spread of 0.517, so some seeds price above the market and others below. Compared with the ForGAN baseline, the seed spread of the PnL_a^* & MSE branch is narrower on most of the generated return statistics (Sec. 5.5) and wider on the option prices. The seed spread is not explained by the excess kurtosis of the generated returns. In the PnL_a^* branch, seed 2 has the highest excess kurtosis at 21.59 and an error of 0.307, close to the errors of the other seeds. In contrast, seed 4 has the largest error of 1.121 with an excess kurtosis of 11.15. As a result, the divergence arises in the recursive rollout of the generated returns (Sec. 4.5), and not in the one-step distribution.

Cost function	Seed	Kurtosis	e	s	Inverted (%)
ForGAN	0	13.24	0.199	-0.161	76.4
	1	7.61	0.247	-0.218	98.6
	2	12.46	0.313	-0.292	80.6
	3	9.38	0.244	-0.184	86.5
	4	7.93	0.278	-0.276	63.0
PnL_a^*	0	10.12	0.193	-0.169	94.3
	1	8.40	0.384	-0.024	92.0
	2	21.59	0.307	-0.307	89.1
	3	8.90	0.280	-0.272	84.3
	4	11.15	1.121	0.937	77.3
PnL_a^* & MSE	0	11.62	0.215	-0.173	95.0
	1	11.88	0.333	-0.325	73.8
	2	12.30	0.230	-0.225	94.4
	3	5.39	0.271	-0.237	82.3
	4	12.23	0.615	0.255	89.7

Table 20: Excess kurtosis of the generated returns and option pricing results per seed, configuration 8

The prices generated by G deviate further from the market than the prices of the benchmarks in every moneyness range. The deviation is a systematic underpricing, and G reaches the fewest strikes. However, G is the only model whose simulated distribution depends on the returns preceding the valuation date, although the response does not reach the level of the market.

5.8 Summary of the Results

The TTF return series shows heavy tails, a positive skew, and volatility clustering, and the test split carries these properties through the 2026 disruption. Among the configurations of the ForGAN baseline, a longer training schedule and a larger discriminator

raise the excess kurtosis of the generated returns, while the generator hidden size, the noise dimension, a longer condition window, and the input scaling produce no separable improvement. Configuration 8 comes closest to the test split among the stable configurations. None of the nine Fin-GAN branches is separable from the ForGAN baseline on the excess kurtosis. The four branches containing SR^* turn the skewness negative. The PnL_a^* and $PnL_a^* \& MSE$ branches come closest to the test split, but their seed spreads are wider than the spread of the ForGAN baseline. Configuration 8 with the ForGAN baseline cost function was selected for pricing.

The selected model reproduces the stylized facts only in part. Its option prices deviate further from the market than the prices of both benchmark models, at an implied volatility error of 0.256 ± 0.043 against 0.119 for the block bootstrap, 0.142 for the iid bootstrap, and 0.149 for GBM. The deviation is a systematic underpricing in every money-ness range. The simulated spread of the selected model responds to the returns preceding the valuation date, but it stays below the market on most valuation dates. These results are discussed in Chapter 6.

6 Discussion

This chapter discusses the findings and the limitations of the study. The study evaluated whether the price paths generated by the selected model reproduce the implied volatility of the market. They do not. The generated prices deviate further from the market than the prices of GBM and of both bootstrap modes in every moneyness range. In addition, the deviation is a systematic underpricing. The result holds for the ForGAN baseline under configuration 8 on TTF front-month futures over the test period of Sec. 4.1.4. Sec. 6.1 examines the sources of the deviation, and Sec. 6.2 discusses the limitations of the study.

6.1 Sources of the Deviation

Several factors contribute to the deviation between the generated option prices and the market prices. These include the spread of the generated distribution, the size of the training sample, the returns used as the condition, and the recursive rollout of Sec. 4.5.

The generated distribution is narrower than the returns of the test split. The standard deviation of the generated returns is 0.0369 against 0.0404 for the test split, and the excess kurtosis 10.12 against 13.27 (Sec. 5.3). As a result, the option prices are underestimated across all moneyness ranges. The narrower distribution is not caused by mode collapse, since no one-step distribution of G collapsed (Sec. 5.2). The underpricing is visible not only in the tails but also at the at-the-money strikes. The selected model has an implied volatility error of 0.242 for at-the-money strikes, against 0.140 for the block bootstrap. The prices at the at-the-money strikes do not depend on the extreme returns, so the deviation is not caused by the tail weight alone.

The size of the training sample limits the tail weight of the generated returns. The training split contains 3,197 returns (Sec. 4.1.4). Fu et al. (2019) report that a cGAN underestimated the excess kurtosis of a simulated series at a training sample of 1,000 observations and matched it at 20,000. The excess kurtosis of the generated returns is below the test split, which is consistent with that finding. The available TTF futures data covers the period from 2010 (Sec. 4.1.1), so a longer training sample would require additional data.

The condition variable used in the GAN is the l past returns, so G has to infer the volatility from them. As seen in Fig. 12, G responds to the conditioning. The spread of the generated returns correlates with the spread of the condition window at 0.758. In contrast, the spread of the bootstrap stays almost constant across the 20 valuation dates. However, the spread of G stays below the implied volatility of the market on 17 of the 20 valuation dates. Fu et al. (2019) propose adding the LSTM cell to the generator so that the historical volatility is extracted from the condition, which ForGAN already contains (Sec. 3.2.3). The response of G to the condition window shows that the volatility is extracted, but this does not translate into a match of the implied volatility. A longer condition window does not close the gap. On the contrary, increasing l from 10 to 30 reduces the tail weight of the generated returns further (Sec. 5.4).

The generated returns are produced recursively (Sec. 4.5). After l steps, the condition window holds only the generated returns. At this point, G operates outside the regime it was trained on. At moneyness 1.5 to 2.0, the share of inverted strikes is 56.1 %, with a seed spread of 44.2 percentage points (Table 19). The seeds disagree about whether the paths reach the outer strikes at all. The seed spread reported in Sec. 5.3 is measured one step ahead and does not reflect the spread after h recursive steps. The rollout was evaluated only at the end of the path, so the contribution of the recursive rollout to the deviation is not established.

6.2 Limitations

The results are subject to four limitations: the model selection criterion, the coverage of the benchmark set, the exclusion of the strikes that carry no implied volatility, and the composition of the test split.

The model was selected on the test split, which is also used for the option pricing evaluation. The ForGAN baseline model was chosen on the seed spread of the generated returns, following the selection of configuration 8 in Sec. 5.3. The PnL_a^* and PnL_a^* & MSE branches have a narrower seed spread than the ForGAN baseline on most of the distributional statistics of Sec. 5.5, with the excess kurtosis as the exception. However, their seed spread is wider on the implied volatility error of Sec. 5.7. As a result, the selection criterion does not predict the seed spread of the option prices.

The benchmark coverage is limited to GBM and the two bootstrap modes. GARCH with t -distribution innovations (Bollerslev 1987) was not included, since fitting it requires choices of the model order and the innovation distribution, which the bootstrap avoids. GARCH- t conditions the simulated variance σ_t^2 on the returns preceding the valuation date, which neither bootstrap mode nor GBM does. GARCH- t also reproduces the heavy tails without the asymmetry, which no other benchmark in the set does (Sec. 4.6). The effect of asymmetry and heavy tails on the option prices is not separated.

The strikes with no implied volatility are excluded from the errors of Sec. 5.7. These are the strikes that lie outside the range the simulated paths cover (Sec. 4.5). As a result, the reported errors are lower bounds for G and for GBM. The moneyness band from 0.5 to 2.0 (Sec. 4.1.6) includes strikes that the simulated distribution does not reach. A narrower band would raise the percentage of inverted strikes but would also discard the strikes where the models differ most.

The 2026 disruption (Sec. 2.2) covers 57 of the 401 trading days in the test split. Excluding the disruption window reduces the excess kurtosis from 13.24 to 1.57 and the skewness from 0.91 to -0.06 . As a result, the distributional comparison of Chapter 5 and the pricing comparison of Sec. 5.7 depend on a single event. Among the valuation dates of Sec. 4.5, 2026-03-25 falls within the disruption. On this date, the market implied volatility reflects expected price moves, while all simulated models are based on past returns (Fig. 11).

7 Conclusion and Future Work

This chapter answers the two research questions of Sec. 1.2 and states what the study establishes. Sec. 7.1 gives the answers, and Sec. 7.2 outlines the directions that follow from the results.

7.1 Conclusion

The thesis examined whether GANs can generate TTF futures returns with the stylized facts of the observed data, namely heavy tails, volatility clustering, and asymmetry. In addition, it assessed whether price paths generated by recursive rollout reproduce the market prices of TTF options.

ForGAN and Fin-GAN were applied to daily TTF front-month futures returns. The generated returns were compared with the test split across training lengths, network capacities, condition window lengths, and input scaling methods. Each configuration was trained with five random seeds. Configuration 8 with the ForGAN baseline cost function was selected on the seed spread of the generated returns. The selected model generated price paths by recursive rollout. From these price paths, call options were priced on 20 valuation dates. The prices were compared with the market through the implied volatility. GBM and two bootstrap modes served as benchmarks.

The selected model reproduces the stylized facts in part. It learns heavy tails, with an excess kurtosis of 10.12 against 13.27 for the test split under configuration 8. The ForGAN baseline reproduces the sign of the asymmetry. The generated skewness of the ForGAN baseline is positive at 0.54, below the 0.92 of the test split, while the SR^* term of Fin-GAN turns it negative. Configuration 8 comes closest to the volatility clustering of the test split at a lag of one day, but its autocorrelation of squared returns decays much more slowly at longer lags. No mode collapse occurred in any run.

The generated price paths do not yet reproduce the market prices of TTF options. The selected model has the highest implied volatility error at 0.256, against 0.119 to 0.149 for the benchmarks. The signed error is negative in every moneyness range. As a result, the deviation of the generated option prices from the market is a systematic underpricing. At the same time, the selected model reacts to the recent returns. The spread of its simulated returns correlates with the spread of the condition window at 0.758, but stays below the market implied volatility on 17 of the 20 valuation dates.

For the setup of this thesis, RQ1 is answered in part and RQ2 negatively. The generated distribution is narrower than the observed returns. In addition, the recursive rollout moves G outside the regime it was trained on.

7.2 Future Work

Although the generated prices do not match the market, the results show that the approach has the potential to capture the stylized facts of TTF futures returns and could be improved to better reproduce market option prices. The results point to several directions for further work.

1. A longer training sample would test whether the tail weight of the generated returns increases with the amount of data.
2. The model selection and the pricing evaluation were both based on the test split. Repeating the selection on the validation split would show whether it produces the same configuration and cost function branch. It would also leave the test split as an independent basis for the pricing evaluation.
3. GARCH- t would add a benchmark that conditions on the preceding returns and would assess the results with a parametric model.
4. The seed spread of the one-step returns did not predict the seed spread of the option prices. A detailed analysis of the recursive rollout could clarify how errors accumulate and influence option pricing.
5. The simulated spread of the selected model responds to the market state but stays below its level. Correcting the level of the generated volatility is left for future work.

These directions address the level of the generated volatility. This level is the gap between the response of the model to the market state and the prices quoted by the market.

References

- Arjovsky, Martin, Soumith Chintala, and Léon Bottou (Dec. 2017). *Wasserstein GAN*. DOI: 10.48550/arXiv.1701.07875. arXiv: 1701.07875 [stat.ML].
- Barsha, Farhat Lamia and William Eberle (June 2025). “An In-Depth Review and Analysis of Mode Collapse in Generative Adversarial Networks”. In: *Machine Learning* 114.6, 141. ISSN: 0885-6125, 1573-0565. DOI: 10.1007/s10994-025-06772-7.
- Black, Fischer (Jan. 1976). “The Pricing of Commodity Contracts”. In: *Journal of Financial Economics* 3.1–2, pp. 167–179. ISSN: 0304405X. DOI: 10.1016/0304-405X(76)90024-6.
- Bohórquez Correa, Santiago, Stephanía Mosquera-López, and Jorge M. Uribe (Mar. 2026). “Time-Varying Systemic Risk in Electricity Markets Using Generative Adversarial Networks: Market Resilience and Policy”. In: *Energy Policy* 210, 115034. ISSN: 03014215. DOI: 10.1016/j.enpol.2025.115034.
- Bollerslev, Tim (Apr. 1986). “Generalized Autoregressive Conditional Heteroskedasticity”. In: *Journal of Econometrics* 31.3, pp. 307–327. ISSN: 03044076. DOI: 10.1016/0304-4076(86)90063-1.
- Bollerslev, Tim (Aug. 1987). “A Conditionally Heteroskedastic Time Series Model for Speculative Prices and Rates of Return”. In: *The Review of Economics and Statistics* 69.3, pp. 542–547. ISSN: 00346535. DOI: 10.2307/1925546. JSTOR: 1925546.
- Brown, Stephen P. A. and Mine K. Yücel (Apr. 2008). “What Drives Natural Gas Prices?”. In: *The Energy Journal* 29.2, pp. 45–60. ISSN: 0195-6574, 1944-9089. DOI: 10.5547/ISSN0195-6574-EJ-Vol29-No2-3.
- Cho, Kyunghyun et al. (Sept. 2014). *Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation*. DOI: 10.48550/arXiv.1406.1078. arXiv: 1406.1078 [cs.CL].
- Cont, Rama (Feb. 2001). “Empirical Properties of Asset Returns: Stylized Facts and Statistical Issues”. In: *Quantitative Finance* 1.2, pp. 223–236. ISSN: 1469-7688, 1469-7696. DOI: 10.1080/713665670.
- Duan, Jin-Chuan and Jean-Guy Simonato (1995). *Empirical Martingale Simulation for Asset Prices*. CIRANO Working Paper 95s-43. Montréal: CIRANO.
- Efron, Bradley (Jan. 1979). “Bootstrap Methods: Another Look at the Jackknife”. In: *The Annals of Statistics* 7.1, pp. 1–26. ISSN: 0090-5364. DOI: 10.1214/aos/1176344552.
- European Central Bank (2026). *Euro Short-Term Rate (€STR)*. URL: <https://data.ecb.europa.eu/data/datasets/EST/EST.B.EU000A2X2A25.WT>.
- European Commission (Nov. 2022). *Questions and Answers on the Gas Market Correction Mechanism*. Tech. rep. QANDA/22/7066. Brussels: European Commission. URL: https://ec.europa.eu/commission/presscorner/detail/en/qanda_22_7066.
- Eurostat (June 2026). *Natural Gas Supply Statistics*. Statistics Explained. Eurostat. URL: https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Natural_gas_supply_statistics.

References

- Fu, Rao et al. (Apr. 2019). *Time Series Simulation by Conditional Generative Adversarial Net*. DOI: 10.48550/arXiv.1904.11419. arXiv: 1904.11419 [stat.ML].
- Geman, Hélyette (2005). *Commodities and Commodity Derivatives: Modeling and Pricing for Agriculturals, Metals and Energy*. Wiley Finance Series. Chichester: John Wiley & Sons. ISBN: 978-0-470-01218-5.
- Gibson, Rajna and Eduardo S. Schwartz (July 1990). “Stochastic Convenience Yield and the Pricing of Oil Contingent Claims”. In: *The Journal of Finance* 45.3, pp. 959–976. ISSN: 0022-1082, 1540-6261. DOI: 10.1111/j.1540-6261.1990.tb05114.x.
- Glorot, Xavier and Yoshua Bengio (2010). “Understanding the Difficulty of Training Deep Feedforward Neural Networks”. In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Vol. 9. Proceedings of Machine Learning Research. Sardinia, Italy, pp. 249–256.
- Goodfellow, Ian (Apr. 2017). *NIPS 2016 Tutorial: Generative Adversarial Networks*. DOI: 10.48550/arXiv.1701.00160. arXiv: 1701.00160 [cs.LG].
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. Adaptive Computation and Machine Learning. Cambridge, MA: MIT Press. ISBN: 978-0-262-03561-3.
- Goodfellow, Ian, Jean Pouget-Abadie, et al. (June 2014). *Generative Adversarial Networks*. DOI: 10.48550/arXiv.1406.2661. arXiv: 1406.2661 [stat.ML].
- Goodfellow, Ian, Jean Pouget-Abadie, et al. (Oct. 2020). “Generative Adversarial Networks”. In: *Communications of the ACM* 63.11, pp. 139–144. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3422622.
- Harris, Larry (2003). *Trading and Exchanges: Market Microstructure for Practitioners*. Financial Management Association Survey and Synthesis Series. New York: Oxford University Press. ISBN: 978-0-19-514470-3.
- He, Kaiming et al. (Feb. 2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. DOI: 10.48550/arXiv.1502.01852. arXiv: 1502.01852 [cs.CV].
- Heather, Patrick (Mar. 2022). *A Series of Unfortunate Events: Explaining European Gas Prices in 2021 – The Role of the Traded Gas Hubs*. OIES Energy Insight 111. Oxford: Oxford Institute for Energy Studies. URL: <https://www.oxfordenergy.org/wp-content/uploads/2022/03/Insight-111-Explaining-European-gas-prices-in-2021-the-role-of-the-traded-gas-hubs.pdf>.
- Heather, Patrick (2026). *European Traded Gas Hubs: Resilience in the Face of Adversity*. OIES Paper NG 206. Oxford: Oxford Institute for Energy Studies. ISBN: 978-1-78467-299-7. URL: <https://www.oxfordenergy.org/publications/european-traded-gas-hubs-resilience-in-the-face-of-adversity/>.
- Hochreiter, Sepp and Jürgen Schmidhuber (Nov. 1997). “Long Short-Term Memory”. In: *Neural Computation* 9.8, pp. 1735–1780. ISSN: 0899-7667, 1530-888X. DOI: 10.1162/neco.1997.9.8.1735.
- Hull, John C. (2022). *Options, Futures, and Other Derivatives*. 11th ed. Global edition. Harlow: Pearson. ISBN: 978-1-292-41065-4.

References

- Koochali, Alireza et al. (2019). "Probabilistic Forecasting of Sensory Data With Generative Adversarial Networks – ForGAN". In: *IEEE Access* 7, pp. 63868–63880. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2019.2915544.
- Korn, Ralf and Laurena Ramadani (Apr. 2025). "Simulating Intraday Electricity Consumption with ForGAN". In: *Algorithms* 18.5, 256. ISSN: 1999-4893. DOI: 10.3390/a18050256.
- Künsch, Hans R. (Sept. 1989). "The Jackknife and the Bootstrap for General Stationary Observations". In: *The Annals of Statistics* 17.3, pp. 1217–1241. ISSN: 0090-5364. DOI: 10.1214/aos/1176347265.
- Merton, Robert C. (Jan. 1976). "Option Pricing When Underlying Stock Returns Are Discontinuous". In: *Journal of Financial Economics* 3.1–2, pp. 125–144. ISSN: 0304405X. DOI: 10.1016/0304-405X(76)90022-2.
- Mirza, Mehdi and Simon Osindero (Nov. 2014). *Conditional Generative Adversarial Nets*. DOI: 10.48550/arXiv.1411.1784. arXiv: 1411.1784 [cs.LG].
- Ni, Hao et al. (2020). *Conditional Sig-Wasserstein GANs for Time Series Generation*. DOI: 10.2139/ssrn.3623086.
- Obadi, Saleh Mothana and Matej Korcek (Oct. 2020). "Driving Fundamentals of Natural Gas Price in Europe". In: *International Journal of Energy Economics and Policy* 10.6, pp. 318–324. ISSN: 21464553. DOI: 10.32479/ijeep.10192.
- Ruf, Johannes and Weiguan Wang (2020). "Neural Networks for Option Pricing and Hedging: A Literature Review". In: *Journal of Computational Finance* 24.1, pp. 1–46.
- Schwartz, Eduardo S. (July 1997). "The Stochastic Behavior of Commodity Prices: Implications for Valuation and Hedging". In: *The Journal of Finance* 52.3, pp. 923–973. ISSN: 0022-1082, 1540-6261. DOI: 10.1111/j.1540-6261.1997.tb02721.x.
- Sharples, Jack and Bill Farren-Price (Mar. 2026). *The Iran War and Disruption to LNG Supply from the Persian Gulf*. Oxford Energy Comment. Oxford: Oxford Institute for Energy Studies. URL: <https://www.oxfordenergy.org/wpcms/wp-content/uploads/2026/03/Comment-The-Iran-War-and-Disruption-to-LNG-.pdf>.
- Vuletić, Milena and Rama Cont (July 2024). "VolGAN: A Generative Model for Arbitrage-Free Implied Volatility Surfaces". In: *Applied Mathematical Finance* 31.4, pp. 203–238. ISSN: 1350-486X, 1466-4313. DOI: 10.1080/1350486X.2025.2471317.
- Vuletić, Milena, Felix Prenzel, and Mihai Cucuringu (Feb. 2024). "Fin-GAN: Forecasting and Classifying Financial Time Series via Generative Adversarial Networks". In: *Quantitative Finance* 24.2, pp. 175–199. ISSN: 1469-7688, 1469-7696. DOI: 10.1080/14697688.2023.2299466.
- Walter, Viktor and Andreas Wagner (Dec. 2024). "Probabilistic Simulation of Electricity Price Scenarios Using Conditional Generative Adversarial Networks". In: *Energy and AI* 18, 100422. ISSN: 26665468. DOI: 10.1016/j.egyai.2024.100422.
- Wiese, Magnus, Lianjun Bai, et al. (2019). *Deep Hedging: Learning to Simulate Equity Option Markets*. DOI: 10.48550/arXiv.1911.01700. arXiv: 1911.01700 [q-fin.CP].

References

- Wiese, Magnus, Robert Knobloch, et al. (Sept. 2020). “Quant GANs: Deep Generation of Financial Time Series”. In: *Quantitative Finance* 20.9, pp. 1419–1440. ISSN: 1469-7688, 1469-7696. DOI: 10.1080/14697688.2020.1730426.
- Yoon, Jinsung, Daniel Jarrett, and Mihaela van der Schaar (2019). “Time-Series Generative Adversarial Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 32, pp. 5508–5518.
- Zhang, Fan et al. (2026). *Beyond Visual Realism: Toward Reliable Financial Time Series Generation*. DOI: 10.48550/arXiv.2601.12990. arXiv: 2601.12990.

A Full Experimental Grid

This appendix reports the generated returns of all ten cost function branches, listed in Sec. 4.3. Each of the ten configurations of Table 1 and the input scaling run is reported with all ten branches. Every combination was trained with five seeds, and the values are reported as mean and seed spread. Each configuration is reported in three tables: distributional statistics, tail statistics, and the autocorrelation of squared returns at $\tau = 1, 2, 5, 10$. The reference values of the test split are reported in Table 5. The first column of each table enumerates the ten cost function branches as listed in Table 21. Kurt denotes the excess kurtosis, q_1 and q_{99} the 1 % and 99 % quantiles, and $P_{0.05}$ and $P_{0.10}$ the fractions $P(|x| > 0.05)$ and $P(|x| > 0.10)$.

No.	Cost function	No.	Cost function
1	ForGAN	6	PnL_a^* & SR^*
2	MSE	7	PnL_a^* & STD
3	PnL_a^*	8	SR^* & MSE
4	SR^*	9	PnL_a^* & MSE & SR^*
5	PnL_a^* & MSE	10	PnL_a^* & MSE & STD

Table 21: Numbering of the cost function branches in the tables of this appendix

	Mean	SD	Skew	Kurt	Min	Max
1	-0.0003±0.0019	0.0303±0.0118	-0.18±0.39	0.32±0.42	-0.1533±0.0474	0.1357±0.0519
2	0.0025±0.0026	0.0294±0.0095	-0.07±0.24	0.19±0.23	-0.1349±0.0300	0.1387±0.0513
3	0.0053±0.0059	0.0289±0.0032	-0.03±0.40	0.26±0.12	-0.1343±0.0292	0.1470±0.0216
4	0.0006±0.0044	0.0190±0.0052	-0.10±0.55	0.56±0.26	-0.1040±0.0434	0.0881±0.0295
5	0.0033±0.0065	0.0274±0.0036	-0.05±0.38	0.26±0.11	-0.1352±0.0389	0.1375±0.0251
6	0.0003±0.0041	0.0189±0.0054	-0.13±0.50	0.55±0.24	-0.1014±0.0392	0.0895±0.0239
7	-0.0012±0.0032	0.0112±0.0090	-0.07±0.48	0.93±0.40	-0.0681±0.0558	0.0536±0.0229
8	0.0006±0.0039	0.0186±0.0050	-0.12±0.53	0.57±0.27	-0.0981±0.0377	0.0905±0.0359
9	0.0010±0.0039	0.0186±0.0050	-0.11±0.53	0.58±0.27	-0.0978±0.0333	0.0884±0.0266
10	-0.0008±0.0021	0.0102±0.0077	-0.08±0.43	0.86±0.35	-0.0623±0.0542	0.0494±0.0235

Table 22: All cost function branches, configuration 1: distributional statistics

A Full Experimental Grid

	q_1	q_{99}	$P_{0.05}$	$P_{0.10}$
1	-0.0739±0.0251	0.0684±0.0272	0.115±0.124	0.008±0.016
2	-0.0674±0.0175	0.0709±0.0248	0.103±0.102	0.005±0.010
3	-0.0635±0.0179	0.0726±0.0098	0.094±0.024	0.002±0.002
4	-0.0471±0.0192	0.0434±0.0141	0.023±0.026	0.001±0.001
5	-0.0626±0.0189	0.0667±0.0099	0.079±0.026	0.002±0.002
6	-0.0475±0.0193	0.0428±0.0137	0.023±0.027	0.001±0.001
7	-0.0308±0.0305	0.0238±0.0120	0.016±0.036	0.001±0.002
8	-0.0465±0.0183	0.0425±0.0131	0.020±0.024	0.000±0.001
9	-0.0460±0.0182	0.0431±0.0131	0.020±0.023	0.000±0.001
10	-0.0277±0.0255	0.0223±0.0113	0.010±0.023	0.000±0.000

Table 23: All cost function branches, configuration 1: tail statistics

	$C_2(1)$	$C_2(2)$	$C_2(5)$	$C_2(10)$
1	-0.015±0.056	-0.009±0.036	-0.009±0.035	0.018±0.032
2	0.040±0.074	0.009±0.056	0.007±0.055	0.024±0.041
3	0.007±0.075	-0.018±0.046	-0.001±0.039	-0.021±0.055
4	0.001±0.024	-0.030±0.034	0.029±0.071	0.017±0.067
5	-0.024±0.034	-0.012±0.044	-0.007±0.067	0.022±0.051
6	-0.033±0.033	-0.012±0.036	-0.019±0.056	-0.004±0.032
7	-0.001±0.033	-0.062±0.014	0.006±0.026	0.002±0.039
8	-0.001±0.051	-0.007±0.041	-0.028±0.034	-0.035±0.024
9	-0.003±0.029	0.010±0.063	-0.025±0.016	-0.024±0.034
10	0.028±0.080	-0.008±0.054	0.009±0.049	-0.005±0.068

Table 24: All cost function branches, configuration 1: autocorrelation of squared returns

	Mean	SD	Skew	Kurt	Min	Max
1	-0.0029±0.0019	0.0365±0.0034	0.19±0.58	4.51±1.52	-0.2234±0.0287	0.2516±0.0761
2	-0.0011±0.0020	0.0354±0.0011	0.39±0.27	4.13±1.20	-0.2151±0.0396	0.2592±0.0469
3	-0.0003±0.0028	0.0298±0.0046	0.54±0.40	2.44±1.01	-0.1502±0.0123	0.2062±0.0531
4	0.0017±0.0007	0.0246±0.0029	0.11±0.24	0.38±0.22	-0.1157±0.0204	0.1237±0.0322
5	-0.0001±0.0026	0.0308±0.0047	0.51±0.41	2.29±1.11	-0.1552±0.0073	0.2061±0.0498
6	0.0011±0.0009	0.0243±0.0032	0.08±0.26	0.36±0.18	-0.1105±0.0166	0.1190±0.0300
7	-0.0007±0.0008	0.0340±0.0276	0.48±0.48	1.40±0.92	-0.1488±0.0908	0.1987±0.1427
8	0.0009±0.0009	0.0244±0.0026	0.11±0.25	0.37±0.19	-0.1138±0.0155	0.1260±0.0223
9	0.0010±0.0012	0.0248±0.0030	0.08±0.24	0.33±0.17	-0.1194±0.0191	0.1212±0.0267
10	0.0008±0.0043	0.0449±0.0282	0.47±0.48	0.93±1.16	-0.1907±0.1011	0.2500±0.1621

Table 25: All cost function branches, configuration 2: distributional statistics

A Full Experimental Grid

	q_1	q_{99}	$P_{0.05}$	$P_{0.10}$
1	-0.1074 ± 0.0162	0.1103 ± 0.0163	0.129 ± 0.021	0.026 ± 0.008
2	-0.0976 ± 0.0083	0.1114 ± 0.0066	0.124 ± 0.006	0.024 ± 0.002
3	-0.0724 ± 0.0076	0.0924 ± 0.0237	0.093 ± 0.032	0.011 ± 0.009
4	-0.0563 ± 0.0069	0.0629 ± 0.0111	0.051 ± 0.027	0.001 ± 0.001
5	-0.0744 ± 0.0068	0.0940 ± 0.0244	0.100 ± 0.037	0.011 ± 0.010
6	-0.0565 ± 0.0070	0.0609 ± 0.0124	0.048 ± 0.030	0.001 ± 0.001
7	-0.0725 ± 0.0473	0.0926 ± 0.0737	0.171 ± 0.227	0.041 ± 0.079
8	-0.0563 ± 0.0057	0.0613 ± 0.0106	0.048 ± 0.025	0.000 ± 0.001
9	-0.0576 ± 0.0084	0.0617 ± 0.0105	0.051 ± 0.028	0.000 ± 0.001
10	-0.0945 ± 0.0506	0.1181 ± 0.0754	0.264 ± 0.212	0.066 ± 0.088

Table 26: All cost function branches, configuration 2: tail statistics

	$C_2(1)$	$C_2(2)$	$C_2(5)$	$C_2(10)$
1	0.125 ± 0.087	0.129 ± 0.055	0.164 ± 0.107	0.073 ± 0.056
2	0.149 ± 0.063	0.141 ± 0.093	0.152 ± 0.088	0.134 ± 0.078
3	0.064 ± 0.068	0.067 ± 0.065	0.056 ± 0.063	0.093 ± 0.074
4	-0.017 ± 0.085	0.066 ± 0.037	0.015 ± 0.094	0.031 ± 0.042
5	0.086 ± 0.070	0.026 ± 0.040	0.067 ± 0.033	0.048 ± 0.075
6	-0.003 ± 0.066	0.028 ± 0.068	0.028 ± 0.064	0.015 ± 0.033
7	-0.011 ± 0.055	0.006 ± 0.021	-0.005 ± 0.077	-0.048 ± 0.025
8	0.020 ± 0.029	0.016 ± 0.087	0.081 ± 0.044	0.045 ± 0.037
9	-0.022 ± 0.035	-0.001 ± 0.039	0.031 ± 0.094	0.039 ± 0.070
10	0.013 ± 0.095	0.018 ± 0.043	0.036 ± 0.101	0.022 ± 0.050

Table 27: All cost function branches, configuration 2: autocorrelation of squared returns

	Mean	SD	Skew	Kurt	Min	Max
1	-0.0016 ± 0.0029	0.0353 ± 0.0035	0.02 ± 0.68	4.70 ± 1.53	-0.2326 ± 0.0675	0.2478 ± 0.0444
2	-0.0004 ± 0.0035	0.0361 ± 0.0031	0.18 ± 0.74	3.66 ± 1.69	-0.2092 ± 0.0597	0.2394 ± 0.0254
3	-0.0011 ± 0.0042	0.0337 ± 0.0185	0.30 ± 0.18	2.27 ± 1.32	-0.1861 ± 0.0708	0.2116 ± 0.1101
4	0.0005 ± 0.0022	0.0250 ± 0.0072	0.11 ± 0.26	0.43 ± 0.23	-0.1254 ± 0.0369	0.1352 ± 0.0464
5	-0.0001 ± 0.0034	0.0259 ± 0.0048	0.30 ± 0.33	2.30 ± 0.97	-0.1536 ± 0.0537	0.1642 ± 0.0090
6	0.0004 ± 0.0027	0.0252 ± 0.0077	0.10 ± 0.22	0.38 ± 0.22	-0.1213 ± 0.0304	0.1352 ± 0.0479
7	-0.0232 ± 0.0349	0.0525 ± 0.0422	0.02 ± 0.61	0.69 ± 0.37	-0.2857 ± 0.1913	0.2214 ± 0.2297
8	-0.0000 ± 0.0012	0.0240 ± 0.0053	0.10 ± 0.21	0.40 ± 0.22	-0.1183 ± 0.0241	0.1301 ± 0.0455
9	-0.0003 ± 0.0014	0.0231 ± 0.0040	0.09 ± 0.22	0.42 ± 0.24	-0.1155 ± 0.0270	0.1237 ± 0.0304
10	0.0067 ± 0.0172	0.0404 ± 0.0465	0.22 ± 0.43	0.52 ± 0.46	-0.1688 ± 0.1842	0.2179 ± 0.2163

Table 28: All cost function branches, configuration 3: distributional statistics

A Full Experimental Grid

	q_1	q_{99}	$P_{0.05}$	$P_{0.10}$
1	-0.1013±0.0213	0.1020±0.0167	0.121±0.022	0.021±0.007
2	-0.0973±0.0226	0.1054±0.0115	0.129±0.020	0.023±0.010
3	-0.0814±0.0362	0.0967±0.0485	0.140±0.171	0.030±0.049
4	-0.0570±0.0117	0.0631±0.0231	0.059±0.061	0.002±0.005
5	-0.0668±0.0198	0.0757±0.0189	0.062±0.031	0.006±0.007
6	-0.0577±0.0119	0.0634±0.0245	0.062±0.070	0.003±0.006
7	-0.1428±0.0987	0.1003±0.1285	0.387±0.327	0.156±0.160
8	-0.0557±0.0087	0.0599±0.0170	0.047±0.040	0.001±0.002
9	-0.0545±0.0080	0.0571±0.0135	0.039±0.025	0.000±0.001
10	-0.0839±0.0936	0.1049±0.1186	0.192±0.294	0.092±0.194

Table 29: All cost function branches, configuration 3: tail statistics

	$C_2(1)$	$C_2(2)$	$C_2(5)$	$C_2(10)$
1	0.159±0.097	0.121±0.028	0.105±0.098	0.074±0.035
2	0.207±0.116	0.123±0.059	0.087±0.079	0.131±0.100
3	0.113±0.145	0.065±0.098	0.075±0.116	0.045±0.104
4	-0.013±0.029	0.035±0.068	-0.009±0.028	-0.005±0.021
5	0.124±0.082	0.097±0.063	0.104±0.057	0.092±0.065
6	0.001±0.051	-0.027±0.079	0.002±0.034	0.028±0.101
7	0.027±0.040	-0.016±0.042	0.000±0.036	-0.004±0.024
8	0.022±0.022	0.024±0.060	0.057±0.062	0.014±0.065
9	0.031±0.067	0.040±0.043	0.027±0.051	-0.007±0.103
10	0.020±0.046	-0.028±0.035	0.005±0.073	-0.008±0.015

Table 30: All cost function branches, configuration 3: autocorrelation of squared returns

	Mean	SD	Skew	Kurt	Min	Max
1	-0.0002±0.0014	0.0369±0.0029	-0.09±0.38	7.95±1.68	-0.3212±0.0454	0.3254±0.0532
2	-0.0001±0.0015	0.0364±0.0026	-0.08±0.34	7.83±1.39	-0.3151±0.0434	0.3416±0.0783
3	-0.0008±0.0013	0.0376±0.0019	-0.50±0.63	7.35±1.58	-0.3360±0.0890	0.3004±0.0711
4	0.0004±0.0008	0.0231±0.0008	0.21±0.16	0.76±0.20	-0.1149±0.0069	0.1357±0.0168
5	-0.0008±0.0010	0.0373±0.0015	-0.51±0.70	7.25±1.65	-0.3309±0.0909	0.2841±0.0626
6	0.0004±0.0004	0.0232±0.0007	0.17±0.13	0.72±0.16	-0.1138±0.0096	0.1299±0.0109
7	-0.0011±0.0009	0.0297±0.0026	0.12±0.44	5.89±1.63	-0.2495±0.0794	0.2579±0.0634
8	0.0006±0.0007	0.0230±0.0010	0.17±0.10	0.71±0.14	-0.1163±0.0095	0.1269±0.0106
9	0.0005±0.0008	0.0232±0.0009	0.19±0.09	0.71±0.16	-0.1178±0.0119	0.1278±0.0123
10	-0.0012±0.0010	0.0293±0.0023	0.14±0.44	5.57±1.48	-0.2458±0.0594	0.2415±0.0308

Table 31: All cost function branches, configuration 4: distributional statistics

A Full Experimental Grid

	q_1	q_{99}	$P_{0.05}$	$P_{0.10}$
1	-0.1228±0.0128	0.1177±0.0184	0.120±0.012	0.033±0.010
2	-0.1199±0.0120	0.1155±0.0163	0.118±0.011	0.031±0.009
3	-0.1287±0.0172	0.1087±0.0140	0.125±0.006	0.032±0.009
4	-0.0544±0.0030	0.0610±0.0042	0.040±0.007	0.000±0.000
5	-0.1278±0.0179	0.1072±0.0124	0.124±0.005	0.031±0.007
6	-0.0552±0.0021	0.0610±0.0033	0.041±0.006	0.000±0.000
7	-0.0921±0.0169	0.0941±0.0079	0.098±0.012	0.015±0.008
8	-0.0544±0.0034	0.0605±0.0027	0.039±0.008	0.000±0.000
9	-0.0546±0.0033	0.0611±0.0029	0.041±0.007	0.000±0.000
10	-0.0901±0.0159	0.0927±0.0074	0.096±0.011	0.014±0.007

Table 32: All cost function branches, configuration 4: tail statistics

	$C_2(1)$	$C_2(2)$	$C_2(5)$	$C_2(10)$
1	0.057±0.071	0.123±0.124	0.120±0.092	0.079±0.038
2	0.109±0.061	0.128±0.076	0.068±0.069	0.128±0.117
3	0.170±0.164	0.242±0.112	0.139±0.128	0.085±0.064
4	0.036±0.055	0.002±0.049	0.026±0.084	-0.009±0.046
5	0.145±0.064	0.102±0.071	0.137±0.105	0.078±0.057
6	0.012±0.055	0.044±0.049	0.062±0.052	0.037±0.047
7	0.077±0.098	0.059±0.070	0.028±0.070	0.011±0.045
8	-0.042±0.021	0.023±0.055	0.064±0.064	0.000±0.049
9	0.014±0.078	0.015±0.085	0.075±0.077	0.033±0.045
10	0.046±0.052	0.067±0.058	0.055±0.053	0.055±0.083

Table 33: All cost function branches, configuration 4: autocorrelation of squared returns

	Mean	SD	Skew	Kurt	Min	Max
1	0.0001±0.0018	0.0354±0.0031	0.41±0.68	7.04±1.83	-0.2746±0.0978	0.3497±0.0556
2	0.0001±0.0018	0.0353±0.0043	0.30±0.63	6.09±1.86	-0.2763±0.0919	0.3237±0.0505
3	-0.0021±0.0016	0.0350±0.0031	-0.30±0.30	5.25±1.61	-0.2728±0.0578	0.2549±0.0499
4	-0.0003±0.0011	0.0234±0.0003	0.19±0.11	0.68±0.19	-0.1217±0.0153	0.1347±0.0078
5	-0.0021±0.0013	0.0347±0.0032	-0.31±0.28	5.03±1.49	-0.2685±0.0604	0.2477±0.0489
6	-0.0000±0.0008	0.0233±0.0005	0.18±0.14	0.66±0.15	-0.1192±0.0202	0.1414±0.0120
7	-0.0013±0.0007	0.0308±0.0029	-0.20±0.42	4.34±1.83	-0.2320±0.0440	0.2376±0.0241
8	-0.0002±0.0008	0.0234±0.0005	0.15±0.15	0.67±0.15	-0.1225±0.0157	0.1334±0.0052
9	-0.0003±0.0008	0.0233±0.0003	0.15±0.14	0.68±0.15	-0.1220±0.0099	0.1324±0.0031
10	-0.0020±0.0010	0.0309±0.0033	-0.23±0.45	4.22±1.62	-0.2318±0.0430	0.2130±0.0237

Table 34: All cost function branches, configuration 5: distributional statistics

A Full Experimental Grid

	q_1	q_{99}	$P_{0.05}$	$P_{0.10}$
1	-0.1007±0.0247	0.1130±0.0101	0.110±0.015	0.025±0.009
2	-0.1009±0.0275	0.1117±0.0146	0.111±0.017	0.024±0.010
3	-0.1124±0.0162	0.0989±0.0066	0.117±0.018	0.024±0.007
4	-0.0552±0.0040	0.0604±0.0010	0.040±0.003	0.000±0.000
5	-0.1110±0.0159	0.0964±0.0076	0.116±0.018	0.023±0.007
6	-0.0549±0.0045	0.0603±0.0019	0.039±0.005	0.000±0.000
7	-0.0929±0.0124	0.0848±0.0117	0.094±0.020	0.013±0.005
8	-0.0558±0.0049	0.0599±0.0014	0.040±0.005	0.000±0.000
9	-0.0555±0.0037	0.0595±0.0025	0.039±0.003	0.000±0.000
10	-0.0943±0.0139	0.0843±0.0152	0.095±0.023	0.014±0.007

Table 35: All cost function branches, configuration 5: tail statistics

	$C_2(1)$	$C_2(2)$	$C_2(5)$	$C_2(10)$
1	0.121±0.111	0.116±0.099	0.132±0.127	0.043±0.033
2	0.160±0.103	0.198±0.065	0.127±0.123	0.088±0.070
3	0.148±0.082	0.182±0.107	0.097±0.109	0.144±0.100
4	0.066±0.077	-0.005±0.046	0.005±0.043	0.013±0.034
5	0.136±0.118	0.186±0.096	0.134±0.069	0.098±0.072
6	0.013±0.049	0.000±0.018	0.038±0.076	0.027±0.049
7	0.174±0.123	0.048±0.053	0.084±0.073	0.144±0.113
8	0.067±0.049	-0.012±0.008	0.024±0.046	-0.000±0.019
9	0.020±0.044	0.007±0.047	0.041±0.046	0.023±0.050
10	0.141±0.119	0.100±0.107	0.109±0.047	0.151±0.172

Table 36: All cost function branches, configuration 5: autocorrelation of squared returns

	Mean	SD	Skew	Kurt	Min	Max
1	-0.0006±0.0026	0.0342±0.0019	0.15±0.45	3.74±1.00	-0.1893±0.0538	0.2314±0.0248
2	-0.0005±0.0019	0.0334±0.0021	0.14±0.48	3.58±1.14	-0.1865±0.0601	0.2223±0.0479
3	0.0001±0.0022	0.0352±0.0011	0.14±0.59	3.73±1.44	-0.2141±0.0614	0.2161±0.0238
4	-0.0007±0.0009	0.0246±0.0020	0.11±0.21	0.95±0.42	-0.1223±0.0201	0.1370±0.0185
5	-0.0000±0.0025	0.0353±0.0010	0.15±0.50	3.51±0.96	-0.2083±0.0522	0.2254±0.0273
6	-0.0006±0.0010	0.0246±0.0022	0.11±0.21	0.96±0.42	-0.1200±0.0173	0.1425±0.0210
7	0.0004±0.0016	0.0326±0.0014	0.20±0.41	3.62±0.97	-0.1859±0.0235	0.2201±0.0247
8	-0.0007±0.0009	0.0245±0.0021	0.09±0.21	0.92±0.41	-0.1228±0.0171	0.1356±0.0183
9	-0.0009±0.0013	0.0246±0.0021	0.11±0.21	0.90±0.44	-0.1213±0.0221	0.1395±0.0183
10	0.0000±0.0017	0.0325±0.0016	0.10±0.50	3.30±1.24	-0.1849±0.0376	0.2088±0.0300

Table 37: All cost function branches, configuration 6: distributional statistics

A Full Experimental Grid

	q_1	q_{99}	$P_{0.05}$	$P_{0.10}$
1	-0.0935±0.0062	0.0930±0.0163	0.119±0.012	0.016±0.006
2	-0.0908±0.0075	0.0912±0.0141	0.115±0.017	0.014±0.005
3	-0.0942±0.0155	0.1009±0.0095	0.125±0.011	0.019±0.004
4	-0.0609±0.0050	0.0633±0.0108	0.048±0.016	0.002±0.002
5	-0.0958±0.0115	0.0995±0.0108	0.128±0.010	0.019±0.004
6	-0.0610±0.0052	0.0628±0.0100	0.048±0.018	0.001±0.002
7	-0.0899±0.0095	0.0930±0.0122	0.103±0.010	0.016±0.004
8	-0.0609±0.0051	0.0622±0.0099	0.048±0.017	0.001±0.002
9	-0.0615±0.0063	0.0630±0.0106	0.049±0.018	0.001±0.002
10	-0.0895±0.0093	0.0887±0.0142	0.106±0.009	0.014±0.005

Table 38: All cost function branches, configuration 6: tail statistics

	$C_2(1)$	$C_2(2)$	$C_2(5)$	$C_2(10)$
1	0.191±0.086	0.151±0.107	0.125±0.048	0.111±0.107
2	0.176±0.096	0.161±0.106	0.125±0.078	0.106±0.040
3	0.220±0.070	0.250±0.074	0.161±0.106	0.111±0.090
4	0.078±0.085	0.071±0.076	0.064±0.040	0.028±0.064
5	0.212±0.070	0.254±0.042	0.166±0.069	0.123±0.064
6	0.091±0.031	0.091±0.054	0.070±0.070	0.085±0.079
7	0.149±0.057	0.209±0.038	0.163±0.064	0.177±0.119
8	0.134±0.078	0.095±0.057	0.046±0.052	0.074±0.110
9	0.078±0.061	0.079±0.049	0.053±0.056	0.061±0.085
10	0.169±0.072	0.188±0.076	0.173±0.065	0.147±0.052

Table 39: All cost function branches, configuration 6: autocorrelation of squared returns

	Mean	SD	Skew	Kurt	Min	Max
1	-0.0004±0.0011	0.0368±0.0018	0.04±0.30	7.97±1.22	-0.3178±0.0355	0.3545±0.0445
2	-0.0001±0.0021	0.0364±0.0020	0.20±0.65	8.60±1.46	-0.3186±0.0553	0.3600±0.0311
3	-0.0009±0.0018	0.0401±0.0030	0.22±0.73	11.09±3.40	-0.3709±0.0232	0.4125±0.0929
4	0.0011±0.0016	0.0358±0.0028	0.07±0.58	4.67±1.31	-0.2535±0.0551	0.2647±0.0569
5	-0.0000±0.0017	0.0386±0.0019	-0.42±0.93	9.96±2.96	-0.3558±0.0485	0.3320±0.0452
6	0.0011±0.0013	0.0357±0.0026	0.10±0.50	4.39±1.39	-0.2431±0.0402	0.2623±0.0380
7	0.0005±0.0020	0.0374±0.0026	0.14±0.58	8.79±2.59	-0.3409±0.0741	0.3541±0.0295
8	0.0007±0.0010	0.0350±0.0033	0.14±0.55	4.47±1.23	-0.2391±0.0567	0.2522±0.0490
9	0.0013±0.0017	0.0349±0.0030	0.31±0.53	4.48±1.56	-0.2293±0.0519	0.2654±0.0480
10	-0.0002±0.0019	0.0369±0.0025	0.01±0.55	9.53±3.06	-0.3344±0.0697	0.3597±0.0396

Table 40: All cost function branches, configuration 7: distributional statistics

A Full Experimental Grid

	q_1	q_{99}	$P_{0.05}$	$P_{0.10}$
1	-0.1166±0.0129	0.1163±0.0084	0.114±0.007	0.030±0.004
2	-0.1116±0.0141	0.1153±0.0123	0.112±0.012	0.029±0.006
3	-0.1243±0.0153	0.1270±0.0167	0.124±0.010	0.034±0.006
4	-0.1006±0.0191	0.1059±0.0024	0.126±0.018	0.023±0.007
5	-0.1252±0.0086	0.1178±0.0209	0.121±0.012	0.033±0.007
6	-0.1001±0.0172	0.1063±0.0031	0.126±0.017	0.022±0.006
7	-0.1159±0.0154	0.1213±0.0173	0.114±0.004	0.030±0.006
8	-0.0967±0.0193	0.1035±0.0019	0.122±0.022	0.020±0.007
9	-0.0921±0.0168	0.1062±0.0027	0.121±0.022	0.020±0.006
10	-0.1168±0.0145	0.1145±0.0108	0.110±0.004	0.029±0.005

Table 41: All cost function branches, configuration 7: tail statistics

	$C_2(1)$	$C_2(2)$	$C_2(5)$	$C_2(10)$
1	0.153±0.070	0.118±0.088	0.143±0.100	0.090±0.058
2	0.122±0.093	0.146±0.125	0.090±0.070	0.054±0.042
3	0.227±0.182	0.134±0.052	0.076±0.066	0.081±0.045
4	0.114±0.085	0.177±0.117	0.043±0.029	0.100±0.064
5	0.098±0.082	0.138±0.061	0.097±0.094	0.070±0.076
6	0.144±0.118	0.207±0.109	0.098±0.074	0.085±0.042
7	0.172±0.110	0.134±0.064	0.074±0.047	0.117±0.058
8	0.115±0.133	0.154±0.103	0.073±0.038	0.057±0.040
9	0.115±0.088	0.132±0.064	0.095±0.055	0.089±0.044
10	0.135±0.129	0.077±0.068	0.093±0.046	0.137±0.067

Table 42: All cost function branches, configuration 7: autocorrelation of squared returns

	Mean	SD	Skew	Kurt	Min	Max
1	0.0019±0.0034	0.0369±0.0036	0.54±0.88	10.12±2.59	-0.2509±0.0482	0.3742±0.0535
2	0.0006±0.0023	0.0377±0.0024	0.23±1.29	10.13±4.39	-0.2644±0.0655	0.3343±0.1207
3	0.0004±0.0030	0.0414±0.0030	0.93±0.30	12.03±5.45	-0.2542±0.0468	0.3950±0.0887
4	0.0010±0.0005	0.0341±0.0036	-0.59±0.16	8.08±2.72	-0.2612±0.0373	0.2268±0.0474
5	0.0009±0.0020	0.0397±0.0032	0.81±0.46	10.69±2.97	-0.2401±0.0302	0.3892±0.0618
6	0.0012±0.0020	0.0351±0.0031	-0.64±0.20	8.03±2.55	-0.2638±0.0303	0.2207±0.0322
7	-0.0007±0.0032	0.0356±0.0017	-0.09±0.95	8.21±3.68	-0.2821±0.0737	0.3181±0.0776
8	0.0007±0.0011	0.0331±0.0034	-0.57±0.20	8.02±2.42	-0.2553±0.0310	0.2154±0.0369
9	0.0007±0.0014	0.0338±0.0034	-0.60±0.24	7.86±2.93	-0.2570±0.0406	0.2231±0.0509
10	0.0012±0.0009	0.0353±0.0032	1.11±0.42	6.78±1.29	-0.1979±0.0157	0.3562±0.0312

Table 43: All cost function branches, configuration 8: distributional statistics

A Full Experimental Grid

	q_1	q_{99}	$P_{0.05}$	$P_{0.10}$
1	-0.1055±0.0304	0.1185±0.0258	0.104±0.016	0.027±0.006
2	-0.1155±0.0144	0.1200±0.0257	0.106±0.006	0.030±0.007
3	-0.1297±0.0187	0.1401±0.0235	0.119±0.010	0.036±0.005
4	-0.0913±0.0171	0.0945±0.0113	0.100±0.025	0.017±0.005
5	-0.1186±0.0200	0.1273±0.0177	0.119±0.019	0.032±0.005
6	-0.0938±0.0122	0.0959±0.0122	0.108±0.021	0.017±0.006
7	-0.1114±0.0101	0.1081±0.0202	0.109±0.008	0.025±0.005
8	-0.0889±0.0153	0.0921±0.0089	0.094±0.022	0.016±0.004
9	-0.0910±0.0147	0.0930±0.0088	0.100±0.023	0.016±0.005
10	-0.0882±0.0114	0.1236±0.0133	0.112±0.021	0.025±0.007

Table 44: All cost function branches, configuration 8: tail statistics

	$C_2(1)$	$C_2(2)$	$C_2(5)$	$C_2(10)$
1	0.263±0.201	0.170±0.113	0.167±0.112	0.144±0.103
2	0.165±0.122	0.181±0.097	0.183±0.069	0.112±0.114
3	0.226±0.077	0.185±0.144	0.182±0.072	0.153±0.134
4	0.229±0.039	0.377±0.156	0.158±0.022	0.060±0.061
5	0.215±0.049	0.183±0.126	0.239±0.132	0.129±0.068
6	0.223±0.089	0.382±0.146	0.163±0.075	0.056±0.047
7	0.281±0.130	0.201±0.092	0.129±0.041	0.125±0.075
8	0.202±0.059	0.274±0.111	0.153±0.025	0.076±0.061
9	0.225±0.103	0.345±0.134	0.147±0.036	0.057±0.038
10	0.234±0.122	0.124±0.127	0.081±0.110	0.067±0.070

Table 45: All cost function branches, configuration 8: autocorrelation of squared returns

	Mean	SD	Skew	Kurt	Min	Max
1	0.0031±0.0022	0.0436±0.0054	1.78±0.99	13.84±8.77	-0.2355±0.0246	0.4608±0.1127
2	0.0048±0.0024	0.0464±0.0019	1.49±0.32	12.07±3.30	-0.3280±0.0373	0.4298±0.0822
3	0.0053±0.0021	0.0539±0.0068	2.79±2.19	26.89±17.12	-0.2908±0.0862	0.5665±0.2015
4	0.0053±0.0021	0.0480±0.0060	1.11±0.93	10.65±5.34	-0.2670±0.0098	0.3896±0.0861
5	0.0018±0.0005	0.0473±0.0046	1.96±0.74	17.86±5.86	-0.2704±0.0841	0.4804±0.0653
6	0.0058±0.0011	0.0499±0.0056	1.59±1.13	14.66±10.04	-0.2441±0.0525	0.4386±0.1384
7	0.0015±0.0031	0.0395±0.0045	0.42±1.12	13.01±8.88	-0.3361±0.1064	0.3476±0.0294
8	0.0074±0.0030	0.0478±0.0028	1.56±1.04	14.59±8.29	-0.2554±0.0491	0.4145±0.1179
9	0.0054±0.0015	0.0489±0.0043	1.92±1.27	17.18±12.71	-0.2364±0.0456	0.4842±0.1132
10	0.0019±0.0025	0.0378±0.0015	0.96±0.41	8.77±1.86	-0.2511±0.0320	0.3811±0.0371

Table 46: All cost function branches, configuration 9: distributional statistics

A Full Experimental Grid

	q_1	q_{99}	$P_{0.05}$	$P_{0.10}$
1	-0.1085±0.0082	0.1566±0.0316	0.137±0.012	0.040±0.010
2	-0.1112±0.0080	0.1923±0.0232	0.141±0.008	0.044±0.005
3	-0.1165±0.0205	0.2109±0.0280	0.139±0.012	0.048±0.006
4	-0.1317±0.0167	0.1639±0.0209	0.159±0.018	0.052±0.012
5	-0.1252±0.0231	0.1738±0.0294	0.140±0.012	0.039±0.004
6	-0.1269±0.0115	0.1761±0.0355	0.160±0.007	0.055±0.013
7	-0.1010±0.0091	0.1436±0.0240	0.116±0.007	0.031±0.005
8	-0.1193±0.0069	0.1729±0.0400	0.158±0.006	0.046±0.006
9	-0.1150±0.0110	0.1673±0.0311	0.164±0.009	0.049±0.006
10	-0.1026±0.0063	0.1329±0.0138	0.119±0.005	0.031±0.002

Table 47: All cost function branches, configuration 9: tail statistics

	$C_2(1)$	$C_2(2)$	$C_2(5)$	$C_2(10)$
1	0.160±0.084	0.199±0.110	0.115±0.059	0.052±0.042
2	0.265±0.137	0.285±0.066	0.158±0.136	0.162±0.092
3	0.275±0.194	0.179±0.174	0.135±0.110	0.175±0.147
4	0.256±0.147	0.195±0.079	0.111±0.069	0.185±0.048
5	0.255±0.085	0.146±0.096	0.126±0.093	0.164±0.133
6	0.158±0.106	0.145±0.084	0.084±0.065	0.181±0.155
7	0.319±0.150	0.197±0.133	0.157±0.127	0.109±0.079
8	0.175±0.133	0.174±0.117	0.092±0.058	0.165±0.096
9	0.161±0.058	0.118±0.031	0.099±0.073	0.140±0.087
10	0.153±0.073	0.156±0.091	0.067±0.036	0.129±0.078

Table 48: All cost function branches, configuration 9: autocorrelation of squared returns

	Mean	SD	Skew	Kurt	Min	Max
1	0.0025±0.0022	0.0368±0.0022	-0.14±0.61	7.85±3.39	-0.2772±0.0539	0.2850±0.0803
2	0.0011±0.0014	0.0377±0.0027	0.81±0.83	9.59±4.19	-0.2630±0.0938	0.3379±0.0987
3	0.0011±0.0028	0.0412±0.0027	0.42±0.69	9.93±3.53	-0.2672±0.0628	0.3318±0.0460
4	-0.0020±0.0010	0.0331±0.0099	-1.05±1.11	13.54±11.94	-0.2749±0.0770	0.1945±0.0533
5	0.0016±0.0018	0.0411±0.0037	0.51±0.55	11.09±3.87	-0.2612±0.0659	0.3723±0.0814
6	-0.0019±0.0010	0.0361±0.0107	-0.99±0.67	12.00±7.00	-0.2953±0.0787	0.2044±0.0556
7	-0.0331±0.0616	0.0400±0.0221	-0.05±0.57	4.92±4.01	-0.2724±0.0965	0.2176±0.1625
8	0.0000±0.0012	0.0391±0.0100	-0.92±0.67	10.96±6.08	-0.3140±0.0812	0.2150±0.0473
9	-0.0013±0.0018	0.0344±0.0080	-0.99±0.68	10.93±5.51	-0.2733±0.0799	0.1852±0.0636
10	0.0400±0.0880	0.0430±0.0297	0.37±0.69	8.48±8.01	-0.2013±0.1266	0.3286±0.1263

Table 49: All cost function branches, configuration 10: distributional statistics

A Full Experimental Grid

	q_1	q_{99}	$P_{0.05}$	$P_{0.10}$
1	-0.1160±0.0179	0.1131±0.0106	0.112±0.015	0.027±0.005
2	-0.1013±0.0091	0.1316±0.0199	0.111±0.009	0.027±0.006
3	-0.1205±0.0188	0.1406±0.0046	0.124±0.012	0.035±0.004
4	-0.0996±0.0354	0.0863±0.0406	0.092±0.063	0.018±0.018
5	-0.1277±0.0246	0.1327±0.0088	0.120±0.005	0.033±0.008
6	-0.1116±0.0354	0.0964±0.0421	0.110±0.068	0.024±0.020
7	-0.1490±0.1053	0.0719±0.0538	0.253±0.328	0.159±0.310
8	-0.1118±0.0352	0.1028±0.0350	0.132±0.064	0.027±0.022
9	-0.0999±0.0242	0.0918±0.0295	0.104±0.050	0.020±0.015
10	-0.0718±0.0478	0.1562±0.1374	0.259±0.366	0.185±0.369

Table 50: All cost function branches, configuration 10: tail statistics

	$C_2(1)$	$C_2(2)$	$C_2(5)$	$C_2(10)$
1	0.206±0.086	0.310±0.174	0.223±0.147	0.057±0.045
2	0.159±0.061	0.204±0.139	0.147±0.096	0.154±0.153
3	0.339±0.080	0.329±0.121	0.204±0.051	0.147±0.137
4	0.077±0.039	0.322±0.198	0.077±0.059	0.045±0.046
5	0.348±0.100	0.304±0.161	0.201±0.134	0.108±0.054
6	0.086±0.041	0.268±0.130	0.081±0.086	0.043±0.031
7	0.029±0.044	-0.015±0.158	0.046±0.069	0.020±0.036
8	0.108±0.052	0.231±0.132	0.082±0.064	0.064±0.041
9	0.090±0.064	0.229±0.089	0.100±0.098	0.064±0.052
10	0.347±0.244	0.244±0.265	0.082±0.072	0.050±0.076

Table 51: All cost function branches, configuration 10: autocorrelation of squared returns

	Mean	SD	Skew	Kurt	Min	Max
1	-0.0001±0.0022	0.0338±0.0018	0.30±0.36	5.55±2.52	-0.2350±0.0682	0.3091±0.0713
2	-0.0004±0.0024	0.0323±0.0025	0.21±0.26	4.51±1.99	-0.2197±0.0611	0.2698±0.0487
3	-0.0008±0.0022	0.0337±0.0008	-0.35±0.50	5.44±3.19	-0.2654±0.0458	0.2309±0.0202
4	-0.0007±0.0008	0.0240±0.0004	0.15±0.10	0.65±0.21	-0.1263±0.0162	0.1377±0.0094
5	-0.0010±0.0014	0.0337±0.0005	-0.31±0.46	5.10±2.48	-0.2546±0.0382	0.2306±0.0181
6	-0.0006±0.0004	0.0239±0.0005	0.12±0.11	0.62±0.17	-0.1274±0.0212	0.1433±0.0100
7	-0.0017±0.0012	0.0274±0.0053	-0.17±0.32	3.12±2.31	-0.1993±0.0703	0.1962±0.0482
8	-0.0006±0.0005	0.0241±0.0006	0.10±0.14	0.63±0.16	-0.1342±0.0140	0.1382±0.0053
9	-0.0007±0.0007	0.0240±0.0004	0.12±0.10	0.62±0.15	-0.1298±0.0102	0.1382±0.0107
10	-0.0024±0.0011	0.0291±0.0041	-0.14±0.29	2.84±1.74	-0.1958±0.0408	0.1947±0.0453

Table 52: All cost function branches, the input scaling run: distributional statistics

A Full Experimental Grid

	q_1	q_{99}	$P_{0.05}$	$P_{0.10}$
1	-0.0953 ± 0.0119	0.1002 ± 0.0123	0.109 ± 0.018	0.019 ± 0.006
2	-0.0904 ± 0.0096	0.0945 ± 0.0121	0.103 ± 0.024	0.016 ± 0.006
3	-0.1009 ± 0.0113	0.0921 ± 0.0156	0.106 ± 0.016	0.019 ± 0.002
4	-0.0576 ± 0.0036	0.0609 ± 0.0008	0.044 ± 0.005	0.000 ± 0.000
5	-0.1033 ± 0.0135	0.0920 ± 0.0125	0.106 ± 0.018	0.019 ± 0.002
6	-0.0575 ± 0.0039	0.0603 ± 0.0013	0.043 ± 0.005	0.000 ± 0.000
7	-0.0797 ± 0.0217	0.0726 ± 0.0132	0.073 ± 0.029	0.007 ± 0.008
8	-0.0585 ± 0.0042	0.0605 ± 0.0016	0.044 ± 0.006	0.000 ± 0.000
9	-0.0581 ± 0.0030	0.0604 ± 0.0021	0.044 ± 0.004	0.000 ± 0.000
10	-0.0852 ± 0.0188	0.0757 ± 0.0078	0.087 ± 0.024	0.008 ± 0.007

Table 53: All cost function branches, the input scaling run: tail statistics

	$C_2(1)$	$C_2(2)$	$C_2(5)$	$C_2(10)$
1	0.121 ± 0.121	0.133 ± 0.125	0.128 ± 0.135	0.109 ± 0.076
2	0.160 ± 0.149	0.167 ± 0.108	0.125 ± 0.054	0.125 ± 0.104
3	0.162 ± 0.119	0.177 ± 0.123	0.143 ± 0.116	0.110 ± 0.098
4	0.062 ± 0.061	-0.006 ± 0.048	0.008 ± 0.035	0.014 ± 0.045
5	0.177 ± 0.140	0.138 ± 0.105	0.152 ± 0.093	0.097 ± 0.112
6	0.032 ± 0.055	0.005 ± 0.018	0.031 ± 0.079	0.020 ± 0.037
7	0.089 ± 0.091	0.008 ± 0.038	0.059 ± 0.089	0.074 ± 0.038
8	0.087 ± 0.062	-0.005 ± 0.028	0.036 ± 0.054	0.014 ± 0.021
9	0.031 ± 0.041	0.004 ± 0.057	0.039 ± 0.048	0.020 ± 0.049
10	0.034 ± 0.082	0.035 ± 0.068	0.099 ± 0.091	0.064 ± 0.104

Table 54: All cost function branches, the input scaling run: autocorrelation of squared returns

Documentation of AI Tool Usage

Claude (Anthropic) was used as a support tool during this thesis for explanations of financial concepts and feedback on the structure and wording of text drafted by the author. Claude was also used for LaTeX formatting and a review of the code for errors. All output generated by the model was reviewed by the author. The research design, the implementation, the experiments, the results, and their interpretation are the work of the author.

Declaration of Independence

Surname, first name: Galstyan, Yeva
Matriculation number: 1507481
Study programme: Global Software Development (M.Sc.)
Module: Master's Thesis

Hereby, I declare that I have independently prepared the examination performance or thesis in the above-mentioned module - for group work, the portion of the work marked with my contribution - and have not used any sources or aids other than those indicated. Texts, ideas, concepts, graphics, etc., taken directly or indirectly from external sources in my statements have been clearly identified as such and provided with full references to the respective source.

All quotations, as well as all sections written or modified using AI-based tools, have been checked for correctness and marked accordingly. Documentation of the AI tools used, along with their purpose of use, is attached to the work. I take responsibility for the quality and all contents of the present work, and my creative influence predominates.

Furthermore, I assure that I have exclusively used AI-supported tools that the examiner explicitly identified as approved aids. The work has not been submitted in the same or a similar form in any other examination.

Signature of the student